

# 企业级时序因子体系设计与实施总说明

面向因子研究组、时序策略组、因子评估组、平台工程组、模型组和研究负责人。

## 0. 这份文档的定位

这份文档不是一份狭义的需求文档，也不是只服务于工程拆解的 PRD。它的定位是时序因子领域的一份总说明书，目标是把时序因子从原始数据、因子构造、模式识别、自动化挖掘、优化、评估、入库、送模型前处理，到工程化治理与长期演进，尽可能完整地整理到一篇文档里。后续你可以把它发给组员当知识补充材料，也可以把它作为你自己后续设计和判断时序因子方案的母文档。

这份文档的写作标准，不是“讲个大概”，而是尽量做到：

- 逻辑完整
- 术语统一
- 工程边界清楚
- 公式尽量明确
- 方法论尽量全面
- 对推荐方案有明确判断

后续主需求文档和 HTML 里的时序因子评估部分，会从这份文档中再抽取和压缩；但本文件本身会保留尽可能多的解释、设计背景和备选路径。

## 1. 时序因子到底是什么

时序因子不是横截面因子的一个附属分支，也不是简单把横截面因子的计算频率从日频调成分钟频。它在数学本质、交易对象、执行约束、噪声结构、预测目标和工程要求上，都是一套独立体系。

横截面因子主要回答的是“同一时刻、不同标的之间，谁更强、谁更弱”，核心动作是排序、分组、多空对冲和截面标准化。时序因子主要回答的是“同一个标的，在接下来的若干个时间步里更可能向上、向下，还是进入某种状态”，核心动作是自身历史比较、滚动窗口变换、阈值触发、模式识别、状态识别和交易时机控制。

因此，时序因子的输出常常不是简单的截面暴露，而更接近：

- 未来方向偏好
- 未来状态偏好
- 某一模式发生的概率
- 某一 regime 的后验概率
- 某一交易动作是否应该开启或关闭

从资产看，时序因子最适合的地方通常是：

- futures
- crypto
- 高频或分钟级 equity
- options

从频率看，时序因子最常见的是：

- daily
- 1min
- 5min
- 15min
- 更高频的快照级、逐笔级、订单簿级

这份文档尤其关注分钟频及其上游更高频的数据，因为这是最容易出现工程复杂度暴涨、未来函数和摩擦成本失真的区域。

## 2. 时序因子和横截面因子的本质差异

如果团队不能先把这两者区分清楚，后面从预处理到评估再到模型输入都会混乱。

第一，目标不同。横截面因子更像排序器，时序因子更像单资产的状态机或方向预测器。横截面因子常常追求相对表现，时序因子追求绝对方向或状态转换。

第二，标准化方式不同。横截面因子依赖截面上的 `rank`、`zscore`、行业中性化、市值中性化。时序因子不应套用这些横截面模板，必须用滚动标准化、波动率缩放、滞后对齐、序列窗口打包等时序模板。

第三，评估指标不同。横截面因子的 `Rank IC`、分位数收益、截面相关性是主指标。时序因子更要看：

- `TS-IC`
- 方向准确率
- 阈值触发胜率
- 持仓期衰减
- 延迟衰减
- 跨点差成本后的净收益
- 不同时段的安全性

第四，交易物理学不同。横截面日频策略的主要摩擦来自调仓滑点、冲击和融券。分钟频时序策略的生死线更直接落在：

- 买卖价差
- 手续费
- 延迟
- 翻仓频率
- 不同时间段的流动性

第五，数据结构不同。横截面因子的底层数据往往是规则时钟切出的 `bar` 或日频数据。时序因子尤其是分钟频及更高频场景，原始数据往往是：

- 逐笔成交
- 订单簿快照
- 委托流
- 成交量事件
- 成交额事件

因此，时序因子的第一步不是“直接算因子”，而是“先决定用什么采样时钟和什么聚合方式来定义样本单元”。

## 3. 时序因子的真实工业生产链

你刚补充的这条链很重要，它比一般教材里“拿分钟 K 线直接算指标”要真实得多。

目前更符合工业实际的时序因子生产链，可以理解为下面这条路径：

原始数据从逐笔成交级开始，先进入 `phase1`，在这一层做聚合、降频和第一层因子计算。随后和低频特征做 `merge`，再进入 `phase2`，在统一后的同频数据上继续做第二层因子计算，最终得到可评估、可入库、可送模型的时序因子数据。

可以写成一条更清晰的链：

```
tick / trade / order-book raw data
-> sampling & aggregation
-> phase1: aggregation-level factor computation
-> low-frequency feature merge
-> phase2: same-frequency factor computation
-> final factor matrix
```

这条链背后的含义是：

- 原始逐笔数据太细、太噪，不能直接把所有计算都堆在 tick 级
- 第一阶段需要先决定样本时钟和事件聚合方式
- 第一阶段因子更多偏底层微观结构、流量、波动和短窗口统计
- 然后再把低频上下文特征并进来，例如：
  - 日内时段信息
  - 日频风格特征
  - 基本面慢变量
  - 过去更长时间窗口的风险状态
- 第二阶段再在统一同频矩阵上做更高层的时序因子、组合式特征、模式识别特征和 gating 特征

工业上真正的难点，不在于某一个公式，而在于把这条链设计得既能防止未来函数，又能在生产环境中长期稳定运行。

## 4. 样本时钟不是只能用时间 K 线

这一节非常关键，因为很多时序因子从一开始就输在采样方式。

如果原始数据是逐笔成交级，那么你并不一定必须用固定时间切片，例如 1min bar、5min bar。固定时间 bar 的问题在于，它会在低活跃时段过度采样，在高活跃时段又采样不足。对时序因子尤其是分钟级策略来说，这往往会导致：

- 低信息密度样本过多
- 高波动关键阶段被压缩
- 统计量不稳定
- 同一时间长度内包含的市场活动强弱差异极大

因此，时序因子的工业级体系中，样本时钟至少要支持四类：

- 时间时钟 time bars
- 成交笔数时钟 tick bars
- 成交量时钟 volume bars
- 成交额时钟 dollar bars

除了这几类，还可以继续扩展事件驱动的聚合方式，例如：

- imbalance bars
- run bars
- delta bars
- quantile bars
- 你们提到的 realized vol bars

这些非时间聚合的核心思想，是按“市场活动”而不是“钟表时间”来切样本。它们的意义不是为了让图更好看，而是为了让每个样本单元尽量承载更接近同类的信息密度。

## 4.1 Time Bars

这是最常见的方式，例如每 1 分钟聚合一次。

优点：

- 最好理解
- 最容易和现有系统对接
- 跨资产对齐最方便

缺点：

- 高活跃时段信息压缩严重
- 低活跃时段无效样本过多
- 容易在高频场景下混入大量微观结构噪声

## 4.2 Volume Bars

每累计到固定成交量就形成一个新 bar。如果某资产在某段时间成交特别密集，bar 会形成得更快；如果市场安静，形成得更慢。

优点：

- 比固定时间采样更贴近真实交易活动
- 对量价型因子更自然
- 有助于降低样本信息密度不均的问题

缺点：

- 跨资产同步更难
- 参数阈值不好定
- 对极端流动性变化较敏感

## 4.3 Dollar Bars

每累计到固定成交额就形成一个 bar。其直觉是，市场真正“有经济意义的活动量”往往比单纯成交量更重要。

优点：

- 对价格差异大的标的更有可比性
- 对交易强度的经济意义刻画更直接
- 对高价股、低价股、不同合约面值的统一更友好

缺点：

- 在极端波动阶段可能过于敏感
- 同样需要做阈值和稳健性校准

## 4.4 Realized Vol Bars

这是你们已经在尝试的一个很重要方向。它的核心不是按时间、量或金额切样本，而是按“累计实现波动”来切。

直觉上，它等于在问：当市场累计走出了足够多的波动时，再形成一个新样本。这样做的好处是：

- 在低波动时段减少无意义采样
- 在高波动时段更细粒度地跟踪状态变化
- 对波动驱动、突破驱动、状态切换驱动的时序因子更友好

工业上要注意的是，`realized vol bars` 的定义必须严格使用当时已知的历史价格路径，不能把未来波动回流进当前 `bar` 的边界判定。

#### 4.5 事件驱动 bars 的工程建议

如果原始数据是逐笔成交级，我建议把样本聚合能力设计成一个可插拔组件，而不是把某一种 `bar` 写死在主链里。

建议统一抽象成：

- `sampling_clock_type`
- `sampling_threshold`
- `sampling_reset_rule`
- `sampling_alignment_policy`

例如：

- `sampling_clock_type = time_lmin`
- `sampling_clock_type = volume_bar`
- `sampling_clock_type = dollar_bar`
- `sampling_clock_type = realized_vol_bar`

这样后面同一个时序因子挖掘框架，才有机会比较“同一个公式在不同采样时钟下是否仍然成立”。

#### 4.6 样本时钟选择的实务原则

研究团队在面对多种 `bar` 设计时，最容易陷入两种极端。

第一种极端是偷懒，永远只用 `lmin time bar`，因为最容易和现有系统对接。

第二种极端是迷信复杂度，一上来就把所有对象都改成 `imbalance bar` 或 `realized vol bar`，仿佛只要采样方式足够高级，因子就一定会更强。

更成熟的做法应该是把样本时钟视为一个需要验证的假设，而不是一个先验真理。因此，研究员在设计因子时，应同时回答：

- 这个因子到底依赖时间均匀推进，还是依赖市场活动推进
- 这个因子最怕丢掉哪类信息
- 这个因子最容易被哪类噪声污染

例如：

- 如果你关心的是日内固定时间段行为，`time bar` 更自然
- 如果你关心的是量能推进与突破真实性，`volume / dollar bar` 更自然
- 如果你关心的是波动切换和压缩释放，`realized vol bar` 更自然

#### 4.7 样本时钟选择错误会导致什么

如果采样时钟选错，后面的问题并不是“效果差一点”，而可能是整个研究对象被错误定义。

常见后果包括：

- 真实有效的波动事件被平均掉
- 低活跃时段的噪声样本占比过高
- 参数对不同市场、不同品种失去可比性
- 同一个公式在不同资产上出现完全相反的统计特征

所以，在企业级体系里，采样时钟本身也应纳入研究审计对象，而不能只作为底层实现细节。

## 4.8 非时间聚合的阈值应该怎么选

一旦接受了 `volume / dollar / realized vol bar` 这样的非时间聚合方式，团队马上会遇到第二个问题：阈值怎么定。

这是一个典型的“看似是参数问题，实则是对对象定义问题”的环节。阈值过小会导致：

- `bar` 数量过多
- 噪声过强
- 计算负担过大

阈值过大又会导致：

- 真正重要的局部结构被压平
- 模式持续时间看起来过长
- 因子的反应变钝

更稳妥的做法通常不是拍脑袋设一个固定常数，而是从以下几个角度共同校准：

- 平均每日形成多少个 `bar`
- 不同时段 `bar` 数量是否极端失衡
- 不同标的之间是否具备基本可比性
- 主要因子的半衰期是否与该时钟相匹配

例如，某类超短 `OFI` 因子如果理论半衰期只有几次活跃成交，那么把 `volume bar` 的阈值设得太大，本质上等于把这个对象的全部信息压成了慢变量。

所以阈值选择不只是为了“让图看起来平滑”，而是为了让样本单位尽量接近策略真正关心的信息推进单位。

## 4.9 时钟选择最好做成并行 A/B，而不是单一路线押注

研究组织上，一个很常见的问题是团队过早押注某一种时钟，结果后续很多方法都被这一路径锁死。

更好的制度是：

- 统一公式母版
- 多套时钟并行计算
- 统一评估协议比较

示意上可以写成：

```
same_formula
-> time_bar version
-> volume_bar version
-> dollar_bar version
-> realized_vol_bar version
-> compare survival / stability / cost
```

这样做的价值有三点：

- 能看清楚“信息到底来自公式还是来自时钟”
- 能避免团队把偶然对某个时钟友好的对象误当成普适方法
- 能为后续总库字典沉淀出更稳定的时钟模板

企业级体系里，时钟本身就应该和公式一样，成为可复用、可评估、可审计的实验维度。

#### 4.10 时钟选择的最终目标不是高级，而是信息密度与执行可行性的平衡

很多团队讨论样本时钟时，会把问题误解成“谁更先进”。其实更关键的是找到：

- 信息密度
- 噪声密度
- 执行可行性

三者之间的平衡。

例如：

- `realized vol bar` 可能更适合捕捉状态切换
- `volume bar` 可能更适合量能推进类对象
- `time bar` 可能更适合和其他低频系统统一节拍

没有任何一种时钟可以在所有目标下都最优。真正成熟的系统不会把某种时钟神化，而是会明确：

- 它更适合哪类对象
- 不适合哪类对象
- 对后续评估和执行意味着什么

这一点非常重要，因为你们最终做的是企业级体系，而不是单次研究比赛。企业级体系的目标是长期稳定复用，而不是一时追求“更复杂的采样名字”。

### 5. 时序因子的核心对象模型与规则型特例

时序因子文档里最容易混乱的地方，是把 `factor`、`feature`、`signal`、`pattern score`、`regime` 混成一团。这里先统一一下，并特别厘清“连续时序因子”与“规则型时序因子”的关系。

#### 5.1 连续时序因子 (Continuous Factor)

这里的连续 `factor` 指可重复计算、具有明确时间对齐规则、输出一个连续数值的对象（如 `TS-IC` 评估的典型对象，对应评估主线的 B 轨）。它回答的问题是：“下一期收益率的预期值是多少？”

#### 5.2 规则型时序因子 (Rule-Based Time-Series Factor)

这是研究员常问的问题：“我的时序因子是个规则（比如双均线金叉、RSI 超卖），它算什么？”答案是：它仍然是时序因子，但它是时序因子的离散化/事件化特例。

**信号强度的点过程建模：**在企业级框架中，因子的“强度”不能仅依赖离散计数，而应通过严谨的随机过程建模：

- **\*\*自激发点过程 (Hawkes Process)\*\***：捕捉波动聚集效应。条件强度  $\lambda(t) = \mu + \sum_{t_i < t} \phi(t - t_i)$ 。当事件密集发生时，强度急剧上升，形成强趋势信号。
- **\*\*自回归条件持续时间 (ACD)\*\***：通过事件间隔反推强度，适用于刻画中低频均值回归信号。

**多模态融合规则：**将 NLP 新闻情感得分 (Sentiment Score) 与传统量价指标 (如 RSI) 融合。利用大语言模型分配 1-100 的情绪强度，并结合动态权重，克服静态规则在快速变动市场中的滞后性。

它不再输出连续的预测值，而是输出离散状态（如 1 代表做多，-1 代表做空，0 代表空仓）或触发事件。因此，虽然它吃的是时序数据，但它不能用普通的 `TS-IC` 来评估（因为大部分时间是 0，或者只有少数几次切换）。它必须走专属的 **C1 轨（时序状态与逻辑规则轨）**。

#### 5.3 Feature

`feature` 是供后续更高层模型或聚合器消费的特征。一个连续的 `factor` 或一个离散的规则状态，都可以作为 `feature`（如 `Tier 3C` 特征池），送给树模型或神经网络。

## 5.4 Signal

`signal` 更接近最终的交易动作建议。一个 `signal` 往往由多个连续 `factor`、规则过滤器（C1轨产物）联合决定。

## 5.5 Pattern Score & Regime

`pattern score` 是对某种价格行为的打分（如 breakout strength）。`regime` 指潜在市场状态（如 trend, low vol）。它们既可以作为连续因子，也可以通过设定阈值转化为规则型因子（C1 轨）。

# 6. 时序数据工程与预处理：从微观噪音过滤到时间安全

分钟频和逐笔级时序因子真正的基础，不是公式，而是数字信号处理（DSP）、微观噪音过滤与时间安全。

## 6.0 信号工程与连续映射 (Signal Engineering)

**\*\*数字信号处理与抗迟滞 (Anti-Hysteresis)\*\***：传统的简单移动平均 (SMA) 或指数平滑 (EMA) 存在严重的相位延迟（迟滞效应），会导致高频非线性噪音的混叠。前沿预处理应引入：

- **Savitzky-Golay 滤波器**：通过局部多项式最小二乘拟合，在平滑的同时更好地保留信号峰值与特征形态，减少迟滞导致的相位偏移。
- **\*\*离散小波变换 (DWT)\*\***：将时序信号分解为不同尺度的分量，应用软硬阈值去噪 (Soft/Hard Thresholding) 剥离高频细节系数，在不增加滞后的前提下保留阶跃突变 (Step Change) 的纯净化信号。
- **\*\*鲁棒卡尔曼滤波 (Kalman Filter)\*\***：建立线性高斯状态空间模型，通过  $x_k = A x_{k-1} + w_k$  和  $y_k = C x_k + v_k$  递归过滤大单砸盘导致的读数飙升，还原真实的低频 Alpha 状态。

**\*\*微观结构过滤 (LOB Filtration)\*\***：对于基于订单簿 (LOB) 的高频因子（如 OBI），必须剥离高频做市商的闪撤单 (Spoofing/Pinging)。系统应基于 **订单存活时间 (Order Lifetime)**、**更新频率** 与 **更新间距** 实施结构化过滤，剔除存活极短的虚假流动性后再计算因子。

**\*\*带强度的规则因子连续映射 (Squashing)\*\***：严禁将厚尾的微观强度数值进行线性直接映射。必须强制在信号生成与收益评估之间串联非线性有界函数，如双曲正切函数  $\tanh$  或具有平滑下界的 **SERF / Softplus** 变体，将极值软性截断并钳制在安全区间，防止极端长尾事件导致杠杆失控。

必须至少保留以下时间字段：

- `event_ts`
- `knowledge_ts`
- `decision_ts`
- `label_start_ts`
- `label_end_ts`

其中：

- `event_ts` 表示原始事件发生时间
- `knowledge_ts` 表示系统知道这条信息的时间
- `decision_ts` 表示策略允许根据该信息做决策的时间
- `label_end_ts` 表示评估目标的未来区间终点

任何时序因子体系，只要这些时间边界模糊，后面所有结果都可能是假的。

## 6.1 Tick 数据的典型风险

- 撮合延迟
- 时区混乱
- 交易所本地时间与系统时间不一致



- 夜盘跨日
- 缺失成交导致空窗
- 异常成交、撤单、修正单

## 6.2 空 K 线和缺失分钟怎么处理

分钟频里经常会出现某分钟没有成交的情况。工业上最重要的一条原则是：

- 可以 forward fill
- 不可以做会借用未来信息的插值

如果是价格路径的状态延续，可以用上一时刻有效值前向延续；但必须保留：

- is\_forward\_filled
- stale\_length

便于后续评估“因子是否在长时间无交易阶段失真”。

## 6.3 多频 merge 的基本原则

你们的真实链路里有一层 low-frequency feature merge，这一层特别容易出错。

合并规则必须满足：

- 低频特征只能向后广播到它已知之后的高频样本
- 合并时必须保留低频源的 knowledge\_ts
- 不允许把未来日频、未来财报、未来宏观状态提前注入分钟频样本

## 6.4 为什么 merge 特别容易制造伪 alpha

很多分钟频研究看上去非常“聪明”，其实问题都出在 merge 层。

原因是低频慢变量天然带有更强的上下文解释力，例如：

- 日频风险状态
- 宏观 regime
- 前一日尾盘到今晨的结构信息

一旦这类变量的对齐稍有问题，就会产生巨大的样本内解释力提升。研究员往往会误以为：

- “分钟频模型终于把低频上下文学进去了”

但实际上学到的可能只是：

- 未来日频变量的提前泄露
- 未来财报或未来状态标签的回流

因此，merge 层必须被视为时序研究中最敏感的审计点之一。

## 6.5 前向填充为什么既必要又危险

前向填充是分钟频体系无法回避的操作，尤其在：

- 个股低活跃时段
- 某些期货冷门时段
- 部分 crypto 交易所深夜时段

但前向填充的危险在于，它会制造一种“看起来序列非常稳定”的假象。系统必须同时告诉研究员两件事：

- 这一步是必要的，因为没有成交时不能凭空插值

- 这一步又是危险的，因为 stale 值会放大某些伪稳定性

所以最好的工程做法，不是简单 fill 完就结束，而是并行输出：

- filled\_value
- stale\_length
- forward\_fill\_ratio

并在评估阶段单独看：

- 因子是否过度依赖 stale bar

## 6.6 merge 层应该做成什么样的审计日志

如果团队真的想把时间安全落到工程里，就不能只停留在口头原则上。merge 层最好输出可追溯日志，让后续任何评估结果都能反查当时到底合并了什么。

建议至少记录：

- 高频样本主键
- 被 merge 的低频源主键
- 低频源原始 event\_ts
- 低频源 knowledge\_ts
- 实际广播起始时间
- merge 规则版本
- 是否发生前向填充

这样做的好处是：

- 一旦怀疑有泄露，可以直接抽样回放
- 不同研究员不会各自实现一版 merge 逻辑
- 评估层可以按 merge 来源分层看风险

很多组织最后不是输在研究方法，而是输在无法证明“这次合并是时间安全的”。

## 6.7 前向填充需要哪些红线规则

前向填充不是不能用，而是必须配红线。建议至少设置以下制度：

1. 超过某个 stale\_length 后，值继续保留但默认降权或禁用
2. 对关键决策因子，forward\_fill\_ratio 过高时直接打风险标签
3. 对微观结构对象，禁止在长时间无更新下继续假设其仍有同等解释力
4. 评估报告必须展示“去掉 stale 样本后”的对比结果

这背后的逻辑是：

- 前向填充解决的是“没有新观测怎么办”
- 它并不代表“旧观测依然同样有信息”

如果系统把这两件事混为一谈，就会让很多低活跃时段的伪稳定对象被错误放大。

## 6.8 时间安全检查不应只在上线前做一次

时间安全不是一次性验收，而应该是持续审计能力。因为研究代码会变，数据源会变，merge 规则也会变。

所以建议至少建立三层检查：

- 单元级：对 as-of join、lag、window 边界做规则校验
- 研究级：每次新因子评估时自动附带时间安全摘要

- 生产级：定期抽样回放线上特征在历史某时点是否真可得

特别是分钟频和逐笔系统里，一些极细微的实现改动就可能让整个研究口径悄悄变化。例如：

- 之前按 `knowledge_ts` 对齐，后来无意改成按 `event_ts`
- 之前保留旧快照，后来改成使用最近完整快照
- 之前禁止某类未来字段广播，后来某个新表默认带进来了

这些问题如果没有持续审计，往往要到真实收益明显偏离研究回放时才会暴露，而那时排查成本已经非常高。

## 7. 时序因子的分层分类体系

时序因子不应该只分成“趋势”和“反转”两类。企业级体系里，我建议至少分成以下九大家族。

### 7.1 趋势跟随类

典型问题是：价格是否沿某一方向持续推进。

常见构造：

- moving average spread
- breakout continuation
- cumulative return slope
- trend persistence

### 7.2 均值回归类

典型问题是：偏离是否过度，是否有回归均值的短期压力。

常见构造：

- rolling zscore
- deviation from rolling mean
- short-horizon reversal
- liquidity vacuum rebound

### 7.3 波动率与波动聚集类

典型问题是：波动是否扩张、压缩、聚集、切换。

常见构造：

- realized volatility
- range expansion
- intrabar variance ratio
- volatility burst

### 7.4 路径依赖类

仅看起点和终点是不够的，还要看价格是怎么走到那里的。

常见构造：

- path efficiency
- direction-change count
- run length
- drawup / drawdown profile

## 7.5 微观结构类

这是分钟频和更高频时序因子的核心高价值区。

常见构造：

- bid-ask spread pressure
- signed volume imbalance
- order flow imbalance
- trade clustering
- queue pressure

## 7.6 成交与流动性类

常见构造：

- abnormal volume
- dollar flow pressure
- turnover burst
- volume-price coordination
- liquidity drought

## 7.7 模式识别类

它更像一种结构打分，而不是普通线性因子。

常见构造：

- breakout score
- squeeze score
- trend exhaustion score
- reversal pocket score
- opening range behavior

## 7.8 状态识别类

这是 regime detection 的地盘。

常见对象：

- hidden state probability
- transition likelihood
- volatility regime
- liquidity regime
- market stress regime

## 7.9 跨周期与跨模态类

例如：

- 1 分钟与 15 分钟共振
- 高频量价和低频风格共振
- tick-level microstructure 与日频风险状态共同决定的 gating 特征

# 8. 模式识别与状态识别

时序因子体系如果只停留在 rolling mean 和 moving average 上，会非常不够。真正完整的时序体系，一定要把模式识别和状态识别单独拿出来。

## 8.1 模式识别

这里指的是对价格行为形态做结构化表达，而不是简单的传统图形学口号。

建议模式识别至少覆盖：

- 趋势启动
- 趋势延续
- 趋势衰竭
- 震荡压缩
- 震荡后的突破
- 假突破
- 高波动回撤
- 低流动性脉冲

模式识别输出不一定要直接变成交易信号，也可以只是一个 pattern score 。

## 8.2 状态识别

状态识别更关注整个系统处于什么“市场机制”里。

常见状态变量：

- 趋势态
- 震荡态
- 高波动态
- 低波动态
- 流动性紧张态
- 消息冲击态

## 8.3 可选方法

时序状态识别可以使用：

- 规则阈值法
- rolling clustering
- HMM
- change point detection
- switching models

企业级建议是：

- 先用规则法和轻量聚类法打底
- 再把 HMM 或更强的状态切换模型作为增强层

不要一开始就把所有状态识别都重压在复杂模型上，因为状态定义本身就需要大量可解释性和回放能力。

## 8.4 模式识别和状态识别的边界

这是团队讨论时最容易混的地方。一个经验上很有用的区分方式是：

- 模式识别更像“局部结构事件”
- 状态识别更像“全局环境判断”

例如：

- `opening range breakout` 更像模式
- `high_vol_regime` 更像状态
- `trend exhaustion` 更像模式
- `liquidity_stress_regime` 更像状态

模式通常持续时间更短、更局部、更容易和具体交易动作关联；状态通常持续时间更长、更抽象、更适合作为上下文变量。

## 8.5 为什么很多模式对象更适合做 gating

研究里一个很常见的误区是，只要某个模式打分能预测未来，就立刻把它变成直接信号。

更成熟的做法往往是：

- 用它决定“什么时候允许其他因子工作”
- 而不是让它自己独立下单

例如：

- `breakout_authenticity_score` 更适合决定“趋势因子是否允许开仓”
- `liquidity_stress_score` 更适合决定“当前是否应减少交易”

这类对象之所以适合 gating，是因为它们提供的是“条件信息”，而不是完整交易逻辑。

## 8.6 状态数量是不是越多越好

不是。状态划分过多，常常会带来：

- 解释困难
- 样本不足
- 过拟合
- 路由和前端展示过于复杂

实践里更稳妥的办法通常是：

- 先从 2 到 4 个核心状态开始
- 例如 `trend / mean_revert / high_vol / low_vol`
- 再根据研究需要逐步细分

如果团队一开始就做 8 个、10 个甚至更多状态，通常会发现：

- 每个状态看上去都有点道理
- 但没有一个足够稳健

## 8.7 模式对象最终应该产出成什么形态

很多团队讲模式识别时，只讲“识别逻辑”，不讲“产物定义”。这会导致下游工程完全不知道模式对象到底应该以什么形式进入评估、入库或模型端。

企业级上，模式对象至少可以有四种标准产物：

- 二值事件：某个模式在当前是否触发
- 连续分数：某个模式成立的强弱程度
- 置信度：当前识别结果可信不可信
- 生命周期字段：模式开始、持续、衰减、失效的阶段

例如一个 `breakout_authenticity` 对象，不应只输出：

```
breakout = 0/1
```

更完整的做法应输出：

```
breakout_flag_t  
breakout_score_t  
breakout_confidence_t  
breakout_age_t  
breakout_decay_t
```

这样做的好处是：

- 研究员可以区分“有没有”与“强不强”
- 评估层可以检查高分区域是否真的更有效
- 模型端可以把它当成连续特征而不是粗糙开关
- 执行层可以根据模式年龄判断是否已经追晚了

模式识别如果只产出二值标签，往往会损失大量有用信息，也会让后续的分层评估做不细。

## 8.8 状态识别上线时最容易犯的时间错误

状态识别在研究环境里看起来常常很漂亮，但一到线上就失真，最常见的原因不是模型本身不够复杂，而是状态定义偷看了未来。

典型错误包括：

- 用整段未来波动来反标当前是否属于高波动状态
- 用完整回看窗口做聚类，然后把聚类结果直接当作当时可知状态
- 用事后确认的转折点给转折点前的样本贴标签

真正可上线的状态对象必须满足：

- `decision_ts` 时刻已经可以被计算
- 若需要平滑，平滑只使用过去窗口
- 若存在状态切换延迟，报告中要明示这部分延迟损失

一个非常实用的纪律是：

- 研究版状态
- 交易版状态

两者必须分开。研究版可以帮助理解市场结构，交易版则必须是严格可前视实现的对象。很多团队失败，就是拿研究版状态直接进生产。

## 8.9 模式和状态应该怎么评估

模式和状态的评估方法，和普通连续因子的评估方法并不完全一样。它们更像“条件对象”，所以除了收益和相关性，更要看覆盖率和条件增益。

建议至少检查：

- `coverage_rate` ：触发样本占比
- `conditional_return_lift` ：触发条件下相对无条件收益是否提升
- `conditional_ic_lift` ：触发条件下其他因子的 `IC` 是否提升
- `stability_by_session`

- `stability_by_regime`
- `event_precision`
- `event_recall`

其中很重要的一点是：

- 如果一个模式触发后自己赚钱一般
- 但能显著提升其他因子的净收益和回撤控制

那么它依然是高价值对象，只是它的角色不是主信号，而是 gating 或路由器。

这也是为什么模式和状态评估里，不能只盯着自己的独立收益曲线。很多条件型对象的真正价值，在“改变别人的有效工作区间”。

### 8.10 模式与状态对象的工程落地建议

为了让这类对象真正可复用，工程上最好统一一套对象协议。建议至少包含：

- `object_id`
- `object_type`
- `clock_type`
- `update_frequency`
- `lookback_window`
- `decision_delay`
- `value_semantics`
- `recommended_usage`

其中：

- `object_type` 用于区分 `pattern / regime / gate / context_feature`
- `value_semantics` 用于说明这是 `binary / score / state_id / probability`
- `recommended_usage` 用于告诉下游更推荐拿它做主信号、过滤器还是模型辅助特征

如果没有这套协议，后面团队协作时会很混乱：

- 有的人把状态编号当作有序变量
- 有的人把二值事件当作可持续暴露
- 有的人把 gating 对象误当成可直接下单因子

这些问题不是研究问题，而是对象定义不标准的问题。

### 8.11 模式识别和状态识别的常见失败案例

下面这些失败案例在真实团队里非常常见：

- 把开盘前十分钟的高波动统一解释成突破机会，结果只是开盘噪声
- 用过多状态去切分市场，导致每个状态都样本太少
- 把模式识别做成图形描述，却没有统一数值化协议，无法入评估框架
- 把状态识别结果直接做 one-hot 后入模，却没有验证状态本身是否稳定
- 把模式对象做成高频开关，结果造成主策略换手暴增

一个成熟体系的目标，不是“识别出尽可能多的名字”，而是只保留那些：

- 定义清楚
- 时间安全
- 能被评估
- 能被工程复现



- 能在执行层真正使用

的对象。

## 9. 时序因子的构造方法与规则型因子 (Rule-Based Factor) 的转化

这一章回答“时序因子到底怎么造”，并特别强调如何将基础的数学变换组合成离散规则型因子。

### 9.1 Rolling 类变换

最基础也最常见。

例如：

```
rolling_mean_t = (1 / W) * sum_{i=0}^{W-1} x_{t-i}
rolling_std_t = sqrt((1 / (W-1)) * sum_{i=0}^{W-1} (x_{t-i} - mean_t)^2)
zscore_t = (x_t - rolling_mean_t) / rolling_std_t
```

### 9.2 规则型因子构造：阈值触发与逻辑组合 (Thresholds & Logic Gates)

连续的时序指标（如 `zscore_t` 或 `breakout_score_t`）往往噪音很大。通过设定上下轨阈值，可以将其转化为离散的规则型因子（对应 C1 轨评估）：

```
# 均值回归规则：RSI超卖/超买触发
rule_signal_t =
    1 if zscore_t < -2.0 # 超跌做多
    -1 if zscore_t > 2.0 # 超涨做空
    0 otherwise
```

**\*\*逻辑组合 (Logic Gates)\*\***：真正的企业级规则因子极少是单维度的。为了大幅减少伪信号 (False Positives)，它们通常是多个连续因子的布尔逻辑组合 (AND/OR/NOT)。

```
# 逻辑门组合示例：必须同时满足“价格动量向上” AND “波动率收缩” AND “非低流动性时间段”
rule_signal_t = (momentum_t > 0) AND (volatility_t < volatility_threshold) AND (NOT is_opening_auction)
```

### 9.3 规则型因子进阶：状态机与迟滞效应 (State Machine & Hysteresis)

普通阈值触发最大的问题是“闪烁 (Flickering)”：当指标在阈值边缘徘徊时，信号会疯狂地在 0 和 1 之间跳动，导致极高的无效换手率。

工业级做法是引入“持仓记忆”，即**\*\*状态机与不对称迟滞 (Hysteresis Band)\*\***。进入阈值与退出阈值必须拉开差距（形成死区），彻底阻断“乒乓效应 (Ping-Pong Effect)”。

```
# 带有持仓记忆的趋势跟随规则（状态机）
if current_position == 0:
    if MA_short > MA_long + threshold_enter:
        current_position = 1 # 突破上轨，建仓
elif current_position == 1:
    if MA_short < MA_long - threshold_exit:
        current_position = 0 # 跌破极低下轨，平仓（注意：exit 阈值远低于 enter 阈值）
```

**动态阈值 (Dynamic Thresholding) 与非线性截断：** 为了应对市场波动率周期放大导致固定阈值失效的问题，需要采用滚动时间窗口实时计算信号的均值与标准差，实施动态 Z-Score 阈值。此外，可采用\*\*非线性截断 (Asymmetric Thresholding)\*\*，将绝大部分中等强度的信号区定义为“死区 (Dead Zone)”，强制系统只在极端肥尾区域进行狙击，大幅压制平庸交易换手。

```
# 波动率突破规则：不仅要价格突破，还要伴随波动率扩张和成交量放大
vol_expansion_t = realized_vol_short_t > realized_vol_long_t
volume_burst_t  = volume_t > rolling_mean(volume, W) * 2

rule_signal_t =
    1 if (price_t > highest_high(W)) AND vol_expansion_t AND volume_burst_t
    0 otherwise
```

这种多条件逻辑门 (Logic Gate) 是降低规则型因子“假触发率 (False Positive)”的最有效手段。

9.4 动态波动率缩放 (Dynamic Volatility Scaling)

分钟频资产经常有强烈异方差。对于带强度的趋势或突破规则，在市场高波动期极易面临“动量崩溃”(Crash Risk)。企业级应用会强制引入动态缩放机制：赋予信号的实际资金权重应当与其近期的实现波动率成反比。

```
scaled_signal_t = signal_t / max(realized_vol_t, eps)
```

注：在更先进的 RL 框架下，还会采用滚动回归估计时变敏感性 (Time-varying Beta)，将强度信号转化为动态适应当前市场 Regime 的资本配置。

9.5 分数差分

当序列不平稳但又不想完全抹去记忆时，可考虑分数差分。

形式上可以写为：

$$(1 - B)^d X_t = \sum_{k=0}^{\infty} w_k X_{t-k}$$

其中：

$$w_0 = 1$$
$$w_k = -w_{k-1} * (d - k + 1) / k$$

它的意义不是“必须使用”，而是当你需要在保留记忆和追求平稳之间做折中时，它是一种重要工具。

9.6 路径效率

一个简单但很有用的路径类特征可以写为：

$$path\_efficiency\_t = |P_t - P_{t-W}| / \sum_{i=1}^W |P_{t-i+1} - P_{t-i}|$$

越接近 1，说明路径越顺；越低说明绕路越多，更像震荡。

9.7 波动压缩与突破评分

可以用：

```
squeeze_score_t = realized_vol_short_t / realized_vol_long_t
breakout_score_t = (P_t - rolling_high_t) / ATR_t
```

这些都可以进一步与量、成交额、spread 或 order flow 联合。

## 10. Phase1 / Merge / Phase2 的推荐设计

你给的这条真实生产链，应该成为文档里的核心范式之一。

### 10.1 Phase1: 聚合 + 降频 + 第一层因子计算

输入：

- tick 级成交
- 可选订单簿快照
- 可选逐笔成交方向标记

输出：

- 时间 bar
- 非时间 bar
- 第一层微观结构因子

Phase1 适合做的因子包括：

- 基于成交量、成交额、逐笔方向的微观结构因子
- 短窗口 realized volatility
- spread pressure
- signed flow
- trade clustering
- intrabar range

### 10.2 Low-Frequency Feature Merge

这一层把更慢的背景变量并进来，例如：

- 日频风险状态
- 更长窗口波动水平
- 时段特征
- 日内位置特征
- 慢变量基本面或风格变量

要求：

- 只能向后扩散，不得前看
- 必须保留 merge 来源和 knowledge\_ts

### 10.3 Phase2: 同频高层因子计算

在已经统一频率和时间对齐后的矩阵上，继续做：

- 组合式因子
- 模式识别特征
- 状态打分特征
- 高频 gating 特征

- 交易友好化版本的时序因子

简言之，Phase1 承担底层状态感知功能，而 Phase2 侧重于高阶信号表达。

## 11. 自动化挖掘时序因子的方法

时序因子的自动化挖掘不应只复制横截面挖掘方式。

### 11.1 基于拓扑与演化轨迹的因子生态系统 (AlphaPROBE & QuantaAlpha)

在自动化因子挖掘领域，AlphaPROBE 和 QuantaAlpha 代表了旨在解决传统方法“冗余探索”与“逻辑缺失”弊端的最先进架构。

- **AlphaPROBE**：将整个挖掘过程重构为在有向无环图（DAG）上的战略导航，将历史因子视为节点，演化路径视为边。其核心组件“贝叶斯因子检索器”通过评估因子的先验概率和似然估计，在探索与利用之间取得完美平衡。**防高换手机制**：引入深度惩罚（Depth Penalty,  $P_d = \exp(-\lambda_d \times \text{depth})$ ）抑制衍生层级过深的过拟合高频因子，并通过似然项评估新特征对生态的边际抗噪贡献，强制算法探索长效低频算子。同时要求数值、语义、语法三维多样性审查打破高频噪声局部拥挤。
- **QuantaAlpha**：将 LLM 的推理能力与遗传算法的交叉变异深度结合。强制执行假设、数学表达式和可执行代码三者之间的语义一致性。**防高换手机制**：在生成端引入严格的 AST 沙盒（限制代码长度  $\leq 250$ 、特征数  $\leq 6$ ），防止向高频噪声拟合器演变；并在轨迹回溯反馈阶段（重组互补高收益片段），针对导致高换手的失效轨迹，由 LLM 定向包裹降噪算子或调整窗口参数进行修复。

### 11.2 自动化因子发现：基于大语言模型与多智能体框架

在传统量化研究中，规则型因子的挖掘高度依赖研究员的经验和直觉。然而，由于市场环境的不演变，启发式和基于人工规则的静态因子挖掘方法表现出极大的局限性。现代企业级量化基金正在转向基于大语言模型（LLMs）和多智能体（Multi-Agent）系统的全自动化策略生成流水线。

- **AlphaCFG (结构化上下文无关文法)**：为了实现合规可解释的探索，前沿研究引入了 AlphaCFG 框架。该框架利用面向 Alpha 的上下文无关文法（Context-Free Grammar, CFG）构建了一个树状结构、尺寸受控且符合人类金融直觉的搜索空间。寻找高绩效因子的过程被建模为一个超大规模的树状语言马尔可夫决策过程（TSL-MDP），结合蒙特卡洛树搜索（MCTS），该系统在计算效率和交易盈利能力上均能显著超越传统技术。
- **LLM 驱动的风险感知多智能体评估系统**：
  - **市场状态感知智能体 (Market Status Agent)**：分析当前市场的波动率体系和流动性指标，判断处于趋势期还是震荡期。
  - **信心评分智能体 (Confidence Scoring Agent)**：基于因子在不同历史周期中的回溯绩效及其内在经济学逻辑，为其分配动态的信心评分。
  - **动态权重优化智能体 (Weight Optimization Agent)**：根据市场条件的变化，执行动态的资金分配，提供具备强大下行保护的风险调整后收益表现。

### 11.3 换手率惩罚与方差约束下的 RL 奖励塑形

自动化挖掘出的时序因子在实盘中无法生存，大多是因为高换手导致成本反噬。

- **交易成本感知 (TCA) 适应度函数**：摒弃纯预测指标，强制转为执行后收益指标： $\text{Net Return}_t = \text{Gross Return}_t - c \times \text{Turnover}_t$ 。或者引入多目标/显式惩罚函数： $\text{Fitness} = f(\text{Performance}) - \lambda \times \max(0, \text{Turnover} - \text{Threshold})$ 。可要求因子  $\text{ACF}(1) > 0.85$ ，从数学本质引导平滑逻辑。
- **AST 的奥卡姆剃刀约束 (Parsimony Coefficient)**： $\text{Penalized Fitness} = \text{Raw Fitness} - \rho \times \text{Length}(\text{AST})$ 。在算子库中主动限制瞬时绝对差分算子（如基于 1 个分钟 Bar 的 `diff(close, 1)`）的权重，强制包裹时序池化算子（如 `ts_ewma` 或 `ts_mean`）。

- **\*\*方差约束 (QuantFactor-REINFORCE)\*\***：传统 PPO 算法在因子挖掘中往往策略方差过大。采用引入方差边界的 REINFORCE 算法辅以专用基线，能显著降低策略梯度评估的方差，提升生成稳定性。
- **\*\*奖励塑形 (Reward Shaping)\*\***：在长期奖励函数设计中实施强力的“换手率惩罚”，将因子调仓造成的边际变动量转化为模型损失项。迫使智能体放弃捕捉微小噪音、频繁翻转的短视投机因子，转而优先分配资本给长效、高确信度波段信号。

#### 11.4 模式识别驱动挖掘

可以先定义某类模式，例如：

- breakout
- squeeze release
- exhaustion
- liquidity vacuum

再让自动化方法围绕这些模式挖不同特征表达。

## 12. 时序因子的优化与版本管理

时序因子优化和横截面因子优化有重叠，但有几类动作特别重要。

### 12.1 参数级优化

最常见的优化来源：

- 换 lookback
- 换平滑参数
- 换滞后
- 换持有期
- 换阈值

### 12.2 降换手优化

分钟频因子最容易死在这里。

常见动作：

- deadband
- hysteresis
- discrete state mapping
- holding period lock
- signal throttling
- time-of-day gating

例如把原本的：

```
f_t > 0 -> long
f_t < 0 -> short
```

改写成：

```
f_t > u_enter -> long
f_t < d_enter -> short
|f_t| < exit_band -> flat or hold
```

这类迟滞区间对分钟频策略的换手控制非常关键。

### 12.3 降频优化

如果信号有效但太频繁，可以：

- 每 N 个 bar 最多改变一次状态
- 强制最短持有期
- 只在确认触发时更新

### 12.4 去同质化

时序总库里也会有“同一家族因子”的问题，所以必须保留：

- source\_factor\_id
- family\_id
- optimization\_type
- structure\_distance\_to\_parent

## 13. 分钟频高换手问题的系统性解决方案

这是时序因子里最值得单独展开的一章。

高换手不是一个后处理小问题，而是会从挖掘源头一路传导到评估、优化、入库和实盘执行的问题。

### 13.1 从自动化挖掘源头就要控制

在 alphaprobe、quantaalpha 或其他自动化因子矿机阶段，建议就加入以下约束：

- reward 里直接惩罚换手
- reward 里惩罚过于频繁的状态切换
- 不鼓励零轴附近来回翻转的表达式
- 优先搜索有自然迟滞结构或更平滑结构的模板

如果一个生成器从一开始只追逐短期预测精度，而完全不管换手，它最终挖出来的大量因子都会在交易层死亡。

### 13.2 挖出来后要做哪些专门处理

挖掘后可用的典型降换手方法包括：

- 死区阈值 deadband
- 迟滞阈值 hysteresis (双阈值防乒乓效应)
- 离散化为  $[-1, 0, 1]$
- 最短持有期
- 最低再入场间隔
- time-of-day gating
- 波动率过滤 (高波动解禁，低波动强制休眠熔断)
- 流动性过滤
- 仅在信号超阈时触发
- **执行微调器 (Execution Overlay) 降维**：对确实包含高 IC 但换手极高的高频因子，剥夺其独立策略资格。改由低换手因子 (如基本面或低频量价) 决定基础配置和调仓方向，高频因子仅作为战术择时执行层，在日内局部极值出现时分批执行订单 (配合 VWAP/TWAP)，彻底消灭高频独立调仓。
- **\*\*成本感知的凸优化器 (Cost-Aware Convex Optimizer)\*\***：在多因子组合构建时，优化目标不仅最大化预期超额  $\alpha^T w_t$  并惩罚方差，还必须加入交易成本惩罚项 (如 L1 正则  $\lambda ||w_t - w_{t-1}||_1$ )。通过惰性调仓 (Lazy Trading) 迫使优化器进行严苛盈亏比计算。

### 13.3 哪些分钟频因子天然更容易高换手

- 零轴穿越类
- 超短窗口反转类
- 过于敏感的 breakout 类
- 微小 spread 变化驱动类
- 高频噪声放大型微观结构类

对这些家族，系统应默认更严格的换手惩罚和更严格的入库标准。

## 14. 时序因子的评估体系：从 Bar-Level 到 Trade-Level

时序因子的评估一定要把“统计有效”和“可交易”拆开看。尤其是对于\*\*规则型时序因子 (Rule-Based)\*\*，必须完成从单根 K 线 (Bar-Level) 到交易笔数 (Trade-Level) 的视角转换。

### 14.1 预测能力指标 (Bar-Level)

主要针对连续时序因子 (B轨)：

#### TS-IC

```
TS_IC = corr(f_t, r_{t+k})
```

表示单一资产的因子序列与其未来  $k$  期收益序列的相关性。

#### Rank TS-IC & Kendall's Tau

```
Rank_TS_IC = spearman_corr(rank(f_t), rank(r_{t+k}))
Kendall_Tau = kendall_corr(f_t, r_{t+k})
```

适合对抗极端厚尾值，有效验证信号强度增加与未来收益提升之间是否具备真实的单调性。

#### 极端不平衡样本分类评估

如果将带有强度的信号视作对极端行情的预判，由于此类行情发生频率极低，传统的胜率 (Accuracy) 指标会失效。必须采用：

- F1 Score
- AUC-ROC
- AUC-PR (精确率-召回率曲线下面积，确保高强度信号严格控制“假阳性”带来的交易磨损)

### 14.1A 交易级核心指标 (Trade-Level，专为规则型 C1 轨设计)

对于 C 轨 (离散/事件/规则型)，严禁看 TS-IC，必须模拟开平仓序列进行 Trade-Level 评估，其中最核心的诊断工具是 MAE、MFE 与 SQN：

#### MAE (最大不利偏移, Maximum Adverse Excursion)

衡量从一笔交易建仓开始，直到平仓或止损前，资产价格曾向不利方向移动的最大幅度。用于“精确诊断入场质量与止损效率”。若盈利交易也普遍经历了巨大的 MAE，说明策略极度脆弱。

#### MFE (最大有利偏移, Maximum Favorable Excursion) 与 MFE 捕获率

衡量一笔交易生命周期内，价格向有利方向移动的最大幅度。

$$\text{MFE\_Capture\_Ratio} = \text{Exit\_PnL} / \text{MFE}$$

- < 40% : 过早离场 (Cutting Winners Short), 需放宽止盈或引入追踪止损。
- 55% - 75% : 健康运作, 在捕捉动能和防范虚假反转间取得平衡。
- > 90% : 过度拟合警告, 或暗示策略逻辑存在严重的前视数据窥探。

### SQN (系统质量指数, System Quality Number)

统合期望收益、胜率、盈亏比与交易频率的全局打分机制：

$$\text{SQN} = (\text{Expectancy} / \text{StdDev\_of\_R\_Multiples}) * \text{sqrt}(N)$$

- < 1.6 : 不适合交易
- 2.0 - 2.4 : 平均水平
- > 3.0 : 优秀系统 注：针对旨在捕捉长尾极端行情的宏观趋势跟踪因子，其盈亏分布具有强烈正偏态与宽尾，必须引入基于下行半方差 (Semi-variance) 的改良版 SQN，剥离正向利润波动对因子的惩罚。

### 顶级学术视角的单调性评估 (Patton & Timmermann 检验)

时序因子评估中最大的陷阱是“非单调的极端收益现象”。如果只对比最高分位与最低分位 (Top-minus-Bottom) 的收益差，容易掩盖中间分位的非单调反转 (如 U 型分布)。必须采用 Patton & Timmermann (2010) 单调性检验：

- \*\*检验统计量 ( $J_T$ )\*\* :  $J_T = \min_{i=1, \dots, N} (\hat{\mu}_i - \hat{\mu}_{i-1})$
- 聚焦偏离单调性的“最薄弱环节”，通过平稳的块重抽样 (Stationary Block Bootstrap) 计算无偏 p-value。若检验拒绝单调性假设，因子必须重新参数化或进行非线性降维，方可转化为资金权重。

此外，基础 Trade-Level 指标依然必不可少：

- trade\_count : 周期内触发的交易笔数。
- win\_rate : 触发后的胜率。
- profit\_factor : 汇总毛盈利 / 汇总毛亏损。
- payoff\_ratio : 平均单笔盈利 / 平均单笔亏损。
- kelly\_criterion : 凯利公式建议仓位。
- max\_consecutive\_losses : 最大连续亏损笔数。
- time\_in\_market : 策略持有头寸的时间占比。
- rule\_turnover\_penalty : 规则来回翻转导致的换手损耗。

## 14.2 衰减与延迟指标

### Holding Period Decay

对每个 k 计算：

$$\text{Decay}_k = \text{mean}(\text{signaled\_return}_{\{t \rightarrow t+k\}})$$

画出从 T+1 到 T+H 的衰减曲线。

### Latency Degradation

$$\text{LatencyLoss}_d = \text{Sharpe}(\text{delay}=0) - \text{Sharpe}(\text{delay}=d)$$

## 14.3 收益风险指标



## Sharpe

```
Sharpe = (mean(r) - r_f) / std(r)
```

## Sortino

```
Sortino = (mean(r) - r_f) / downside_std(r)
```

## Calmar

```
Calmar = ann_return / |max_drawdown|
```

## Gain-to-Pain Ratio

```
GPR = sum(max(r_t, 0)) / |sum(min(r_t, 0))|
```

## 14.4 执行与摩擦指标 (含 Sim2Real 闭环)

### Turnover

```
turnover_t = 0.5 * sum_i |w_{i,t} - w_{i,t-1}|  
turnover = mean(turnover_t)
```

对于单资产离散状态，也可以写成状态切换频率或绝对持仓变动率。

### Bid-Ask Crossing Cost

若做多必须按 ask 成交，做空必须按 bid 成交，则净收益回测要显式扣除跨点差成本。

### Dynamic Impact Cost (Sim2Real 模拟器与平方根模型)

企业级评估中，静态点差公式往往不足以模拟真实高频滑点。时序评估层应打通执行算法 (Execution Algo/RL) 的真实日志：

- 收集真实的 Slippage 与 Fill Rate。
- 训练一个微观市场冲击模拟器 (Market Impact Simulator)。
- \*\*平方根冲击模型 (Square Root Impact Model)\*\*：量化冲击成本是一个随交易量非线性递减增加的凹函数： $\text{Impact}(q) = \gamma \cdot \sigma \cdot \left(\frac{q}{V}\right)^{0.5}$ ，其中  $q$  是交易股数， $V$  是历史平均交易量。
- **极端压力测试**：利用基于微观结构的多智能体仿真器 (Agent-based Simulators) 重建“闪崩 (Flash Crash)”场景，测试策略在流动性真空下的韧性。
- 用模拟器替代静态冲击扣费，实现基于实盘反馈的动态成本评估，收敛 Sim2Real 缝隙。并基于 TCA 损耗曲线测算绝对资金容量上限 (Capacity Ceiling)。

### WQFitness

可按常见的 WorldQuant 风格口径写为：

```
WQFitness = Sharpe * sqrt(abs>Returns) / max(Turnover, 0.125))
```

它的意义是同时奖励收益和夏普，并惩罚过高换手。

### 14.5 分时段稳定性

必须拆不同时间段评估：

- 开盘
- 午盘
- 尾盘
- 亚洲时段
- 欧美时段

分钟频策略经常只在某几个时段有效，如果不拆时段看，结果会被平均掩盖。

### 14.6 指标的读取顺序

分钟频因子评估里，读取指标的顺序会极大影响研究员的判断。更推荐的顺序是：

1. 先看净口径是否存活
2. 再看延迟和 spread 是否可承受
3. 再看预测指标是否稳定
4. 再看 session / regime 切片
5. 最后才看 family 同质化和总库席位价值

原因在于：

- 如果净口径已经死了，后面一切都只是统计兴趣
- 如果延迟一格就崩，说明执行层无法支撑
- 如果 session 和 regime 不稳定，说明它不是普适对象

### 14.7 为什么净收益必须是默认主口径

很多团队习惯在主报告里先看 gross，再在附录里补 net。这对分钟频来说非常危险。

更好的制度应该是：

- 主视图默认只看 net
- 毛收益只作为研究解释辅助

因为分钟频策略的核心矛盾正是：

- 统计上有一点点优势
- 但成本非常容易把这点优势吞掉

如果默认展示毛收益，会不断强化研究员对伪 alpha 的错觉。

### 14.8 指标之间如何组合解读

下面是一组更接近实务的组合读法：

- TS-IC 为正，Threshold Hit Rate 也不错，但 Latency Degradation 极大
  - 说明信号可能存在，但执行要求过高
- net\_sharpe 不错，但 session\_slice 极不均匀
  - 说明它更像条件性因子，不应当天候总库因子
- Profit Factor 尚可，但 family\_corr 极高
  - 说明它可能只是已有对象的近邻版本

企业级评估的本质，不是看某个指标是否漂亮，而是看一组指标共同讲出来的故事是否一致。

#### 14.9 一份合格的时序评估报告最少要回答什么

一份成熟的分钟频或时序因子评估报告，至少应回答九个问题：

1. 它在净口径下是否存活
2. 它在延迟和 spread 后是否还存活
3. 它的有效区间到底多长
4. 它在哪些 session 有效，哪些 session 失效
5. 它在哪些 regime 有效，哪些 regime 失效
6. 它的收益来自少数极端事件，还是来自大量稳定样本
7. 它与父因子或同类因子的差异到底在哪里
8. 它更适合作为主信号、过滤器还是模型辅助特征
9. 它进入总库后是否真能提供新增席位价值

如果一个报告只回答了：

- Sharpe 漂不漂亮
- IC 好不好看

则该报告尚未达到工程交付标准。能够指导下游团队直接落地的评估报告，必须清晰界定对象的生存边界与使用边界。

#### 14.10 为什么不能只挑最好看的切片汇报

时序研究里非常容易发生“漂亮切片叙事”。例如：

- 只展示午后有效的结果
- 只展示高波动 regime 下的优异表现
- 只展示某一合约品种上最强的一段时间

这些结果不是不能看，而是必须放在全量视角之后再。正确顺序应该是：

- 先看全样本
- 再看切片
- 再判断这个切片价值是否足以支撑“条件入库”

否则很容易出现一种错觉：

- 看切片时觉得这个对象特别强
- 回到整体时才发现覆盖率极低
- 最终它只是在很少时间里表现出彩

企业级研究不反对条件有效，但必须把条件有效的代价说清楚：

- 覆盖率损失多少
- 容量损失多少
- 是否值得为它单独加路由逻辑

#### 14.11 预测指标好看但交易指标差，应该如何解释

这是一类非常关键的对象。很多分钟频研究对象都会落在这个区域：

- TS-IC 为正
- 命中率也不差
- 但净收益不行

这并不等于它完全无价值。更成熟的解释路径是：

1. 先判断它是不是纯执行问题
2. 再判断是不是 horizon 太短
3. 再判断是不是应改成 gating 或模型辅助特征
4. 最后才决定是否淘汰

常见分型包括：

- 预测存在，但 spread 吞没优势
- 预测存在，但延迟一格即消失
- 预测存在，但只能在特定流动性环境下交易
- 预测存在，但独立下单不优，作为组合器输入反而有价值

### 弱单体与强正交信息 (Weak Alpha, Strong Orthogonality)

在现代机器学习驱动的量化工厂中，必须破除“只有高夏普率才是好因子”的线性叠加迷思。许多新闻情绪因子、资金流向因子或宏观状态因子，其单体回测往往夏普率极低（甚至  $\alpha < 0.5\%$ ），但它们包含着极其珍贵的“横向非线性上下文”。例如，一个纯粹的“波动率状态识别”因子本身不提供买卖方向，但它能指导下游树模型“在高波动时赋予动量因子更高的权重”。

**\*\*宽容入库机制与抢救标签 (Salvage Tags)\*\***：当因子未达到主库的独立盈利标准，但证明其具有高条件互信息或高边际 Shapley 交互价值时，系统不应将其丢弃，而应：

- 打上失败原因标签：`failed_standalone_sharpe` / `failed_ic_threshold`。
- 打上抢救保留标签：`retained_for_orthogonality`（高正交信息）、`weak_alpha_strong_context`（弱 Alpha 强上下文）。
- 路由动作：将其宽容入库至 Tier 3C（特征原料库），专供非线性模型或 DRL 智能体提取交叉特征。

因此评估框架不应该只有“保留”或“删除”两个结论，还应支持：

- `promote_to_gate`
- `promote_to_context_feature`
- `maker_only_candidate`
- `research_archive_only`

这能显著减少好对象被误杀，也能减少伪 alpha 被硬推入总库。

### 14.11.2 跨品种与窄横截面的另类因子 (Cross-Asset & Narrow Universe CS)

在拓展到期货、Crypto、期权等非股票品种时，除了量价因子外，同样存在大量非量价（另类/基本面）因子。但其逻辑与股票有本质异同，尤其在“如何评估其截面有效性”上，存在更深层次的工程和数理难题：

#### 1. 宏观与异步数据的“多重时钟漂移与对齐” (Asynchronous Clock Alignment)

- 股票财报通常在盘后发布，对齐逻辑相对简单。但对于跨品种的宏观事件（如美联储议息、CPI发布）或 Crypto 的全天候链上数据，其数据时钟 (Knowledge Time) 与不同资产的交易时钟 (Decision Time) 存在严重的**异步错位**。
- **深层难题**：美股 16:00 收盘，而 CME 股指期货几乎 24 小时交易，Crypto 永不休市。如果用一个统一的“日频截面”去评估宏观新闻情绪因子，会导致在美股截面上发生了严重的“前视偏差（未来函数）”，或在 Crypto 截面上发生了严重的“信息滞后”。
- **解决方案**：严禁使用传统的 `pd.merge` 对齐。必须引入双时态数据库 (Bitemporal DB) 级别的 **Asof Join**，按不同品种的独立 `Market_Calendar` 生成各自的 `Point-in-Time Snapshot`。宏观另类因子的评估，必须是“多资产异构时钟下的连续状态更新”，而非静态切片。

#### 2. 期权因子的“三维波动率曲面” (3D Volatility Surface)

- **深层难题**：股票因子的横截面是一维的（股票 A vs 股票 B）。而期权的另类因子（如隐含偏度突变预期）面对的是一个包含标的 (Underlying)、期限 (Tenor) 和价值状态 (Moneyness) 的\*\*三维曲面截面 (Surface CS)\*\*。
- **评估修正**：期权因子不能简单计算全截面 Rank IC。评估必须强制要求\*\*希腊值中性化 (Greeks-Neutrality)\*\*。例如，如果一个新闻情绪因子只是简单地做多看涨期权，它赚取的只是 Delta 收益（股票涨了期权跟着涨），而非期权特有的 Alpha。评估期权非量价因子时，必须计算其在 Delta-Neutral 和 Vega-Neutral 组合下的超额贡献。

### 3. 期货基本面因子的“期限结构与展期收益” (Term Structure & Roll Yield)

- **深层难题**：农产品仓单、黑色金属库存等期货基本面因子，其作用往往不是决定品种的绝对涨跌，而是决定**期限结构 (Contango vs Backwardation)** 的形状。
- **评估修正**：评估期货基本面/另类因子时，绝不能只看主力合约的单体收益。必须引入 **曲线凸度 (Curve Convexity)** 和 **展期收益捕获率 (Roll Yield Capture)** 等专属指标。此外，由于活跃合约仅几十个，必须采用\*\*板块内相对排序 (Sector-Relative Ranking)\*\*，且因样本极小，IC 计算必须依赖 **Stationary Block Bootstrap** 验证其统计显著性。

### 4. Crypto 社交/链上因子的“碎片化流动性与 Beta 剥离”

- **深层难题**：Crypto 资产的 Beta (与 BTC/ETH 的相关性) 极高，且社交情绪因子极易同涨同跌。同时，链上出块存在 Finality 延迟，且流动性极度碎片化 (CEX 与 DEX 之间)。
- **评估修正**：必须执行严格的\*\*Beta 剥离 (Beta-Neutralization)。**情绪因子在评估前，必须将大盘驱动成分残差化，只评估其横向特异性 (Idiosyncratic) Alpha。同时，在回测评估时必须引入出块延迟惩罚 (Block Finality Penalty)\*\*，防止将“尚未被全网确认的链上异动”提前作为信号输入。**

#### 14.11.3 Crypto 与期货时序因子的专属底层数据陷阱 (Asset-Specific Time-Series Traps)

对于分钟频及更高频的时序研究，除了上述横截面评估差异外，底层特征的预处理更是布满深水炸弹。如果把股票量价的清洗方法直接套给永续合约或期货，回测将充满幻觉：

#### 1. Crypto 永续合约的“资金费率双刃剑” (Funding Rate Trap)

- **陷阱**：永续合约 (Perpetual Swaps) 通过资金费率锚定现货。很多高换手因子回测极佳，实盘却被资金费吃光。另一方面，资金费率本身是一个极佳的“大众情绪反指因子”。
- **工程要求**：严禁对资金费率做全局 Z-score（因为牛熊市基准完全不同）。若将其作为负反馈阻力层，必须实现**逐小时/逐 8 小时的精确序列拼接**；若作为输入特征，必须使用滚动时序 Z-score。

#### 2. Crypto 的“插针”现象与非对称去极值 (Asymmetric Wick Winsorization)

- **陷阱**：Crypto 常发生因单边爆仓引起的连环“插针 (Wicks)”。传统的 MAD 去极值会无差别地把这些带有重要微观结构信息（如流动性真空、多空爆仓热图）的尖峰削平，导致因子失效。
- **工程要求**：严禁盲目削峰。必须采用\*\*非对称截断 (Asymmetric Winsorization)\*\*——对顺势方向的盈利尖峰做温和保护，对逆势的异常噪音做截断；或保留高阶矩（滚动峰度/偏度）作为独立特征输入模型。

#### 3. 期货连续合约的“换月跳空” (Roll-over Gap)

- **陷阱**：期货合约在交割换月时，由于基差存在，新老合约价格会产生硬性跳空。如果不加处理直接算收益率或动量，会产生极其巨大的虚假利润或亏损信号。
- **工程要求**：系统底层必须支持 **Panama Canal Stitching (巴拿马平移拼接)** 来消除绝对价格差（用于计算均线/动量），或 **Ratio Adjustment (比率复权)** 保持收益率对齐。因子提取必须建立在已平滑的连续合约序列上。

#### 4. 期货量价共振中的“持仓量衰减法则” (Open Interest Time-to-Maturity Scaling)

- **陷阱**：研究员常喜欢用“价升量增仓增”作为看多信号。但在期货中，合约临近交割时，资金必然移仓换月，持仓量 (OI) 必然呈现单边下降。这是交易所规则，不是市场看空信号。

- **工程要求**：任何涉及持仓量的时序因子，都必须首先进行 \*\*距离到期日动态缩放 (Time-to-Maturity Scaling)\*\*。必须剥离由交割规则引起的确定性 OI 衰减，提纯出真正的“超额资金博弈净流入量”。

对于这些跨资产的另类/基本面因子，即使单体 IC 极低，只要它们具备强烈的“状态指示”作用（如宏观 Regime 切换、链上资金枯竭），同样适用上述的“**宽容与抢救标签机制 (Salvage Tags)**”，将其作为环境特征 (Context Feature) 宽容入库至 Tier 3C。

## 14.12 时序评估中最容易被误读的几类指标

以下几类指标最容易被误读：

- 高 Sharpe
- 高命中率
- 低回撤
- 高 Profit Factor

它们之所以容易误导，是因为都可能在特定情况下“看起来很好”但本质并不稳。

例如：

- 高 Sharpe 可能来自很低频、很少交易、样本稀疏
- 高命中率可能只是赚小亏大
- 低回撤可能来自长期空仓
- 高 Profit Factor 可能来自少数极端盈利样本

所以这些指标必须结合以下对象一起读：

- 覆盖率
- 换手
- 样本数
- 单笔盈亏分布
- 持有期衰减
- session / regime 切片稳定性

单看某个“好看指标”做推进，是分钟频研究里最容易反复犯的组织性错误。

## 14.13 一套更接近实盘推进的评估结论模板

建议评估报告最终不要只给一个分数，而要给结构化结论。示意如下：

```
结论类型: promote / conditional_promote / reserve / archive
主要角色: core_signal / gate / context / model_feature
生存口径: ideal_survive / taker_fail / maker_only
脆弱来源: delay / spread / session_instability / regime_instability
与父因子关系: incremental / near_duplicate / degraded
建议动作: 入总库 / 入条件库 / 入优化储备库 / 仅保留研究记录
```

这样做的意义非常大，因为它把评估从“看图说话”变成“标准化对象决策”。后续无论是主库评审、总库前端展示，还是模型端消费，都会更容易复用同一套结论。

## 15. 时序稳健性检验：抵御数据挖掘与过拟合陷阱

随着自动化挖掘的普及，巨大的参数空间使得过拟合成为量化投研的致命威胁。必须建立一套涵盖样本外参数稳定性、蒙特卡洛抗噪性和宏观状态感知的稳健性检验框架。

## 15.1 步进式向前优化 (Walk-Forward Analysis, WFA)

传统的样本内/样本外静态切分法无法反映“概念漂移 (Concept Drift)”。WFA 被公认为时序策略稳健性验证的行业黄金标准：

- 将历史切分为多个连续重叠的滑动窗口或锚定窗口。
- 在过去的 IS 数据块搜索最优参数，直接应用于紧接着的 OOS 时间块进行未见数据的真实验证。
- **\*\*参数稳定性聚类 (Cluster Tightness)\*\***：核心不仅是看 OOS 收益，更深层是监测最优参数在时间轴上的跳跃稳定性。若参数发生剧烈跳变，发出最强烈的“曲线拟合”警报。真正稳健的因子，其最优参数在三维图上应呈现高度密集的簇状分布。

## 15.2 蒙特卡洛路径置换与抗噪性隔离

为彻底排除“运气”成分的干扰，必须进行多维蒙特卡洛模拟：

- **\*\*交易序列重采样与洗牌 (Trade Resampling)\*\***：随机打乱历史交易记录生成数万条虚拟资金曲线，重新描绘最大回撤的置信区间。若在 95% 的虚拟路径中回撤远超原始序列，说明策略极度依赖特定历史的“好运气”簇，极易崩盘。
- **\*\*历史数据随机化与噪音注入 (History Randomization)\*\***：对底层 K 线加入高斯白噪声或局部跳跃扰动。若微小噪音导致收益清零，说明规则中包含了极其脆弱的硬编码阈值，绝不具备实盘价值。

## 15.3 市场状态感知 (Regime Filtering) 与危机阿尔法

稳健性评估不能脱离市场状态。时序规则因子（如趋势跟踪）往往在通胀飙升或地缘危机时展现优异的“危机阿尔法”，但在低波温和牛市中陷入反复止损。

- 系统应通过 HMM 或无监督聚类将历史划分为“高波/低波”、“趋势/震荡”等状态。
- 稳健性报告必须明确标识出该因子在哪些特定 Regime 下产生爆发式收益，并在其不适应的 Regime 下能自动降低风险敞口，为后续的动态资本分配提供数据基石。

## 15.5 非量价、极度稀疏与图谱溢出数据的时序处理痛点

当我们目光从量价数据投向非量价（另类）因子时，时序处理面临的将是降维打击级别的挑战。这在 Crypto 链上网络（图谱因子）和宏观经济周期（机制因子）中尤为突出。

### 15.5.1 极度稀疏脉冲因子的 LASSO 重构与覆盖度惩罚

- **痛点**：对于卫星热图、信用卡流水或突发性新闻事件，这类数据在时间轴上是“极度稀疏的 (Sparse)”。在任意一个 30 分钟的窗口内，可能只有 1% 的标的有信号。如果直接应用传统的 OLS 时序回归，模型会迅速陷入过拟合的深渊。
- **解决方案**：
  - **LASSO 正则化重构**：必须引入带有  $L_1$  正则化的 LASSO，通过 10-fold CV 动态确定惩罚参数  $\lambda$ 。LASSO 能够毫不留情地将弱预测因子的系数压缩为绝对的零，从而敏锐捕获存活期极短的脉冲信号。
  - **\*\*覆盖度惩罚 (Coverage Penalty)\*\***：由于另类数据往往无法实现 100% 的覆盖，评估时必须引入惩罚参数  $\alpha / \sqrt{\beta}$ 。最终时序评定因子的不是单纯的准确率，而是 **有效准确率 = 预测准确率  $\times$  覆盖率**，严防因罕见偏误而导致的系统性估值误判。

### 15.5.2 网络图谱的时序溢出：Crypto 链上交互与供应链

- **痛点**：传统的时序回测假设每个标的（节点）是独立同分布 (IID) 的。但在 Crypto 链上转账网络或传统供应链中，一个节点的崩盘（如某大户爆仓、某核心供应商停产）会沿着图谱网络产生强烈的**\*\*时空溢出效应 (Momentum Spillover)\*\***。
- **解决方案**：
  - 必须从线性欧式空间跃升至非欧几何图谱，使用 **Graph IC (Network IC)** 代替传统 TS-IC，整合图的拉普拉斯矩阵，评估信号沿着网络枢纽节点传导的衰减。

- 引入 \*\*属性驱动图注意力网络 (AD-GAT)\*\*。结合 RNN 时间序列分析，对源公司的属性矩阵和邻接网络矩阵进行元素级别的张量融合 (Tensor Fusion)。模型可以动态判定某一股价跳水信号，是否应该在接下来的几个时序 Bar 中在网络里发生真实的势能溢出。

### 15.5.3 宏观大周期的非线性降维：马尔可夫机制转换 (MSM)

- **痛点：**传统回测隐含着—一个危险的假设——因子的风险溢价在全样本历史中是一个常数。但在时序维度上，流动性充裕期的优异因子，在信用枯竭期可能产生巨大的负向拖累（符号反转）。
- **解决方案：**
  - 必须引入 \*\*马尔可夫机制转换模型 (Markov-Switching Models, MSM)\*\*。不依赖全样本静态方差，而是利用 VIX 波动率、泰德利差 (TED rate) 等宏观先导指标，通过时变转移概率 (TVTP) 预测状态切换。
  - 评估输出不再是绝对收益，而是 \*\*条件期望与条件 IC (Conditional IC)\*\*。量化投资的核心演变为：利用宏观变量的先导变化，在“机制边缘”进行因子权重的动态调度。

## 16. 下游消费端对接与特征工程前置约束

虽然本规范侧重于因子本身的评估与入库，但因因子组必须明确“交付给下游系统（如机器学习预测、组合优化器）的数据应该长什么样”。**评估通过并不等于直接塞给模型。**

### 16.1 特征专属预处理 (Model-Specific Preprocessing)

因子的“纯度”对于不同消费方意义截然不同：

- **树模型阵营：**必须向其交付未被中性化、未被极度缩放的 `raw_exposure`。树模型需要利用原始特征值之间的非线性关系来进行分裂。
- **神经网络阵营：**必须交付严格去极值和标准化的 `winsorized_exposure`，且标准化 Scaler 必须严格隔离训练集与测试集。
- **多因子线性优化器：**必须交付剥离了已知风险因子的 `residual_exposure`，以防止共线性摧毁优化权重。

### 16.2 时序特征的全局对齐与兜底

在将不同频率、不同发布周期的时序因子组合成宽表时：

- 必须强制使用基于 `knowledge_ts` 的 `ASOF JOIN`，彻底斩断未来函数。
- 对于因子断更或稀疏缺失，必须在交付前实施全局兜底策略（如填 0 或极端标识值），严禁让下游处理含有大面积 NaN 的不合规矩阵。

### 16.3 避免重复评估的边界

一旦因子交付至特征装配环节 (Feature Assembly)，因子团队的任务即告完结。后续下游是否使用复杂的网络结构去融合这些特征，不再属于因子组的“单因子评估”职责范畴。

## 17. 工程化架构建议

### 17.1 库分层

时序因子也应走完整的分层库：

- Tier 0
- Tier 1
- Tier 2
- Tier 2X
- Tier 3A / 3B / 3C / 3D



- Tier 4

## 17.2 总库坐标

时序因子必须独立于横截面总库，至少保留：

- `signal_structure = time_series`
- `asset_class`
- `frequency_bucket`
- `domain_root`

## 17.3 产物面

建议至少保留：

- 评估报告
- 衰减图
- 延迟损失图
- 时段切片图
- spread / cost 诊断
- 状态切换图

# 18. 常见错误与误区

## 18.1 把横截面模板直接套到时序上

这是最常见的错误。

## 18.2 只看预测，不看交易成本

分钟频里这是致命错误。

## 18.3 只看时间 bar，不考虑非时间聚合

这会让许多活动驱动型因子在源头就损失信息质量。

## 18.4 把模式识别和交易信号混为一谈

很多模式分数适合作为 `feature` 或 `gating`，并不一定适合直接下单。

## 18.5 大量近邻版本都入总库

会污染总库并夸大虚假多样性。

# 19. 最终推荐的企业级时序因子方案

如果让我来给出一个最终推荐版本，我会这样定：

第一，时序因子体系必须独立成文档、独立成总库、独立成评估模板，不能当作横截面体系的附注。

第二，原始生产链建议以逐笔数据为起点，明确采用：

- `Phase1 = sampling + aggregation + first-layer factors`
- `low-frequency merge`
- `Phase2 = same-frequency high-layer factors`

第三，采样时钟不应只支持时间 bar，而应至少支持：

- time bars
- volume bars
- dollar bars
- realized vol bars

第四，分钟频时序因子的工业核心不是“找更花的公式”，而是：

- 控换手
- 控延迟敏感性
- 控跨点差后净收益
- 控不同时段失效

第五，自动化挖掘时必须从源头惩罚高换手，不应等挖出来后再被动修补。

第六，主评估体系必须把统计预测能力、收益风险、执行摩擦和稳定性并列，而不是只看 IC 。

第七，时序因子和模式识别、状态识别应统一纳入一个体系，但对对象边界要清楚。

## 20. 后续与主需求文档的衔接

后续要回填到主需求文档和 HTML 的重点内容，建议从本文件中抽以下几块：

- 时序总库定义
- Phase1 / merge / Phase2 真实生产链
- 非时间聚合采样
- 时序专属预处理模板
- 时序专属评估指标表
- 高频高换手的优化动作
- 时序详情页与图表产物

## 21. 核心总结

时序因子不是横截面体系的高频版，它是一套从样本时钟、数据工程、因子构造、模式识别、自动化挖掘、换手治理、评估体系、入模前处理到工程架构都必须独立思考的完整生产系统。真正企业级的时序因子方案，核心不只是“不能找到一些有效公式”，而是能不能在高噪声、高摩擦、高漂移的真实市场里，把这些因子长期稳定地构造成、评估掉、筛选掉、优化掉，并最终留下少量真正可交易、可复现、可治理的时序资产。

## 22. 时序因子分类总表

前文已经给出了时序因子的九大家族，但如果这份文档要承担“团队知识手册”的作用，就必须再给出更结构化的分类总表。下面这张表的目标不是穷举所有因子，而是给出一个系统性坐标，帮助研究员、工程师和评估人员在同一个框架下讨论问题。

| 家族   | 典型预测目标 | 常见输入               | 常见变换                  | 典型风险        | 更适合的资产 / 频率                                    |
|------|--------|--------------------|-----------------------|-------------|------------------------------------------------|
| 趋势跟随 | 绝对方向延续 | 价格、收益率、均线、波动率      | MA spread、突破、斜率、回撤后延续 | 滞后、假突破、延迟敏感 | futures<br>1min/5min、crypto<br>1min            |
| 均值回归 | 短期反向修复 | 偏离度、极值、成交密度、spread | zscore、偏离度、短窗反转       | 高频噪声、来回翻仓   | equity intraday、<br>futures micro<br>reversion |

|            |                 |                                     |                                            |               |                              |
|------------|-----------------|-------------------------------------|--------------------------------------------|---------------|------------------------------|
| 波动率类       | 波动扩张/收缩         | 高低价、逐笔波动、成交加速                       | realized vol、ATR、波动比率                      | 过于依赖极端阶段      | crypto、options、高波期货          |
| 路径依赖类      | 路径顺滑/扭曲         | 连续价格路径、run、回撤路径                     | 路径效率、方向切换次数、run length                     | 对窗口极敏感        | 多市场通用                        |
| 微观结构类      | 买卖力量、盘口失衡       | LOB、逐笔方向、spread、queue               | OFI、signed volume、queue pressure           | 数据质量要求极高      | 高频 futures / equity / crypto |
| 流动性类       | 资金推进与拥塞         | 量、额、成交簇、无成交区间                       | volume burst、dollar flow、liquidity drought | 在不同交易所差异大     | crypto、中高频期货                 |
| 模式识别类      | 结构事件发生概率        | OHLCV、非时间 bars、波动、流量                | breakout、squeeze、exhaustion、opening range  | 规则过拟合         | 1min/5min 场景尤其重要             |
| 状态识别类      | 当前 regime 或切换概率 | returns、vol、volume、flow、macro state | HMM、change point、聚类、规则 gating              | 定义不稳、可解释性不足   | futures、crypto、跨市场           |
| 跨周期 / 跨模态类 | 多频共振、上下文 gating | 高频价量 + 低频风险状态                       | merge、stack、multi-scale transform          | merge 泄露、同步错误 | 企业级多策略体系                     |

### 22.1 这张总表怎么用

如果团队把这张总表当成“分类标签”就太浅了。更好的用法是：

- 挖掘阶段：先明确准备探索哪一类家族，而不是让生成器无目标乱搜
- 评估阶段：不同家族默认加载不同的模板和风险提示
- 优化阶段：趋势类重点控延迟与衰减，均值回归类重点控换手与微噪音
- 入库阶段：同一家族内部更容易同质化，应更严格去重和家族限额
- 模型端：模式识别类和状态识别类经常更适合作为 gating 特征，而非直接信号

### 22.2 企业级团队为什么必须做分类

一支成熟的团队如果没有时序因子分类体系，就会出现三个问题。

第一，研究没有边界。每个人都在说“我有一个分钟频因子”，但并不知道这个因子究竟属于趋势、反转、模式识别还是状态识别。

第二，评估模板无法统一。趋势类和微观结构类的衰减速度、延迟敏感性、成本结构根本不同，不能共享同一套判断标准。

第三，模型端会误采样。很多模式识别对象更适合作为“事件触发条件”，如果把它们当成连续信号直接吃进模型，很容易得到非常不稳定的结果。

## 23. 采样、聚合与 Bar 设计总表

原始数据是逐笔成交级时，真正决定上限的，往往不是你后面用了什么模型，而是你最开始选择了什么采样时钟。为了让这部分可以直接拿去给组员讲，我把常见 bar 体系再系统化成一张表。

| Bar 类型           | 定义                                   | 最适合的问题    | 优点            | 风险点            | 建议用途                   |
|------------------|--------------------------------------|-----------|---------------|----------------|------------------------|
| time bar         | 固定时间切样本                              | 基础分钟级策略   | 实现简单、跨资产对齐方便  | 高活跃压缩，低活跃过采样   | 基础版和对齐主线               |
| tick bar         | 固定笔数形成 bar                           | 看逐笔行为密度   | 对交易活跃度更敏感     | 忽略单笔金额大小差异     | 逐笔行为研究                 |
| volume bar       | 固定累计成交量形成 bar                        | 量驱动型策略    | 更贴近实际活动量      | 大小盘、不同合约间阈值难统一 | 量价与微结构研究               |
| dollar bar       | 固定累计成交额形成 bar                        | 经济活动强度    | 对价格和数量统一更自然   | 极端行情会很密集       | 多资产统一采样                |
| imbalance bar    | 累计不平衡达到阈值形成 bar                      | 买卖不平衡驱动策略 | 更能捕捉方向性流量变化   | 定义和参数校准复杂      | OFI、签名流量研究             |
| run bar          | 连续方向 run 达阈值形成 bar                   | 追踪持续性动量   | 对趋势 burst 更敏感 | 容易过拟合 run 长度   | 趋势 / momentum burst    |
| delta bar        | 累计 signed volume 或 signed dollar 达阈值 | 买卖压力量化    | 更贴近订单方向信息     | 方向标记误差会放大      | 流量推进类因子                |
| quantile bar     | 根据历史分位阈值自适应切样本                       | 活跃度动态变化大  | 参数自适应         | 稳定性与解释性较差      | 高动态市场实验层               |
| realized vol bar | 累计实现波动达到阈值                           | 波动驱动、状态切换 | 对关键波段更敏感      | 边界定义容易泄露       | 波动 / regime / breakout |

23.1 为什么非时间聚合值得认真做

你们已经在做非时间聚合，这是一个非常正确的方向。它的本质不是“换一种 K 线画法”，而是承认市场并不是按钟表均匀提供信息。市场可能在 30 秒内发生一天最重要的价格发现，也可能在 20 分钟内几乎没有有效信息。

固定时间采样的问题，在分钟频上尤其明显：

- 低波动横盘阶段会产生大量几乎无信息的样本
- 高波动突破阶段却可能只有寥寥几根 bar
- 许多短生命周期 alpha 会被错误地平均化

非时间聚合的价值就在于：

- 让样本更接近“信息事件”而非“物理时钟”
- 让同一模型看到更稳定的样本密度

- 降低训练集中由无效时段主导的噪声

## 23.2 聚合阈值怎么定

这是工程上最容易失控的部分。阈值太低，会过度采样；阈值太高，会压缩关键事件。建议至少同时保留三层阈值策略：

- 静态阈值：便于基准实验和长期对比
- 训练样本内自适应阈值：适合动态市场
- 线上保守阈值：避免实盘中 bar 数暴增或暴减

建议每个 bar 配置至少保留：

- bar\_type
- threshold\_value
- threshold\_update\_rule
- expected\_bar\_count\_per\_day
- fallback\_policy

## 23.3 什么时候不建议上复杂 bars

并不是所有任务都需要 imbalance bar 或 realized vol bar。如果团队当前连：

- 时间对齐
- PIT (Point-in-Time, 防穿越时间)
- forward fill
- 基本回测成本扣除

都还没有完全收敛，那先用 lmin time bars 把主流程打通更稳。复杂 bars 更适合作为增强层，而不是团队的第一天入口。

# 24. 时序预处理细则 (Advanced Time-Series Preprocessing)

时序因子的一大难点，是大家很容易把“构造公式”和“预处理步骤”混在一起。实际上，很多时序因子是否可靠，并不主要取决于那个核心公式，而是取决于它前后的预处理是否合理。在企业级量化工厂中，时序预处理必须引入最高标准的防泄漏和记忆保留机制。

## 24.1 核心高级预处理概念补全

结合 AI 生成的量化因子预处理全矩阵方案，时序预处理必须在以下几个维度达到工业标准：

- **双时态数据模型 (Bitemporal) 与 ASOF JOIN**：在时序拼接中，最致命的是前视偏差 (Look-ahead Bias)。底层数据必须同时记录 Valid Time (业务发生时间) 和 Transaction Time / knowledge\_ts (系统实际获取时间)。在拼表时，严禁使用常规的外连接，必须使用 **ASOF JOIN** (匹配最接近且绝对不超过当前时间戳的记录)，从物理层面阻断未来数据渗透。
- **\*\*时序平稳化：分数阶差分 (Fractional Differentiation)\*\***：为了满足机器学习模型对平稳性的要求，传统的“一阶差分”会彻底抹杀长程记忆 (过度差分)。必须引入分数阶差分技术 (允许差分阶数  $d$  在 0 到 1 之间)，将序列差分到恰好满足 ADF 平稳性检验的临界点，最大化保留 Alpha 记忆。
- **\*\*离散规则因子的衰减 (Signal Decay)\*\***：规则型因子触发后，其 Alpha 预测能力会随时间被套利抹平。必须对脉冲信号引入 **指数衰减 (Exponential Decay)** 或 **\*\*双曲线衰减 (Hyperbolic Decay)\*\***，将其转化为时间相依的连续变量，帮助模型理解事件的“新鲜程度”。
- **\*\*对称正交化 (Symmetric Orthogonalization)\*\***：在剥离时序因子的已知风险暴露时，若存在大量高度共线性的基础特征，传统的 Gram-Schmidt 逐步正交化会因特征顺序不同而导致信息丢失。必须引入基于特征值分解的对称正交化算法，平等对待所有因子并最小化信息损耗。

- **\*\*组合净化交叉验证 (CPCV, Combinatorial Purged Cross-Validation)\*\***：针对具有序列自相关特性的时序特征，常规的 K-Fold 交叉验证会导致严重的序列信息渗透。必须实装 CPCV：包含 **\*\*Purging (净化)\*\***，即剔除与测试集发生标签重叠的训练样本；以及 **\*\*Embargoing (隔离)\*\***，即在测试集之后插入一段空白期，强行斩断序列记忆的自相关连结。

## 24.2 输入清洗

在时序因子场景中，输入清洗至少包括：

- 异常价格点过滤
- 重复 tick 清理
- 无效成交和修正单处理
- 跨日、夜盘、节假日断裂处理
- 不同市场时间带统一

如果是订单簿类数据，还要额外考虑：

- 盘口深度缺失
- 队列更新不同步
- 中间价、最优价、成交价不一致

## 24.2 滚动标准化

时序因子最大的禁忌之一，就是用全样本均值和全样本方差去做标准化。因为这样做相当于把未来的统计信息回流进当前样本。

正确做法应该是：

- 用历史滚动窗口
- 或只在训练窗内拟合再复用

例如：

```
z_t = (x_t - mean(x_{t-W:t-1})) / std(x_{t-W:t-1})
```

这里的核心，不是公式本身，而是 `t` 时刻不允许看见 `t+1` 之后的任何信息。

## 24.3 Rolling MAD

对分钟频数据来说，极端值很多，直接用标准差未必稳。MAD 是更鲁棒的替代：

```
MAD_t = median(|x_{t-i} - median_window|), i = 1...W
robust_z_t = (x_t - median_window) / max(MAD_t, eps)
```

## 24.4 波动率缩放

如果不做波动率缩放，很多分钟频因子在高波动时段会被误以为信号强，实际只是市场本身变得更吵。

常见做法：

```
vol_scaled_t = raw_signal_t / max(realized_vol_t, eps)
```

也可以用：

- ATR

- rolling std
- bpower variation
- realized kernel 的近似指标

## 24.5 死区与迟滞作为预处理还是后处理

这是实务里非常值得说明的一点。很多团队把 `deadband` 和 `hysteresis` 只当成“交易执行层的小技巧”，其实它们经常应该被视为时序信号工程的一部分。

例如：

```
if state == flat and signal_t > enter_up:
    state = long
elif state == long and signal_t < exit_up:
    state = flat
```

它本质上是在给信号加入状态记忆和抗抖动约束。对分钟频策略来说，这往往比“把原公式再调得更花”更有效。

## 24.6 时间段过滤

有些因子不是全天都有效。工业上建议把以下时段特征做成显式上下文变量：

- 开盘前若干分钟
- 开盘冲击期
- 午盘低流动性时段
- 收盘再平衡时段
- 夜盘开盘段
- 宏观数据发布时间段

这些变量既可以用作：

- 因子过滤条件
- 模式识别上下文
- 风险控制 gating

## 24.7 分数差分与平稳化的取舍

分数差分不是时序因子体系里的必选项，但它之所以重要，是因为它代表一种非常典型的取舍思路：尽量让数据更平稳，同时尽量保留原始记忆。

如果团队准备把时序因子送入某些对平稳性更敏感的模型，分数差分是一个值得保留的工具层，但我不建议一上来把所有因子都统一做分数差分。更好的方式是：

- 先做诊断
- 对确实存在明显漂移或趋势污染的特征单独处理
- 保留原始版与差分版并行评估

# 25. 模式识别方法库

模式识别这一章我要再拉得更细一点，因为它是时序体系和普通因子体系差异最大的地方之一。

## 25.1 趋势启动模式

趋势启动不是简单的“均线金叉”，而更像是多个条件共同表明一段新的方向性行为正在形成。常见信号包括：

- 波动率从压缩转向扩张

- 成交加速
- 突破过去窗口高点 / 低点
- 微观结构上买方或卖方压力持续推进
- pullback 后重新站回趋势方向

可以抽象成：

```
trend_start_score_t = w1 * breakout_score_t
                    + w2 * volume_acceleration_t
                    + w3 * flow_imbalance_t
                    - w4 * reversal_pressure_t
```

## 25.2 假突破识别

很多分钟频策略死在这里。真正强的时序体系，不只是会找突破，还要能识别“哪些突破非常像假动作”。

假突破常见迹象：

- 突破时量能不足
- 突破后无法维持在阈值之上
- spread 异常放大但成交推进并不强
- 随后立刻回到原区间

## 25.3 波动压缩后释放

这是最典型的一类模式识别对象。

关键特征：

- 近期 realized vol 下降
- range 收缩
- 成交密度下降
- 随后一旦放量突破，走势容易加速

常见表达：

```
squeeze_score_t = vol_short_t / vol_long_t
release_score_t = breakout_score_t * volume_ratio_t
```

## 25.4 趋势衰竭

很多趋势因子不懂何时退出。趋势衰竭类模式的价值，往往不在于单独做交易，而在于给趋势因子加退出条件。

常见迹象：

- 创新高但量价配合转弱
- 波动放大但净推进减弱
- 路径效率下降
- order flow 不再支持原方向

## 25.5 开盘区间模式

尤其对分钟频 equity 和部分 futures 很重要。

可研究对象包括：



- opening range breakout
- 开盘后回踩确认
- gap 延续还是回补
- 开盘前若干分钟价格发现是否有效

## 25.6 模式识别对象的工程归属

很多团队在这里最容易犯的错误是：一旦识别出某个模式，就立刻把它当成“最终交易信号”。更成熟的做法是把模式识别对象分成三类：

- 直接信号类
- 过滤器类
- 上下文特征类

例如 `squeeze_score` 很多时候更适合作为过滤器，而不是直接多空方向信号。

## 26. 状态识别与 Regime 检测展开

前文只是简单提到 HMM、change point 和规则法。这里再展开，因为它在时序体系里经常是“提升上限”的关键。

### 26.1 为什么需要 regime

一个在趋势市里很强的时序因子，到了震荡市可能完全反着来；一个在高波市里有用的 spread 因子，到了低波市里可能只剩噪音。如果团队没有 regime 视角，就会出现：

- 因子整体平均看还行，但实盘一段时间完全失效
- 模型端把不同状态下相互冲突的样本混在一起学
- 优化器不断为了局部阶段去调参数，越调越过拟合

### 26.2 规则型 regime

最容易落地，也最容易解释。

例如：

- `high_vol`：短窗波动大于长窗波动某阈值
- `trend`：均线斜率和路径效率同时较高
- `liquidity_stress`：spread 扩大且成交稀疏

优点：

- 可解释
- 便于复现
- 工程实现最简单

缺点：

- 容易僵化
- 对边界条件较敏感

### 26.3 HMM

HMM 的价值在于，它允许我们认为市场状态是“隐藏的”，只能通过收益、波动、成交等观测量去反推。

最基础的思路是：

- 输入若干观测特征，如 returns、realized vol、volume ratio、range
- 学出若干隐状态

- 输出每个时刻属于这些状态的后验概率

企业级上更推荐把 HMM 当作：

- 状态特征生成器
- 策略 gating 层
- 解释市场状态迁移的工具

而不是直接让它一把梭成为交易策略本身。

## 26.4 Change Point Detection

这一类方法更关注“什么时候机制发生了突变”，而不一定先假设固定状态数。

适用场景：

- 波动结构突然切换
- 微观结构噪声模式改变
- 流动性断层
- 新闻冲击前后结构变化

## 26.5 状态识别的输出怎么用

状态识别输出最好不要只有一个硬标签。建议至少同时保留：

- regime\_label
- regime\_prob\_vector
- regime\_confidence
- transition\_risk\_score

这样模型端、评估端、优化端和前端详情页都能用。

# 27. 自动化挖掘的完整方法论

这一章把自动化挖掘再系统化，不让它停留在“LLM、GP、人工模板”三个词上。

## 27.1 搜索空间设计

自动化挖掘成败的第一步，在于搜索空间怎么定义。

搜索空间建议至少分四层：

- 原子输入层：价格、收益、量、额、spread、imbalance、状态变量
- 基础变换层：rolling、lag、diff、ratio、rank within window、threshold
- 组合结构层：加减乘除、门控、条件分支
- 交易友好层：平滑、迟滞、降频、离散化

如果一开始就让生成器直接在无限表达式空间里乱走，绝大多数结果要么不可解释，要么高换手，要么根本算不动。

## 27.2 Reward 设计

对于分钟频时序因子，reward 绝对不能只看预测准确率。建议 reward 至少是多目标的：

```
Reward = a * predictive_score
        + b * net_return_score
        + c * stability_score
        - d * turnover_penalty
```

```
- e * latency_penalty
- f * complexity_penalty
- g * correlation_penalty
```

这比后面再补救高换手，要有效得多。

27.3 LLM 在时序因子上的最优角色

我更倾向于把 LLM 定位成：

- 结构变体生成器
- 参数搜索建议器
- 模式识别逻辑重写器
- 失败归因总结器

不建议让它直接毫无限制地端到端输出“可实盘”的分钟频信号。

27.4 GP 在时序因子上的价值

GP 的优势在于：

- 算子空间可控
- 血缘天然清晰
- 适合做结构近邻搜索

但它特别容易出现的问题是：

- 搜到大量参数近邻
- 高换手
- 复杂度膨胀

因此必须配合：

- 家族管理
- 换手惩罚
- 复杂度上限
- 同质化惩罚

28. 评估指标总表

前文已经讲了很多指标，但如果没有一张总表，下游很难形成稳定认知。这里给一个更完整的总表。

| 指标族 | 指标                   | 目的           | 适用性     | 备注    |
|-----|----------------------|--------------|---------|-------|
| 预测  | TS-IC                | 方向预测相关性      | 全时序场景   | 基础主指标 |
| 预测  | Rank TS-IC           | 对抗极值         | 高频噪声大时  | 建议并行  |
| 预测  | Directional Accuracy | 猜对涨跌比例       | 单资产择时   | 必看    |
| 预测  | Threshold Hit Rate   | 极端信号命中率      | 阈值策略    | 必看    |
| 衰减  | Holding Period Decay | 看 alpha 存活多久 | 分钟频尤其重要 | 必看    |
| 衰减  | Latency Degradation  | 看延迟杀伤力       | 高频      | 必看    |
| 收益  | Gross Return         | 毛收益          | 全部      | 不能单独看 |

|     |                       |            |              |       |
|-----|-----------------------|------------|--------------|-------|
| 收益  | Net Return            | 扣成本后收益     | 高频必看         | 实盘更关键 |
| 风险  | Sharpe                | 风险调整收益     | 全部           | 基础指标  |
| 风险  | Sortino               | 只惩罚下行      | 尾部偏态明显时      | 推荐    |
| 风险  | Calmar                | 收益 / 回撤    | 高频曲线         | 推荐    |
| 风险  | GPR                   | 利润痛苦比      | 高频一致性        | 很有用   |
| 执行  | Turnover              | 换手压力       | 分钟频必看        | 生死线   |
| 执行  | Bid-Ask Crossing Cost | 跨点差损耗      | 高频必看         | 必做    |
| 执行  | Participation Rate    | 参与率压力      | 大资金时         | 可扩展   |
| 容量  | Break-even AUM        | 可承载规模      | 期货/股票        | 入库要看  |
| 同质化 | library_corr_max      | 与库内重合度     | 总库治理         | 必看    |
| 同质化 | family_corr           | 与父因子家族重合度  | 优化家族         | 必看    |
| 稳定性 | intraday_slice_sharpe | 不同时段表现     | 分钟频          | 必看    |
| 稳定性 | regime_stability      | 不同状态下是否稳定  | regime-aware | 必看    |
| 综合  | WQFitness             | 收益、夏普、换手综合 | 高频因子治理       | 建议纳入  |

### 28.1 指标之间的主次关系

如果只让团队记一件事，那就是：分钟频时序因子的指标不能平铺看，必须分层。

第一层先看能不能交易：

- Net Return
- Turnover
- Bid-Ask Crossing Cost
- Latency Degradation

第二层再看预测和稳定：

- TS-IC
- Threshold Hit Rate
- Holding Period Decay
- intraday\_slice\_sharpe

第三层看长期治理：

- family\_corr
- library\_corr\_max
- WQFitness
- regime\_stability

## 29. 工程对象、文件与产物总表

如果这篇文档只讲研究逻辑，不讲工程对象，团队后面还是会断层。下面把企业级时序因子体系里建议保留的核心对象和产物统一列出来。

29.1 对象总表

| 对象                   | 含义            | 由谁生产                | 谁消费                  |
|----------------------|---------------|---------------------|----------------------|
| RawTickEvent         | 原始逐笔成交 / 快照事件 | 数据接入层               | Phase1               |
| SamplingBar          | 聚合后的基础 bar 对象 | Sampling Engine     | Phase1 / 回测 / 详情页    |
| Phase1Feature        | 第一层微观结构和底层因子  | Phase1 Engine       | Merge Layer / Phase2 |
| MergedContextFeature | 并入慢变量后的上下文特征  | Merge Layer         | Phase2 / 模型层         |
| Phase2Factor         | 第二层高层因子       | Phase2 Engine       | 评估 / 入库 / 模型层        |
| PatternScore         | 模式识别打分        | Pattern Engine      | 评估 / gating / 模型层    |
| RegimeState          | 状态识别结果        | Regime Engine       | 评估 / 模型 / 风控         |
| TSThresholdAsset     | 正式登记后的时序因子资产  | Registry            | 评估 / 入库 / 前端         |
| OptimizationFamily   | 父因子家族         | Optimization Engine | 总库治理 / 前端            |

29.2 文件产物表

| 文件                            | 作用                 |
|-------------------------------|--------------------|
| sampling_manifest.json        | 描述 bar 类型、阈值、时钟策略  |
| phase1_feature_matrix.parquet | 第一层特征矩阵            |
| merge_manifest.json           | 记录低频 merge 来源和时间边界 |
| phase2_factor_matrix.parquet  | 第二层因子矩阵            |
| pattern_score_panel.parquet   | 模式识别分数面板           |
| regime_state_panel.parquet    | 状态标签或后验概率          |
| ts_evaluation_report.json     | 时序因子标准评估报告         |
| latency_degradation.json      | 延迟损失分析             |
| intraday_slice_report.json    | 分时段稳定性分析           |
| cost_breakdown.json           | spread、手续费、冲击拆解    |
| family_rank_report.json       | 家族内排序与回总库配额        |
| ts_dashboard_payload.json     | 前端详情页渲染数据          |

29.3 为什么文件产物层必须这么细

因为时序因子的争议比横截面更大。如果不保留这些中间产物，后续任何一次争论都会变成：

- 到底是原始因子有问题
- 还是聚合方式有问题
- 还是 merge 泄露了

- 还是延迟扣完以后就没了

没有证据链，就没有企业级治理。

## 30. 长案例

为了让组员能把前面的抽象内容真正串起来，这里给三个更完整的案例。

### 30.1 案例 A：期货 1min 趋势类时序因子

假设原始数据是股指期货逐笔成交。系统先用 `volume bars` 做 `Phase1` 聚合，形成底层样本。第一层计算：

- 过去若干个 `volume bar` 的方向 `run length`
- `signed volume imbalance`
- `realized volatility short window`
- `breakout pressure`

之后 `merge` 低频特征：

- 日内位置
- 日频波动 `regime`
- 昨日尾盘到今晨开盘的风险状态

进入 `Phase2` 后，再计算：

- 多尺度 `breakout score`
- `trend persistence score`
- `regime-aware trend score`

评估时重点看：

- `TS-IC`
- `Holding Period Decay`
- `Latency Degradation`
- 开盘时段与午盘时段的分离表现

如果因子预测不错但换手极高，先进入 `Tier 2X`，加入：

- `deadband`
- 最短持有期
- `time-of-day gating`

优化后的少量版本如果同时改善换手和净收益，则允许小比例回总库，其余进入 `Tier 3D`。

### 30.2 案例 B：Crypto 逐笔数据上的 `realized vol bars`

原始数据是 `BTC` 或 `ETH` 的逐笔成交。系统不先按时间聚合，而是使用 `realized vol bars`。这样每个 `bar` 更接近“累计完成了一定波动事件”。

`Phase1` 重点做：

- 波动突变检测
- 量价冲击比
- `signed trade clustering`

`merge` 阶段加入：

- 更长时间尺度风险状态

- 交易时段特征
- 链上慢变量或资金流上下文

Phase2 再构造：

- squeeze release
- high-vol continuation
- volatility exhaustion

这种场景里，时间 bar 往往会对关键信息过度压缩，而 realized vol bars 更容易暴露波动驱动的真实结构。

### 30.3 案例 C：模式识别对象只做 gating，不直接下单

某团队构造了一个 breakout authenticity score。它本身并不是一个直接预测未来收益方向的连续因子，但它能够很好地区分“突破是真的”还是“突破大概率是假动作”。

这时最好的用法不是直接拿它：

- 分数大就做多
- 分数小就做空

而是把它作为趋势因子的过滤器：

- 只有当趋势因子想做多，且 authenticity score 也高时，才允许执行

这类对象经常在模型端或组合层发挥巨大作用，但如果研究员一开始就误把它当成普通因子，可能会得出错误结论。

## 31. FAQ

### 31.1 时序因子是不是频率越高越好

不是。频率越高，理论上的样本越多，但噪声、延迟、spread、系统不稳定性也会指数级放大。团队应优先寻找“在目标执行架构下仍然可活”的频率，而不是盲目追求更快。

### 31.2 非时间 bars 是不是一定优于时间 bars

不是。非时间 bars 很有价值，但复杂度更高，参数风险更高，跨资产对齐更难。它们通常应作为增强层，而不是默认替代一切时间 bar。

### 31.3 HMM 是不是一定要上

不是。HMM 很有用，但不是时序因子体系的起点。一个不具备稳定数据工程、稳定预处理和稳定评估的团队，贸然引入 HMM 往往只会增加复杂度。

### 31.4 分钟频高换手是不是只能靠执行层解决

不是。执行层当然重要，但高换手问题必须从挖掘、预处理、信号工程、优化和准入多层同时解决。

### 31.5 为什么要把模式识别单独拿出来

因为很多高价值对象根本不是线性暴露型因子，而是结构事件打分。如果不把它们单列，团队会错误地按普通因子思路去评估和使用。

## 32. 术语表

| 术语 | 含义 |
|----|----|
|----|----|

|                            |                          |
|----------------------------|--------------------------|
| time series factor         | 针对单一资产时间演化进行预测或状态判断的因子   |
| pattern score              | 对某种结构行为打分的对象             |
| regime                     | 市场处于的潜在状态                |
| sampling clock             | 决定样本边界的时钟或事件规则           |
| Phase1                     | 聚合后第一层底层因子计算阶段           |
| Phase2                     | merge 之后的高层因子计算阶段        |
| deadband                   | 死区，忽略小幅信号波动的区间           |
| hysteresis                 | 迟滞，进入和退出阈值不同             |
| latency degradation        | 执行延迟带来的性能衰减              |
| family_id                  | 同一家族优化版本的统一标识            |
| WQFitness                  | 综合收益、夏普和换手的惩罚型指标         |
| PIT (Point-in-Time, 防穿越时间) | point-in-time，保证使用当时可见信息 |

### 33. 实施路线图

如果要把这份文档里的内容真正落成企业级体系，我建议按三阶段推进。

#### 33.1 第一阶段：先把基础主线打通

- 明确 `time_series` 总库
- 打通 `tick -> Phase1 -> merge -> Phase2`
- 先支持 `time bars` 和 `lmin`
- 建立基础时序评估模板
- 引入高换手惩罚

#### 33.2 第二阶段：补时钟与模式层

- 增加 `volume bars / dollar bars / realized vol bars`
- 增加模式识别对象
- 增加 `regime` 层
- 增加高换手优化工厂专用动作

#### 33.3 第三阶段：补企业级治理

- 家族治理
- 详情页
- 回放与审计
- `Tier 3D`
- 跨模型、跨策略、跨总库复用

### 34. 最后说明

这一版文档已经从“体系提纲版”往“长教材版”迈了一大步，但如果按你要求的最终目标，它后面还应该继续扩。真正达到你说的“至少几千行、几万字、发给组员也能系统吸收”的版本，后续还可以继续加强四个方向：

- 继续补更多案例和反例



- 继续补公式字典与统一符号表
- 继续补不同资产专章，例如 `futures / crypto / options`
- 继续补回测与入库治理的更细条款

不过从体系、方法论、工程、评估、优化、模式识别和实施路线这些核心维度看，现在已经搭好了可以持续扩写的大骨架，而且后面每一章都能继续独立往深处挖。

## 35. 不同资产的时序因子差异

同样叫“时序因子”，放在不同资产上完全不是一回事。企业级团队如果不单独拆资产差异，后面会把很多错误包装成“因子失效”。

### 35.1 Futures

期货是时序因子最天然的土壤之一，原因是：

- 方向交易逻辑天然成立
- 不同品种趋势、反转、波动、期限结构特征明显
- 流动性通常比个股分钟级更稳
- 执行链路比股票融券空头更直接

但期货的难点也很鲜明：

- 主连映射和换月
- 夜盘跨日
- 不同品种交易时段差异
- 不同合约乘数、最小变动价位差异

对期货来说，时序因子尤其要强调：

- tick size 归一化
- 成本和滑点模板按品种区分
- 夜盘与日盘分时段评估
- 主力合约切换时的连续性处理

### 35.2 Crypto

`crypto` 的优势在于：

- 24/7 连续交易
- 数据量巨大
- 微观结构和量价互动非常活跃

但也有独特问题：

- 不同交易所市场结构差异大
- 价量数据质量参差不齐
- 刷量和异常流量可能污染特征
- 没有统一休市时段，状态漂移更频繁

对 `crypto` 来说，时序因子尤其要重视：

- 交易所间一致性
- 24 小时不同时段的 regime 差异
- 跨交易所价差与流动性分层
- 事件和新闻冲击传播速度

### 35.3 Equity Intraday

股票日内时序因子与期货、crypto 都不同。它更强依赖：

- 开盘竞价与开盘前若干分钟
- 午盘流动性下降
- 尾盘再平衡
- 行业、指数、被动资金和新闻共振

股票日内时序因子最容易犯的错误是：

- 忽略个股流动性断层
- 把市场 beta 或行业 beta 误认成个股时序 alpha
- 忽略撮合结构和涨跌停等制度约束

### 35.4 Options

期权上的时序因子既可以做在标的价格序列上，也可以直接做在：

- 隐含波动率
- skew
- term structure
- greeks
- 期权成交与持仓变化

它的复杂度比普通价量时序高很多，因为：

- 合约更替频繁
- 时间价值衰减
- 波动率曲面在变化
- 流动性高度不均匀

所以期权时序因子更适合作为高阶研究方向，不建议团队一开始就把它作为时序体系的主入口。

## 36. 公式字典与统一符号表

如果要把这份文档真正做成可长期复用的讲解材料，统一符号非常重要。否则后面不同人对同一公式会有不同读法。

### 36.1 基础符号

| 符号     | 含义                         |
|--------|----------------------------|
| $P_t$  | 时刻 $t$ 的价格                 |
| $V_t$  | 时刻 $t$ 的成交量                |
| $DV_t$ | 时刻 $t$ 的成交额                |
| $r_t$  | 时刻 $t$ 的收益率                |
| $f_t$  | 因子或模式分数                    |
| $s_t$  | 信号状态                       |
| $W$    | lookback window            |
| $H$    | forecast / holding horizon |

|         |               |
|---------|---------------|
| sigma_t | 波动率估计         |
| tau     | 阈值            |
| u_enter | 做多进入阈值        |
| d_enter | 做空进入阈值        |
| eps     | 防止分母为 0 的极小正数 |

## 36.2 常用收益率公式

简单收益率：

$$r_t = P_t / P_{t-1} - 1$$

对数收益率：

$$r_t^{\log} = \log(P_t / P_{t-1})$$

多期累计对数收益：

$$R_{t \rightarrow t+H}^{\log} = \sum_{i=1}^H r_{t+i}^{\log}$$

## 36.3 方向性信号映射

最简单连续映射：

$$s_t = \text{sign}(f_t)$$

带死区的离散化：

$$s_t = \begin{cases} 1, & \text{if } f_t > u_{\text{enter}} \\ 0, & \text{if } d_{\text{enter}} \leq f_t \leq u_{\text{enter}} \\ -1, & \text{if } f_t < d_{\text{enter}} \end{cases}$$

带迟滞状态机的信号：

```
if s_{t-1} == 0 and f_t > u_enter: s_t = 1
elif s_{t-1} == 1 and f_t < u_exit: s_t = 0
elif s_{t-1} == 0 and f_t < d_enter: s_t = -1
elif s_{t-1} == -1 and f_t > d_exit: s_t = 0
else: s_t = s_{t-1}
```

## 36.4 波动率相关公式

滚动标准差：

```
sigma_t = std(r_{t-W+1:t})
```

实现波动率：

```
RV_t = sqrt(sum_{i=1}^W r_{t-i+1}^2)
```

波动率比率：

```
vol_ratio_t = sigma_t^{short} / max(sigma_t^{long}, eps)
```

### 36.5 趋势和路径类公式

均线差：

```
ma_spread_t = MA_short(P)_t - MA_long(P)_t
```

趋势斜率近似：

```
slope_t = (MA(P)_t - MA(P)_{t-k}) / k
```

路径效率：

```
path_efficiency_t = |P_t - P_{t-W}| / sum_{i=1}^W |P_{t-i+1} - P_{t-i}|
```

run length：

```
run_length_t = number of consecutive bars with same sign(return)
```

### 36.6 模式类公式

突破强度：

```
breakout_score_t = (P_t - rolling_high_t) / max(ATR_t, eps)
```

均值回归压力：

```
reversion_pressure_t = -(P_t - rolling_mean_t) / max(rolling_std_t, eps)
```

压缩得分：

```
squeeze_score_t = sigma_t^{short} / max(sigma_t^{long}, eps)
```

### 36.7 流动性与微观结构公式

成交额：

$$DV_t = P_t * V_t$$

signed volume :

$$SV_t = \text{sign}(\text{trade}_t) * \text{volume}_t$$

订单流不平衡简化形式 :

$$OFI_t = \text{buy\_flow}_t - \text{sell\_flow}_t$$

spread :

$$\text{spread}_t = \text{ask}_t - \text{bid}_t$$

相对 spread :

$$\text{relative\_spread}_t = (\text{ask}_t - \text{bid}_t) / \text{mid}_t$$

### 36.8 评估指标公式补全

时序信息系数 :

$$TS\_IC = \text{corr}(f_t, r_{\{t+H\}})$$

方向准确率 :

$$DA = \text{count}(\text{sign}(f_t) == \text{sign}(r_{\{t+H\}})) / N$$

阈值命中率 :

$$THR = \text{count}(\text{valid\_trigger and correct\_direction}) / \text{count}(\text{valid\_trigger})$$

夏普 :

$$\text{Sharpe} = (\text{mean}(r_t) - r_f) / \text{std}(r_t)$$

索提诺 :

$$\text{Sortino} = (\text{mean}(r_t) - r_f) / \text{downside\_std}(r_t)$$

最大回撤 :

$$MDD = \max(1 - \text{NAV}_t / \text{peak}_t)$$

换手 :

```
turnover_t = 0.5 * sum_i |w_{i,t} - w_{i,t-1}|
```

WQFitness :

```
WQFitness = Sharpe * sqrt(abs>Returns) / max(Turnover, 0.125))
```

### 36.9 为什么公式字典重要

团队里最容易发生上下游偏差的地方，不在于“知不知道有这个指标”，而在于：

- Returns 是毛还是净
- Turnover 是按日还是按 bar
- Sharpe 是否年化
- TS-IC 的未来 horizon 是多少

因此公式字典后续还可以继续扩成更严格的表：

- 输入字段
- 计算窗口
- 口径版本
- 是否 after-cost
- 是否硬门槛

## 37. 研究、评估与入库检查清单

最后补一个非常工程化的部分。因为再完整的说明，如果最后没人知道“怎么检查自己有没有漏”，还是会断。

### 37.1 研究前检查

- 是否明确这个对象属于哪一类时序因子
- 是否明确它的目标是方向预测、状态识别还是模式打分
- 是否明确它适用的资产和频率
- 是否明确它要使用什么样本时钟
- 是否明确它的执行假设

### 37.2 构造前检查

- 原始数据是否具备可追溯时间戳
- 是否有 knowledge\_ts
- 是否定义了空 bar 处理方式
- 是否明确了 merge 的慢变量来源
- 是否定义了阈值和窗口的搜索范围

### 37.3 评估前检查

- 是否已经扣除了手续费和点差
- 是否已经评估延迟损失
- 是否已经评估不同时段表现
- 是否已经看过衰减曲线
- 是否已经看过换手和状态切换频率

### 37.4 入库前检查

- 是否与现有时序总库对象过于同质化
- 是否属于父因子近邻家族

- 是否已经打上正确标签
- 是否明确该对象是总库资产、储备资产还是只供模型端
- 是否已生成完整详情页产物

### 37.5 优化前检查

- 这是参数问题、结构问题，还是交易友好化问题
- 是不是应该先做 `deadband` 而不是先重写公式
- 是否需要先做时间段 `gating`
- 是否需要先做 `family` 限额
- 是否需要先从 `reward` 层重新设计挖掘目标

## 38. 横截面评估框架为什么不能直接套到时序上

你给的报告里有一个很重要的提醒，我认为应该明确写进这份总说明书里：把成熟的横截面评估框架直接套到分钟频时序因子上，是系统性误区。不是说横截面体系没有价值，而是它默认回答的问题、默认的收益来源和默认的风险消除方式，与分钟频时序问题根本不是同一类。

### 38.1 收益来源不同

横截面因子的收益来源，本质上是“相对强弱的重新排序”。即使其中包含市场方向信息，最终构造出来的组合通常也会在一定程度上通过多空结构对冲掉全局市场风险，剩下的是相对价值收益。

时序因子的收益来源，本质上是“单一资产未来方向或状态”的预测。它常常保留了更强的绝对方向暴露。这意味着：

- 横截面里好用的市场中性审查思路，不足以覆盖时序风险
- 时序因子的高收益，可能只是市场大趋势的外溢
- 如果不做专门的时序正交和状态分解，很容易把 `beta` 漂移误当成 `alpha`

### 38.2 信息系数的对象不同

横截面里的 `IC` 是同一时间截面上，不同资产之间的相关性。

时序里的 `TS-IC` 是同一资产在不同时间上的相关性。

这两者不能混：

- 截面 `IC` 看的是“谁比谁更强”
- 时序 `TS-IC` 看的是“这个资产自己的历史信号对自己未来还有没有预测力”

分钟频择时如果没有 `TS-IC`，只看横截面排序意义很有限。

### 38.3 风险暴露的处理方式不同

横截面体系中，行业中性化、市值中性化、Barra 风险剥离是主流。

时序体系中，更重要的是：

- 波动率状态剥离
- 时间段效应剥离
- 微观结构偏误剥离
- 父因子家族同质化剥离
- 全局趋势暴露与已知高频基准风险因子的滚动正交化
- **因果推断与抗虚假相关审查**：通过反事实推断和 do-演算，剥离由混淆变量 (Confounders) 造成的虚假相关性，确保提取出真实的因果结构。

### 38.4 时序正交化应该怎么理解

如果有一批高频基准因子，例如：

- 市场方向
- 波动率水平
- 流动性压力
- spread 扩张

那么一个新时序因子的收益可以理解为：

$$r(t) = \alpha(t) + \beta_1 * F_1(t) + \beta_2 * F_2(t) + \dots + \epsilon(t)$$

其中更值得入库的，是剥离掉这些已知解释项后剩下的：

- $\alpha(t)$
- 或残差项  $\epsilon(t)$

这也是为什么企业级时序体系里，不能只看“这个新因子收益挺高”，而必须问：

- 它到底是不是只是另一个高频趋势  $\beta$
- 它是不是只是波动率 regime 的代理变量
- 它是不是只在某些时段的流动性失衡里有效

### 38.5 一个简单的判断标准

如果一个因子满足以下特征：

- 横截面上看着不错
- 时序上 TS-IC 很弱
- 只在强趋势日赚钱
- 去掉市场大方向后明显失效

那它更像是“宏观趋势的微观映射”，而不是高质量时序  $\alpha$ 。

### 38.6 Fama-MacBeth、Barra 和分钟频时序的关系

横截面体系里，很多团队习惯用：

- Fama-French
- Fama-MacBeth
- Barra

来组织风险解释和因子检验。这些框架当然非常重要，但它们的默认语境更偏：

- 截面排序
- 风险暴露解释
- 低频样本期

分钟频时序环境并不是说完全不能借鉴这些思想，而是不能直接原样照搬。更合理的理解方式是：

- Fama-MacBeth 提醒我们要区分“因子解释力”和“系统性风险暴露”
- Barra 提醒我们要做风险剥离和残差化
- 但分钟频时序必须把这些思想改写为：
  - 更短频率
  - 滚动窗口
  - session / regime 切片
  - 高微观摩擦环境下的动态正交化



因此，思想虽可借鉴，但评估协议必须彻底重构。

### 38.7 市场中性与定向暴露的差别

横截面因子经常通过多空对冲在组合层逼近市场中性，因此很多低频评估天然就把“市场大方向的影响”削弱掉了。

时序因子不同。大量时序因子本质上就是在预测绝对方向，天然带有更强的：

- directional exposure
- market trend sensitivity
- regime dependence

这意味着系统必须明确区分：

- 因子是在预测独立 alpha
- 还是只是在跟随全局市场方向

如果不区分，团队会把很多“方向 beta”误认成“高质量时序 alpha”。

### 38.8 工业级因子的四个标准

无论是低频还是高频，工业级因子都不该只看某一个样本内结果。这里给出四个我建议长期写进团队认知里的标准：

- overallity
- parsimony
- orthogonality
- tradability

它们在时序体系中的含义分别是：

#### Overallity

不是要求因子在所有市场、所有 regime、所有 session 都最强，而是要求：

- 它的有效性不能只停留在一个极窄的局部切片
- 它应该有可解释的作用范围

#### Parsimony

时序因子尤其容易结构膨胀。一个分钟频因子如果依赖：

- 大量子表达式
- 复杂条件分支
- 多层参数堆叠

即使样本内有效，也很可能不稳。

#### Orthogonality

这不是说和所有东西都不相关，而是说：

- 在剥离已知基准风险后
- 它仍然保留足够独立的信息

#### Tradability

这在分钟频里是最残酷的一条：

- 能不能成交
- 扣完点差后还剩多少

- 延迟一格还活不活
- maker / taker 下是否 still valid

如果 tradability 不成立，其他一切都只是研究幻觉。

## 39. 时序专属标签字典与动作标签

前文已经有通用标签和优化标签，但你报告里提到的分钟频时序标签值得单独扩成一章，因为它们和执行、生存度、优化工厂直接相关。

### 39.1 风格标签

建议至少支持：

- `ts_trend_following`
- `ts_mean_reversion`
- `volatility_breakout`
- `path_dependency`
- `microstructure_flow`
- `intraday_range_breakout`
- `open_drive`
- `close_rebalance_sensitive`

这些标签的意义不只是展示，还包括：

- 选择评估模板
- 选择优化模板
- 选择合适的执行引擎

### 39.2 执行与摩擦标签

这些是分钟频时序体系非常重要的一类标签。

建议明确支持：

- `latency_hyper_sensitive`
- `maker_only`
- `high_spread_crossing_risk`
- `slippage_fragile`
- `queue_position_critical`
- `session_sensitive`

其中：

- `latency_hyper_sensitive` 表示 alpha 的生存时间极短，稍有延迟就会显著衰减
- `maker_only` 表示只有被动挂单执行时才可能保留正的净收益，若使用吃单执行则大概率亏损

### 39.3 动作标签

针对分钟频高换手问题，建议补充以下动作标签：

- `needs_deadband`
- `needs_hysteresis`
- `needs_time_of_day_gating`
- `needs_downsample`
- `needs_vol_scaling`

- `needs_latency_review`
- `needs_maker_constraint`

### 39.4 生命周期标签

分钟频时序因子的生命周期管理，也可以比普通因子更细：

- `ts_new_candidate`
- `ts_incubating`
- `ts_execution_review`
- `ts_maker_only_restricted`
- `ts_watchlist_latency`
- `ts_archived_cost_killed`

### 39.5 为什么标签在时序体系里更重要

因为分钟频因子的“死法”比日频因子更多。

一个因子可能不是预测没用，而是：

- 预测有用，但一扣 `spread` 就没了
- 预测有用，但延迟一分钟就死
- 预测有用，但只在某个时段成立
- 预测有用，但只适合 `maker`，不适合 `taker`

如果没有这些标签，研究、评估、执行和优化团队会不断重复踩坑。

## 40. 高频执行生存度指标与工业级门槛

你报告里对这部分强调得很对，我把它单独拉出来，因为在分钟频里，“能不能赚钱”和“能不能活下来”是两回事。

### 40.1 为什么单看收益没有意义

分钟频因子经常会出现一种错觉：

- 理论回测毛收益很好
- 方向准确率也不差
- 但只要加上点差、手续费、少量延迟，净收益立刻翻负

所以高频执行生存度指标不是锦上添花，而是决定入库与否的生死线。

### 40.2 核心工业级指标

#### Out-of-Sample $R^2$

在时序体系里，哪怕很小的样本外  $R^2$  也可能有经济价值，但它必须稳定存在，不能只是样本内幻觉。

#### Profit Factor

```
ProfitFactor = GrossProfit / max(|GrossLoss|, eps)
```

这个指标直观地回答：总利润是否足够覆盖总亏损。

#### Gain-to-Pain Ratio

```
GPR = sum(max(r_t, 0)) / max(|sum(min(r_t, 0))|, eps)
```

比单纯看夏普更容易看出策略是在“稳步赚钱”，还是在“大亏后偶尔暴赢”。

### Ulcer Index

它衡量的不只是回撤有多深，还包括回撤持续多久。对高频一致性很重要。

### Intraday Seasonality Stability

不是只看全天的平均，而是必须分：

- 开盘
- 午盘
- 尾盘
- 不同时区

如果一个分钟频因子只在少量时间段有效，系统必须明确知道，而不是被平均值掩盖。

## 40.3 Bid-Ask Crossing Cost 为什么是生死线

分钟频里，最容易把策略从正收益打到负收益的，不是复杂的冲击模型，而是最朴素的“每次翻仓都要跨一次点差”。

因此建议系统至少强制评估两套净收益：

- mid-price idealized
- bid-ask crossed executable

如果一个因子在第二套下已经翻负，那么即使第一套看上去很好，也不应直接进入可交易总库。

## 40.4 Maker-Only 的工程含义

maker\_only 不是一个展示标签，而是执行约束。它意味着：

- 该因子若以吃单方式执行，大概率失去全部 alpha
- 只能在挂单成交概率、排队位置和撤单逻辑可控的架构下使用
- 风险管理和执行系统必须感知这个标签

这也说明，时序因子的评估和执行不能割裂。

## 41. 时序优化工厂的专属修复动作

前文已经写过参数级优化、降换手和去同质化，但你给的报告里提到的一些动作应该更明确地作为“工业级修复动作库”写下来。

### 41.1 Schmitt Trigger / Deadband Rewrite

这是分钟频高换手最经典的一类修复动作。

原始逻辑可能是：

```
f_t > 0 -> long
f_t < 0 -> short
```

优化后改成：

```
if state == flat and f_t > 1.5: long
if state == long and f_t < 0.5: flat
if state == flat and f_t < -1.5: short
if state == short and f_t > -0.5: flat
```

它的关键价值在于：

- 在震荡市降低零轴附近来回翻仓
- 给信号加记忆
- 用进入阈值和退出阈值分离的方式抑制抖动

## 41.2 Signal Discretization

连续信号有时过于敏感，可以离散化为：

- `[-1, 0, 1]`
- 或少量风险层级状态

它的好处是：

- 更利于控制换手
- 更利于和执行系统对接
- 更容易解释

## 41.3 Downsampling

如果分钟频信号预测有效，但每分钟都在变，系统可以强制：

- 每 5 分钟或 15 分钟才允许改一次状态
- 或者持仓状态至少保持 `N` 个 bars

这类动作有时比重新发明公式更有效。

## 41.4 Time-of-Day Gating

如果某个因子只在：

- 开盘后 30 分钟
- 或尾盘重平衡阶段
- 或某个特定市场时区

有价值，那么最好的优化动作不是勉强让它全天都用，而是：

```
if session in allowed_sessions:
    use factor
else:
    output 0
```

## 41.5 Volatility-Aware Gating

如果某因子只在高波环境或低波环境下有意义，则能让优化器自动加入：

- `if high_vol_regime then factor else 0`
- 或者根据波动 regime 切换不同参数模板

## 41.6 Maker Constraint Rewrite

如果因子在 `taker` 执行下会死，但在 `maker` 约束下还能活，优化器应当允许把该因子标记成：

- `maker_only`
- 并重评其在挂单逻辑下的可执行性

这类优化已经不是单纯公式改写，而是公式和执行协同重写。

## 42. 如何把报告内容继续系统地补进来

你现在的要求不是让我只补我自己想到的，而是尽量把报告里已有但当前文档还没完全吸收的东西继续往里补。这一步最合理的做法，是后续继续按下面这个顺序扩：

第一批继续补：

- 高频微观结构指标专章
- 高频执行与滑点仿真专章
- 时序因子的正交化与基准风险因子剥离专章
- 分时段评估与全球时区切片专章

第二批继续补：

- 更完整的分钟频回测协议
- 逐笔与盘口快照对象定义
- `maker / taker / queue position` 的工业级解释
- 订单簿型与成交型时序因子的分流设计

第三批继续补：

- 报告里涉及的更完整工业案例
- 细化 `TS-IC / Rank TS-IC / OOS R2 / Profit Factor / GPR / Ulcer Index`
- 高频时序因子的入库门槛字典

换句话说，后面不是“继续润色”，而是继续把报告里的工业级内容一点点消化成这份总说明文档的正式章节。

## 43. 时序因子的正交化与基准风险剥离

分钟频时序因子里非常容易出现一种错觉：一个因子在回测里看起来稳定赚钱，但真正贡献收益的并不是该因子的独特信息，而是它间接暴露到了某个更基础、更粗粒度、也更拥挤的风险源。为了防止把这些风险映射误收进总库，企业级体系必须单独做时序正交化。

### 43.1 为什么分钟频更需要正交化

横截面体系中，大家已经比较习惯做：

- 行业中性化
- 市值中性化
- 风险因子回归取残差

而在分钟频时序里，同样存在一批“伪 alpha 来源”：

- 市场整体趋势推进
- 高波动 regime 本身
- 开盘和尾盘的结构性价流动
- spread 突然放大时的微观结构共性

- 极端行情中的单边 order flow

如果一个分钟频因子没有剥离这些共性风险，就可能只是：

- 另一个趋势代理
- 另一个波动状态代理
- 另一个开盘时段代理
- 另一个 liquidity stress 代理

### 43.2 推荐的高频基准风险因子

企业级时序因子体系建议维护一组“高频基准风险因子库”，至少包括：

- market\_direction\_fast
- realized\_vol\_level
- spread\_regime
- liquidity\_pressure
- open\_session\_bias
- close\_session\_bias
- intraday\_momentum\_beta
- intraday\_reversal\_beta

这些对象本身未必直接进入生产，但它们应该被当成新时序因子的参考基准。

### 43.3 滚动回归残差化

一种实用的工业方法是滚动窗口残差化：

$$r\_factor(t) = \alpha(t) + \sum_k \beta_k(t) * F_k(t) + \epsilon(t)$$

其中：

- $r\_factor(t)$  可以是因子的策略收益、信号触发收益、或标准化因子暴露
- $F_k(t)$  是高频基准风险因子
- $\epsilon(t)$  是更值得关注的残差部分

与低频世界不同的是：

- 这里的  $\beta_k(t)$  不应假定长期稳定
- 更适合用滚动回归、滚动 ridge 或带遗忘因子的方式估计

### 43.4 哪些因子必须做时序正交化审查

以下对象特别值得强制做正交化审查：

- 趋势类分钟频因子
- 高波动时更强的 breakout 类因子
- 开盘前后强依赖的模式分数
- 和 spread / order flow 高度耦合的微观结构因子

如果一个对象：

- 正交化前非常亮眼
- 正交化后几乎失去全部预测力

那么它在总库中应被更谨慎地定位，可能更适合放入：

- Tier 3D 作为条件性特征
- 或仅作为模型端上下文变量

## 44. 分钟频回测协议与执行仿真

如果时序因子文档不单独讲分钟频回测协议，后面几乎必然会出现“研究里好看，实盘里死掉”的问题。因为分钟频最大的难点之一，就是回测和真实执行之间的缝隙远大于日频。

### 44.1 回测引擎至少要支持两层价格语义

第一层是理想化价格：

- mid
- close
- VWAP

第二层是可执行价格：

- bid
- ask
- bid/ask adjusted VWAP

研究人员可以看第一层价格来理解信号本身，但总库准入和工业级评估必须看第二层价格。

### 44.2 延迟建模

分钟频策略中，一个非常关键的问题是：预测在  $t$  时刻产生，但成交不一定发生在  $t$ 。因此回测至少要支持：

- `delay = 0 bar`
- `delay = 1 bar`
- `delay = 2 bars`
- 指定毫秒 / 秒级延迟近似

所有时序因子都建议输出：

- `latency_degradation_curve`
- `delay_sensitivity_score`

### 44.3 部分成交与排队问题

很多分钟频信号，尤其是需要挂单执行的信号，会受到：

- 排队位置
- 成交概率
- 被动挂单撤单率

影响。第一阶段可以不把这套仿真做得极其复杂，但至少要有三档执行假设：

- `taker executable`
- `maker optimistic`
- `maker conservative`

### 44.4 时间段与交易会会话仿真

同一个因子在以下时段可能完全是不同物种：

- 开盘冲击段
- 午盘低流动性段



- 尾盘再平衡段
- 夜盘开盘段
- 周末或节假日前后

因此分钟频回测协议必须支持：

- 按 session 切片
- 按时区切片
- 按交易日类型切片

#### 44.5 交易成本拆解

成本不应只做成一个黑盒数字。建议至少拆成：

- 手续费
- 点差穿越成本
- 冲击近似成本
- 延迟损耗
- 撤单 / 未成交机会成本

并分别输出到：

- `cost_breakdown.json`
- `slippage_breakdown.json`
- `latency_breakdown.json`

#### 44.6 为什么分钟频回测协议要比横截面更重

横截面日频策略的研究结果，通常不会因为 20 个基点的额外点差建模就完全翻脸。分钟频策略完全不同，它的 alpha 空间很薄，任何一个小假设都有可能从：

- 正夏普 变成
- 负净值

这就是为什么分钟频回测协议必须被写进总说明书，而不能被默认为“实现细节”。

### 45. 逐笔成交、盘口快照与订单流对象层

前文虽然已经提到了逐笔数据和盘口，但还需要更明确的对象层说明。否则团队在真正实现时，还是容易把所有高频对象混成一个模糊的数据源。

#### 45.1 Raw Trade

原始逐笔成交对象建议至少保留：

- `trade_id`
- `event_ts`
- `price`
- `volume`
- `notional`
- `side_inference`
- `exchange`

其中 `side_inference` 可以是：

- 主动买
- 主动卖

- 未知

## 45.2 Order Book Snapshot

如果有盘口快照，建议至少保留：

- `bid_1 ... bid_n`
- `ask_1 ... ask_n`
- `bid_size_1 ... bid_size_n`
- `ask_size_1 ... ask_size_n`
- `snapshot_ts`

## 45.3 Event Stream 与 SamplingBar 的关系

逐笔和盘口并不是直接拿来给所有人用的最终对象。工业上通常会有两层：

- 事件流层：用于最底层重建和 replay
- bar / feature 层：用于因子和回测主线

这样做的好处是：

- 能保留原始可回放性
- 不强迫所有下游都直接消费 tick 级对象
- 让 `Phase1` 有明确输入

## 45.4 订单流类因子的关键输入

如果团队要做更高级的微观结构因子，可以重点围绕：

- `signed volume`
- `order flow imbalance`
- `queue depletion`
- `trade clustering`
- `spread bounce`
- `book pressure`

这批对象展开。它们往往比单纯的分钟收盘价更贴近真正的信息流。

# 46. 时序入库门槛字典

这部分非常适合直接变成未来规则引擎的策略表。先说明一点：分钟频时序因子的入库门槛不应该只看一个收益或一个相关性指标，而应该分层设门槛。

## 46.1 第一层：生存门槛

如果过不了这一层，后面不用看了。

- `Net Return after spread` 不能为明显负
- `Latency Degradation` 不能极端恶化
- `Turnover` 不能无限高且没有相应收益补偿
- 至少一个可执行假设下仍为正期望

## 46.2 第二层：预测门槛

- `TS-IC`
- `Directional Accuracy`
- `Threshold Hit Rate`

- 00S  $R^2$

这层回答的是“有没有预测价值”。

### 46.3 第三层：稳定性门槛

- Holding Period Decay
- intraday\_slice\_stability
- regime\_stability
- Gain-to-Pain Ratio

这层回答的是“是不是只在某个小角落里看起来成立”。

### 46.4 第四层：总库治理门槛

- library\_corr\_max
- family\_corr
- family\_rank\_within\_batch
- optimization\_type
- source\_factor\_id

这层回答的是“即使它有效，是否值得占用总库席位”。

### 46.5 一个推荐的路由逻辑

如果一个分钟频时序因子满足：

- 预测显著
- 成本后仍正
- 高换手但可通过 deadband 改善

则不应直接判死刑，而更适合：

- 路由到 Tier 2X
- 优先尝试 Schmitt Trigger
- 再试 downsample
- 再试 time-of-day gating

如果一个因子满足：

- 预测显著
- 净收益显著
- 但与现有父因子家族高度接近

则更适合：

- 仅保留少量代表版本进总库
- 其余进入 Tier 3D

## 47. 分时段与全球时区切片

报告里强调了 Intraday Seasonality，这一点必须再单独抬高一级。因为对分钟频来说，全天不是一个统一市场。

### 47.1 为什么必须切片

分钟频因子很容易只在：

- 美股开盘前 15 分钟
- 开盘后一小时
- 尾盘再平衡
- 亚洲凌晨的低流动性时段

有效。如果只看全天平均，很多高风险的局部模式会被掩盖。

## 47.2 建议至少切这些片

- 开盘冲击段
- 上午连续交易段
- 午盘低流动性段
- 尾盘再平衡段
- 夜盘起始段
- 欧洲开盘前后
- 美国开盘前后

## 47.3 切片结果怎么用

切片结果不只是用来看图，而要直接影响：

- 标签
- 优化动作
- 路由
- 模型端 gating

例如：

- 如果因子只在尾盘有效，就可以直接打上 `close_rebalance_sensitive`
- 如果午盘系统性失效，则可自动建议 `time-of-day gating`

## 48. 下一轮继续补什么

当前这份时序总说明文档已经把很多核心部分立起来了，但如果继续吸收你报告里的内容，下一轮最值得继续补的是：

- 高频微观结构因子专章
- 更完整的分钟频执行仿真专章
- `maker / taker / queue` 的更细解释
- `TS-IC`、`OOS R2`、`Profit Factor`、`Ulcer Index` 的更完整口径表
- 更详细的主连、夜盘、跨交易日处理

因此，后续工作重点不再是重构框架，而是持续深化工业级实施细节，使文档达到企业级参考书的标准。

## 49. 时序因子评估体系总览

前文已经分散地讲过很多评估点，但如果后面这份文档要成为“回填主需求文档和 HTML 的母版”，就必须把时序因子评估体系整理成一个更完整、更工业化的总览。

企业级时序因子评估，不应再被理解为“跑个回测，看个 Sharpe”。真正完整的评估体系至少要同时覆盖七层：

第一层，预测能力。

第二层，收益风险。

第三层，执行与摩擦。

第四层，稳定性与衰减。

第五层，样本外与抗过拟合。

第六层，同质化与家族治理。

第七层，准入、详情页产物与审计闭环。

只有把这七层同时搭起来，时序因子评估才算真正工业化。

### 49.1 为什么分钟频评估必须是多层体系

因为分钟频时序因子最容易出现三类错觉：

- 统计上看起来有效，但交易后完全不赚钱
- 回测里看起来赚钱，但只在单一时段或单一 regime 成立
- 净收益还不错，但其实只是同一家族已有因子的近邻版本

如果系统只看单层指标，就会把大量伪 alpha、脆弱 alpha 和冗余 alpha 收进总库。

## 50. 预测能力评估专章

### 50.1 TS-IC 与 Rank TS-IC

TS-IC 是时序评估的主轴之一，但它不是“看一下相关系数就完了”。真正企业级做法至少包括：

- 整体 TS-IC
- 分 regime TS-IC
- 分 session TS-IC
- 分资产 TS-IC
- 滚动窗口 TS-IC

同理，Rank TS-IC 不只是为了稳健，它还能帮助识别：

- 信号方向确实对
- 但幅度容易被极端值污染

### 50.2 Directional Accuracy 的正确理解

方向准确率并不是越高越好。分钟频场景下：

- 胜率略高于 50% 也可能有巨大价值
- 胜率很高但盈亏比极差，同样没用

因此方向准确率应至少和以下量联动看：

- average win / average loss
- 持仓时长
- 状态切换频率
- 成本后盈亏比

### 50.3 Threshold Hit Rate 的工业价值

很多分钟频因子不是全时间都值得交易，而是只有在信号足够极端时才有价值。此时：

- TS-IC 只能反映连续预测能力
- Threshold Hit Rate 更能反映“真正要开仓时是否靠谱”

建议系统支持：

- 多阈值扫描
- 多 horizon 扫描
- 阈值命中率热力图

## 50.4 Out-of-Sample $R^2$

分钟频环境中，样本外  $R^2$  往往不会很大，但这并不代表它没意义。关键在于：

- 是否稳定为正
- 是否在不同滚动窗下重复出现
- 是否在成本后仍能转化为净收益

如果一个因子：

- 样本内  $R^2$  很亮眼
- 样本外显著掉到接近 0

这通常意味着它更接近过拟合结构，而不是稳定时序 alpha。

## 51. 收益风险评估专章

### 51.1 Gross vs Net 必须并列展示

时序分钟频因子的收益展示，必须默认同时输出：

- gross return
- net return

而且要能拆成：

- 扣手续费前后
- 扣点差前后
- 扣延迟前后

如果只展示毛收益，几乎一定会夸大分钟频因子的真实质量。

### 51.2 Sharpe 为什么不能单独看

Sharpe 很重要，但分钟频里单独看它太危险。因为：

- 少数极端阶段会推高收益
- 高换手可能掩盖真实执行脆弱性
- 非正态分布会让标准差低估真实尾部风险

所以 Sharpe 至少要联动：

- Sortino
- Calmar
- GPR
- Ulcer Index

### 51.3 最大回撤与恢复时间

对于分钟频时序策略，很多团队只盯最大回撤深度，但恢复时间同样关键。

建议至少同时记录：

- max\_drawdown
- drawdown\_duration

- `time_to_recovery`

原因很简单：

- 两个策略回撤都 8%
- 一个 2 天恢复
- 一个 2 个月恢复

它们在资金占用和风险承受意义上完全不是一回事。

## 51.4 Gain-to-Pain Ratio 和 Ulcer Index 的必要性

这两个指标特别适合分钟频。

GPR 强调的是：

- 最终赚的钱相对最终亏的钱是否够多

Ulcer Index 强调的是：

- 资金曲线在回撤状态里“难受了多久”

对高频策略来说，这两个维度经常比“单点最大回撤”更接近真实体验。

## 52. 执行与摩擦评估专章

### 52.1 成本后收益必须成为主视图

企业级时序评估系统不应把成本后结果放在附录，而应该把它放在主视图。

建议默认主视图就是：

- `net_sharpe`
- `net_return`
- `net_calmar`

毛口径只作为对照。

### 52.2 跨点差成本的两级审查

建议至少审查两层：

第一层：

- 每次翻仓强制跨一次点差

第二层：

- 在点差动态扩大阶段，按更保守的 spread 假设重评

因为很多分钟频因子在正常 spread 下还能活，一到流动性差时段就会被 spread 吃光。

### 52.3 延迟衰减为什么要单独列章

延迟衰减不是普通成本的一部分，它本质上在衡量：

- alpha 的生存时间有多短
- 团队的执行架构是否跟得上

建议至少输出：

- `delay_0`
- `delay_1_bar`
- `delay_2_bars`
- `delay_5_seconds`

并画出完整 degradation curve。

## 52.4 Maker/Taker 分层评估

一个成熟的时序评估系统，至少应支持三种执行结果：

- `taker executable`
- `maker optimistic`
- `maker conservative`

如果某个因子只有在 `maker optimistic` 下才能活，那它的标签和路由应明显不同于正常可交易因子。

## 53. 稳定性与衰减评估专章

### 53.1 Holding Period Decay

分钟频因子如果连自己的  $\alpha$  寿命都不知道，是无法工业化使用的。

建议对每个因子至少输出：

- `T+1`
- `T+5`
- `T+15`
- `T+30`
- `T+60`

这些 horizon 的累积收益和风险指标。

### 53.2 Session Stability

不同时段的表现必须作为一级评估对象，而不是可选分析。

建议至少切：

- 开盘
- 上午
- 午盘
- 尾盘
- 夜盘
- 欧美时区
- 亚洲时区

### 53.3 Regime Stability

如果一个因子只在高波趋势市赚钱，在低波震荡里系统性亏损，那么它不一定不能用，但它必须被明确地归入：

- 条件性因子
- regime-aware 因子
- 或 gating 对象

而不能被误判成全天候通用因子。



## 53.4 Universe / Asset Stability

时序因子除了切时间，还要切资产：

- 只在少数高流动性品种有效
- 还是在更广泛的 futures / crypto / intraday equity 上都能复现

这直接决定它适合：

- 单资产策略
- 小范围专用库
- 还是更广泛总库

## 54. 样本外与抗过拟合评估专章

### 54.1 Walk-Forward 不是可选项

分钟频时序因子的样本外验证，默认就应该是：

- walk-forward（滚动推进）
- 或 rolling window

而不是静态 train/test 一刀切。

### 54.2 Purged / Embargo 的必要性

只要：

- 标签 horizon 重叠
- 持有期重叠
- 同一事件影响跨多个样本

就必须认真考虑 purged（标签清理）/ embargo（防泄露隔离验证）。否则很多样本外结果只是样本之间相互穿透。

### 54.3 多窗口复现

建议时序因子不仅要过一个样本外窗口，而要看：

- 不同年份
- 不同波动环境
- 不同 regime
- 不同 session

是否都能维持最基本的有效性。

### 54.4 过拟合警报信号

以下现象都应该被系统明确打标：

- 参数稍微一改就失效
- 只在单一年份成立
- 只在一个小时段成立
- 样本内很好，样本外快速崩
- family 里只有一个孤立版本极度突出

## 55. 同质化、家族治理与总库席位评估

### 55.1 library\_corr\_max 还不够

分钟频时序总库不能只看和“全库”的最大相关性，还应看：

- 和父因子的相关性
- 和同一家族的相关性
- 和同一 session 主导因子的相关性
- 和同一 regime 优势因子的相关性

### 55.2 Family Rank

同一父因子下通过参数、平滑和局部改写生成的版本，通常只允许极少数代表进入总库。

因此建议每个 batch 都输出：

- family\_rank\_within\_batch
- family\_redundancy\_score
- representative\_flag

### 55.3 总库席位经济意义

总库席位不是荣誉，而是稀缺资源。一个对象即使表现不错，如果只是：

- 已有对象的参数近邻
- 只在很窄的 regime 下有用
- 对模型端提供的信息几乎重复

那它更适合放进：

- Tier 3D
- 或模型端候选池

而不是占用主总库席位。

## 56. 准入决策与详情页产物

### 56.1 时序因子的准入不是二元 approve/reject

分钟频因子的路由尤其需要细分：

- 可直接入总库
- 需进入 Tier 2X 做降换手或 gating 优化
- 仅适合 Tier 3D
- 仅适合作为模型端条件特征
- 成本后失败，直接归档

### 56.2 详情页必须有什么

时序因子的详情页比横截面更需要“动态诊断图”。

建议至少包括：

- 资金曲线
- 毛/净收益对照
- 持有期衰减曲线
- 延迟衰减曲线
- session 切片表现

- regime 切片表现
- cost breakdown
- spread sensitivity
- family 关系图
- 标签与执行限制

### 56.3 为什么详情页本身也是评估的一部分

因为分钟频时序因子的很多问题，不看图很难发现：

- 开盘暴赚，其他时间段全亏
- 加一格延迟就崩
- spread 一扩大就死
- 只在某个 regime 下活

如果详情页产物不完整，很多劣质因子会被指标平均值掩盖。

## 57. 一套推荐的时序评估流水线

如果后面要把这一套回填进主需求文档，我建议的流水线顺序是：

1. 数据和时间戳审查
2. sampling / bar 规则确认
3. Phase1 / merge / Phase2 产物落盘
4. 时序预处理
5. 预测能力评估
6. 收益风险评估
7. 成本与执行评估
8. 衰减与稳定性评估
9. 样本外与抗过拟合评估
10. 家族同质化与总库席位评估
11. 标签、路由和详情页产物生成

### 57.1 这套顺序为什么重要

因为时序评估如果一开始就只看最终收益，会错过大量上游结构问题。

例如：

- 如果 sampling 就有问题，后面所有收益都不可信
- 如果 merge 泄露了，后面所有预测都被污染
- 如果成本层没过，后面再稳也没意义

## 58. 这部分后续如何回填到主需求文档

你已经说过，后面会让我把这份文档里的时序因子评估部分补进原来的需求文档和 HTML。

为了方便后续回填，我建议后面抽取时重点拿这几块：

- 49-57 这组章节，作为评估母版
- 39-41 的时序标签与优化动作，补到主文档的标签和 Tier 2X 章节
- 44-47 的执行、对象、入库门槛和 session 切片，补到 HTML 的时序分支

这样后续回填会更稳，不需要再重新组织逻辑。

## 59. 高频微观结构因子专章

前文已经多次提到微观结构，但还没有把它展开成真正独立的一章。考虑到分钟频和更高频时序因子的上限，很大一部分就来自微观结构层面的细节，因此这一章必须写厚。

### 59.1 微观结构为什么重要

在低频世界里，价格和成交量往往已经足够承载大部分可交易信息；但在分钟频尤其是更高频场景中，市场中的一大部分信息其实隐藏在以下对象里：

- 买卖盘深度如何变化
- 谁在主动吃单
- 队列在某一侧是否持续被抽空
- spread 是自然扩大还是因为瞬时流动性断层
- 成交是否成簇爆发
- 订单流是否连续单边推进

这些对象之所以重要，是因为它们更靠近“交易行为本身”而不是“价格结果的最终投影”。

### 59.2 微观结构因子的主要家族

我建议把高频微观结构因子至少分成六类：

- order\_flow\_imbalance
- queue\_pressure
- spread\_dynamics
- trade\_clustering
- book\_shape
- microprice\_and\_imbalance

### 59.3 Order Flow Imbalance

最典型的一类微观结构因子，是订单流不平衡。直觉上，如果主动买单持续压过主动卖单，那么短时间内价格继续上行的概率会提高。

一个最简化的表达可以写为：

```
OFI_t = buy_initiated_volume_t - sell_initiated_volume_t
```

再做标准化：

```
OFI_norm_t = OFI_t / max(total_volume_t, eps)
```

更工业化的做法还会考虑：

- 多层盘口的变动
- 新挂单与撤单
- 主动成交对队列的消耗

### 59.4 Queue Pressure

盘口队列压力关注的是“某一侧盘口是否正在被快速消耗”。这类信息对短时趋势和冲击延续特别有价值。

常见直觉：

- 如果卖一、卖二、卖三的量被持续消耗，而买盘又没有同步后退，向上突破更容易成立
- 如果买一队列不断撤退，价格支撑会变弱

一个简化示意可以写为：

```
queue_pressure_t = (bid_size_1_t - ask_size_1_t) / max(bid_size_1_t + ask_size_1_t,
eps)
```

但工业上通常会扩展到多档盘口，并且引入时间变化率。

## 59.5 Spread Dynamics

spread 本身就是一类强上下文信号。

一个突破如果发生在：

- spread 收敛
- 盘口稳定
- 主动买盘推进

和发生在：

- spread 突然放大
- 流动性断层
- 价格被空洞跳空抬高

它们的真实性完全不同。

因此微观结构系统应至少保留：

- 绝对 spread
- 相对 spread
- spread 变化率
- spread regime

## 59.6 Trade Clustering

成交成簇爆发，往往对应更强的信息到达或更强的交易意图。分钟频体系里，这类特征常常比简单的“本 bar 成交量大”更有价值。

可以关注：

- 单位时间内成交笔数突增
- 同方向主动成交连续聚集
- 成交额与价格推进同向共振

## 59.7 Book Shape

盘口形状也是一类重要信号。不是所有盘口都长得一样：

- 有的盘口非常平滑
- 有的盘口近端厚、远端薄
- 有的盘口一侧存在明显的“墙”

这些结构可能决定：

- 突破是否容易持续
- 价格是否更容易回弹

- 某个方向的冲击成本是否更高

## 59.8 Microprice

如果只看中间价，往往会忽略盘口两侧量的不对称。可以考虑用 `microprice` 来更精细地描述短时价格重心：

```
microprice_t = (ask_t * bid_size_t + bid_t * ask_size_t) / max(bid_size_t + ask_size_t, eps)
```

当 `microprice` 明显偏向某一侧时，短时价格更容易向该方向推进。

## 59.9 微观结构因子的常见误区

- 只看成交，不看盘口
- 只看盘口，不看主动成交方向
- 把异常 spread 放大误认为趋势信号
- 把撮合噪声当成结构 alpha
- 不做跨交易所或跨品种的数据质量审查

## 59.10 微观结构对象更适合作为什么

不是所有微观结构对象都适合直接当最终因子。

很多更好的用法是：

- 作为模式真实性打分
- 作为趋势因子的过滤器
- 作为执行风险提示
- 作为模型端辅助特征

## 59.11 微观结构特征最怕时钟和对齐出错

微观结构特征是所有时序对象里最怕时钟错位的一类。因为它们的有效 horizon 本来就短，一旦对齐做错，即使只错一格，也可能把真实信号直接抹掉，或者反过来制造伪 alpha。

最常见的错误包括：

- 用 bar 结束后的盘口状态去预测 bar 内已经发生的收益
- 用撮合后的最终成交量解释撮合前的决策
- 把不同交易所、不同源头的事件流强行对齐到同一时间戳
- 在异步订单簿更新里误把“最后一次快照”当成“该时刻真实可见状态”

因此微观结构研究必须格外强调：

- 事件时间顺序
- 决策快照时点
- 观测延迟
- 市场数据落地延迟

如果这些边界不清楚，那么再精致的 `OFI`、`microprice` 或 `queue_pressure` 都没有意义。

## 59.12 微观结构特征为什么要去做去噪

微观结构层面的信息密度极高，但噪声密度也极高。许多短时尖峰并不代表真实交易意图，而可能来自：

- 瞬时撤单
- 小额试探单

- 数据抖动
- 单笔异常成交
- 交易所撮合细节导致的短时跳变

所以工业化体系里，微观结构特征几乎都需要做某种温和去噪。例如：

- rolling median
- winsorization
- 限幅处理
- 短窗口平滑
- 异常点标记而非直接删除

这里最重要的原则不是“越平滑越好”，而是：

- 保留真正的结构变化
- 去掉纯撮合噪声

如果平滑过度，会把真正的短时 alpha 一起抹掉；如果完全不平滑，又会让策略看上去充满信号但其实不可重复。

### 59.13 跨交易所、跨品种时为什么不能直接复用同一套微观结构定义

很多研究员第一次做微观结构时，习惯在一个市场里找到有效定义，然后直接平移到其他市场。这往往会出问题。

原因包括：

- 不同市场的最小变动价位不同
- 不同市场的盘口深度结构不同
- 不同市场的撮合规则不同
- 不同市场的主导流动性提供者行为不同
- 加密、期货、股票的日内节律显著不同

例如：

- 在某些加密交易所里，盘口撤挂频率极高，`queue_pressure` 的含义会更脆弱
- 在流动性更好的股指期货里，`microprice` 偏移可能更稳定
- 在小盘股票或弱流动标的中，`spread` 扩张本身就可能支配全部结果

因此企业级体系应把微观结构模板设计成：

- 共性母版
- 市场特定校准层

而不是假设“一个公式 everywhere”。

### 59.14 哪些微观结构对象更适合极短 horizon

不是所有微观结构对象都适合拿去做分钟级甚至更长持有。一般来说，以下对象更偏向极短 horizon：

- `queue_pressure`
- `microprice`
- 超短期 OFI
- 即时 `spread` 扩张/收敛

它们的问题不是没用，而是：

- 信息半衰期极短
- 对延迟极敏感
- 对成交建模依赖很强

如果把这类对象强行延长持有期，常见结果是：

- 预测能力迅速衰减

## 59.A 深度隐式与高级动力学特征

前文探讨了大量基于量价、统计和规则的时序因子。然而，随着深度学习和复杂系统理论的引入，时序特征提取已经演化出四大高级流派。这些流派不看简单的“图形”，而是从信息论、频域、状态空间和微观动力学角度提取高阶暗知识。

在工业级数据表中，这些高级特征对应 `ts_signal_type` 的扩展枚举：`info_entropy`、`spectral_decomp`、`hidden_state`、`microstructure_flow` 和 `deep_latent_embed`。

### 59.A.0 滚动 Z-Score 与微观防漏机制 (Rolling Feature-wise Normalization)

在高频限价指令簿和分钟频环境中，直接使用绝对价格或全局 Z-score 会引入严重的未来函数 (Look-ahead Bias) 与非平稳性。必须采用严格的**滚动特征 Z-score 标准化**：
$$x_{\text{Z-score}}(t) = \frac{x(t) - \mu_w}{\sigma_w}$$

- **预处理机制**：通过一阶差分或对数收益率转换去除价格趋势，并利用滚动窗口自适应微观结构演变。
- **极端值处理 (Winsorization)**：强制约束在  $\pm 3\sigma$  个标准差范围内，防止错单交易或闪电崩盘导致局部异常放大。
- **日内季节性处理**：对开盘/收盘成交量激增等系统性偏差进行显式去趋势处理，防止噪音侵入模型。

### 59.A.1 信息论与熵 (Information Theory & Entropy)

核心思想不是预测涨跌，而是测算市场的“混沌度”。

- **核心算法**：样本熵 (Sample Entropy, SampEn)、近似熵 (ApEn)。
- **物理意义**：输出一个连续数值，数值越低说明序列自我相似度越高（被大资金主导，趋势明确）；数值越高说明序列越像随机游走（散户和噪音主导）。
- **实盘应用**：作为极品级的“风控门控因子 (Gating)”。当 1min 时序的样本熵极高时，自动关停趋势追踪策略；当熵值急剧下降时，意味着市场即将选择方向，可放大突破策略的杠杆。

### 59.A.2 频域与谱分解 (Spectral Decomposition)

普通的均线 (MA) 存在严重延迟 (Lag)，因为它把所有频率混在一起。

- **核心算法**：经验模态分解 (EMD) 或 奇异谱分析 (SSA)。
- **物理意义**：不预设固定周期，而是根据数据本身的波动，自动拆解为几个本征模态函数 (IMFs)：`原始价格 = 长期趋势 + 低频大周期 + 高频小周期 + 纯白噪音`。
- **实盘应用**：切掉高频噪音，只提取“低频大周期”的导数作为因子，在趋势真正转折的零延迟时刻 (Zero-Lag) 发出信号，是反转策略的利器。

### 59.A.3 隐马尔可夫与状态空间 (Hidden Markov Models, HMM)

市场并不连续，而是在几个“隐秘状态 (Hidden States)”之间切换。

- **核心算法**：隐马尔可夫模型 (HMM)。
- **物理意义**：通过贝叶斯概率论，根据近期收益率和波动率，反向推断当前市场处于某状态（如“恐慌抛售流动性枯竭”）的后验概率。
- **实盘应用**：作为顶级的离散状态因子 (Regime Factor)。下游模型无需猜涨跌，只需根据状态概率切换底层策略权重。

### 59.A.4 点过程与自激动力学 (Hawkes Processes)

这是高频交易和 1min 频独有的微观特征挖掘，用于建模订单的“传染病”效应。



- **核心算法**：霍克斯过程 (Hawkes Process)。
- **物理意义**：将每笔主动买卖单视为“事件点”，计算出“传染强度因子 (Intensity Rate)”。
- **实盘应用**：如果买单的霍克斯传染强度激增，说明大量算法交易正在被触发（自我激发）。因子会提前输出多看信号，捕捉微观盘口动能。

### 59.A.5 深度隐式特征 (CNN Embeddings) 与实盘推理流水线

不要期望深度学习模型直接输出交易信号。在企业级流水线中，视觉模型被当成“雷达阵列”。

- **产物形态**：CNN 在倒数第二层输出一组高维稠密特征向量 (Dense Embedding Vector，如 64 维浮点数组)，代表 K 线在抽象维度上的特征（如隐藏吸筹、逼空等）。
- **规范要求**：此类黑箱特征严禁直接参与单因子传统 IC 评估。模型端调用时，必须强制执行 PCA 降维或采用 Wide & Deep 双通道架构，实现白盒规则与黑箱暗知识的异构共振。

**\*\*实盘在线推理流水线 (Online Inference Pipeline)\*\***：在实盘高频环境中，系统必须在几十毫秒内完成从数据到开仓信号的全过程。必须严格拆分为四个物理步骤：

1. **\*\*流式窗口截取 (Streaming Window Buffer)\*\***：内存中仅维护定长队列（如长度 60）。新 K 线产生时推入队尾，挤掉老 K 线。
2. **\*\*实时二维图像映射 (On-the-fly GAF Transform)\*\***：用 C++/NumPy 算子，瞬间将队列映射为 GAF/MTF 热力图矩阵（当前市场的“X光片”）。
3. **\*\*CNN 模型前向传播 (Forward Pass Inference)\*\***：将图像送入预训练并固化 (Frozen) 的 CNN 模型。模型不训练只推理，在毫秒级吐出高维特征向量。
4. **\*\*下游决策引擎融合开火 (Downstream Fusion & Trigger)\*\***：隐形特征向量与传统量价因子、规则因子汇合，送入下游树模型 (LightGBM) 综合评估胜率，最终输出交易指令。

### 59.A.6 随机矩阵理论与高频协方差清洗 (RMT & Covariance Shrinkage)

在多品种分钟频投资组合中，高频采样伴随的微观噪音会严重污染真实的相关性结构。系统必须引入**随机矩阵理论 (RMT)** 进行清洗：

- **MP 分布定律**：纯噪音矩阵特征值分布于  $[\lambda_-, \lambda_+]$ ，其中  $\lambda_{\pm} = \sigma^2 (1 \pm \sqrt{N/T})$ 。
- **\*\*清洗与裁剪 (Clipping)\*\***：落入  $[\lambda_-, \lambda_+]$  的特征值视为噪音，平滑替换为算术平均值。仅保留大于  $\lambda_+$  的真实系统性主成分 (Latent Factors)。
- **重构**：重构去噪后的相关系数矩阵，结合高频 GARCH 或 ATR 波动率，生成稳健的协方差矩阵，避免优化器陷入过度配置。

### 59.A.7 动态风险平价与波动率缩放 (Risk Parity & Volatility Scaling)

为建立信号强度与实际资金分配的稳健映射，系统采用**动态风险平价框架**：

- **\*\*逆波动率加权 (Inverse Volatility Weighting)\*\***：确保每个品种对整个投资组合的边际风险贡献趋于均等。
- **权重映射公式**： $w_{i,t} = \frac{\text{Z-score}_{i,t}}{\sum_j |\text{Z-score}_{j,t}| / \sigma_{j,t}}$ 。
- **下行风险缓冲**：当日内波动率（如 15min ATR 或高频 GARCH）飙升时，系统在极短时间内自动削减该资产的名义敞口，控制左尾风险。

### 59.A.8 最优动态头寸规模控制 (Risk-Constrained Kelly)

在微观权重确定后，宏观杠杆由**风险受限的 Kelly 判据**决定：

- **理论核心**：最大化多期对数增长率，同时引入“回撤概率约束”。
- **凸优化求解**：不同于简单的“半仓 Kelly”，该模型通过凸优化求解满足  $P(\text{Drawdown} > \alpha) < \beta$  约束下的最优杠杆  $b$ 。

- **自适应性**：当市场波动率扩张或预期胜率衰减时，系统自动缩减整体资产包敞口上限，使回测曲线更平滑。

### 59.A.9 高频交易摩擦与无交易区间 (No-Trade Zone, NT-Zone)

针对高频重平衡带来的成本侵蚀，系统引入**多期凸优化与 NT-Zone 策略**：

- **摩擦模型 (3/2 次幂定律)\*\***： $\phi(z) = a|z| + b\sigma \frac{|z|^{3/2}}{(V/v)^{1/2}}$ ，捕捉买卖价差与非线性市场冲击成本。
- **无交易区间 (1/3 成本律)\*\***：最优缓冲带宽  $\Delta\theta \propto (\text{Cost})^{1/3}$ 。
- **执行逻辑 (DT-NT-DT)\*\***：当持仓落在 NT-Zone 内时“按兵不动”；只有越过边界时才触发指令，且仅调仓至缓冲带边缘，在追踪误差与摩擦成本间达到帕累托最优。

## 59.B 下游消费与 AI 融合架构

提取出几千个因子后，如果仅使用传统的 Barra 风险模型或线性加权，在百亿规模高频博弈中往往会失效。树模型 (Tree Models)、神经网络 (NN) 和强化学习 (RL) 在系统中的定位是完全不同的，它们是流水线上下游的协同关系。

### 59.B.1 树模型与神经网络：超级预测器

- **核心算法**：LightGBM, XGBoost, AlphaNet。
- **解决痛点**：
  1. **非线性特征交叉**：自动学习“条件触发逻辑”（如换手率极低时动量有效，极高时反转有效）。
  2. **门控机制 (Regime Gating)\*\***：将离散状态因子 (如 HMM 的 Cluster\_ID) 作为开关，在不同状态下自动切换底层因子权重。
  3. **共线性免疫**：自动在高度相关的因子中挑出最有效的，抛弃噪音。
- **定位**：负责把几千个散兵游勇的因子（连续、规则、形态、隐式向量），揉合成一个极其精准的综合预测分 (Master Rank Score / Expected Return)。

### 59.B.2 强化学习：首席交易员与风控官

强化学习 (RL) 的主战场不是预测涨跌，而是决策与控制 (Decision & Control)。

- **核心算法**：PPO, DDPG, AlphaQuanter, Janus-Q。
- **机制与演进**：
  - **工具增强单智能体 (AlphaQuanter)\*\***：模拟交易员推理闭环。动作空间解耦为“查询动作”（主动从微观流、基本面、情绪模块获取证据）与“决策动作”，提高微观冲击下的决策一致性。
  - **分层门控奖励模型 (HGRM)\*\***：将财富增长目标拆解为细粒度惩罚与激励。
    - **摩擦虚拟征税**：内生学习 DT-NT-DT 无交易区间法则。
    - **波动率感知门控**：遭遇结构性回撤风险时瞬间激活风险厌恶支路。
- **定位**：接收中游的 Alpha，结合实时账户资金和盘口深度，在充满摩擦的真实物理世界里，以利益最大化的方式完成资金分配和订单执行。

至此，时序因子从最底层的逐笔数据清洗、非时间 Bar 聚合，到微观动力学与 CNN 隐式特征提取，再到树模型预测和强化学习执行，形成了一套无可挑剔的百亿级量化工厂流水线。

- 成本占比上升
- 研究结果变成“看上去抓到了方向，实际上赚不到钱”

因此微观结构对象在入库时应同时记录：

- 推荐 horizon
- 推荐执行语义
- 对延迟的敏感等级

### 59.15 微观结构研究的最小检查清单

如果要让微观结构对象进入正式评估，建议至少完成以下检查：

1. 检查数据源是否稳定，是否存在快照缺失和时间漂移
2. 检查特征定义是否严格满足 `decision_ts`
3. 检查不同品种、不同 session 下的分布是否异常漂移
4. 检查理想价格口径和 `taker` 口径之间的落差
5. 检查一格、两格、三格延迟下是否迅速失效
6. 检查它更适合独立交易、gating 还是执行辅助

如果以上检查做不完，最好不要急着把微观结构对象推入主总库。因为微观结构对象不是不能做，而是特别容易做出“统计上非常漂亮、实盘上非常脆弱”的伪成果。

## 60. Maker / Taker / Queue / Partial Fill 执行仿真专章

前文讲了执行仿真，但还需要把最关键的执行物理学单独拉成一章。

### 60.1 Maker 和 Taker 的差异不是手续费那点事

很多研究员把 `maker` 和 `taker` 的差异理解成手续费表不同，这太浅了。更深层的区别在于：

- `taker` 可以立即成交，但要跨点差
- `maker` 可能拿到更好价格，但不一定成交
- `maker` 的真实收益取决于排队位置、撤单逻辑和成交概率

### 60.2 Taker 仿真

`taker` 仿真最基础，也最保守。典型规则：

- 做多按 `ask`
- 做空按 `bid`
- 平仓同理

这是高频策略最重要的生存底线。如果在 `taker` 语义下完全无法生存，研究层就不应该对其抱有过高预期。

### 60.3 Maker Optimistic / Conservative

为了评估只适合挂单执行的对象，建议至少保留两档：

- `maker_optimistic`
- `maker_conservative`

前者可以近似认为：

- 挂单后较高概率成交

后者则更保守：

- 成交概率较低
- 有明显 missed fill
- 可能因为价格跳走而丢失机会

### 60.4 Queue Position

队列位置在真正的高频执行里至关重要。即使价格触及到你的挂单价位，如果你排得太靠后，也可能根本轮不到你成交。

因此工程上即便做不到完整撮合级仿真，也建议给出近似的：

- `queue_position_score`
- `fill_probability_estimate`
- `missed_fill_rate`

## 60.5 Partial Fill

部分成交问题不能忽略。许多分钟频信号在理论上是全仓翻转，但实际只能成交一部分。如果回测里默认全部成交，会系统性高估策略表现。

建议回测引擎至少能模拟：

- 全部成交
- 部分成交
- 未成交

并记录：

- `fill_ratio`
- `partial_fill_loss`
- `missed_opportunity_cost`

## 60.6 哪些因子必须标成 `maker_only`

如果一个因子存在以下特征，应优先考虑打上 `maker_only`：

- `taker` 执行下净收益稳定为负
- 预测 horizon 极短
- spread 占收益空间的比例极高
- 延迟一点点就明显失效

## 60.7 为什么执行仿真必须进入主评估主线

因为在分钟频领域，“执行”不是下游附属工程，而是因子质量本身的一部分。如果回测层不把：

- `maker / taker`
- `queue`
- `partial fill`
- `delay`

纳入主线，那么研究层会长期高估因子真实质量。

## 60.8 执行仿真里最容易被忽略的乐观假设

很多团队第一次做分钟频执行仿真时，最容易不自觉地加入乐观假设。例如：

- 默认每次都能全部成交
- 默认成交发生在最优价位
- 默认订单不会因为队列靠后而落空
- 默认网络延迟稳定且很低

这些假设之所以危险，是因为它们每一个都会把结果往更好方向推一点点。多项乐观假设叠加后，最终回测结果可能和真实交易环境完全不是同一个世界。

## 60.9 为什么同一因子需要多套执行视角

一个成熟的分钟频评估报告，最好同时展示：

- 理想价格视角

- `taker` 视角
- `maker optimistic`
- `maker conservative`

因为不同视角回答的问题不同：

- 理想视角回答“统计上有没有东西”
- `taker` 视角回答“最保守执行下活不活”
- `maker` 视角回答“若配合更复杂执行架构是否还有价值”

## 60.10 执行仿真不只是给执行团队看的

研究员也必须理解执行仿真，因为：

- 很多所谓“优化公式”其实不如优化执行方式
- 很多高换手对象不是没有信息，而是执行方式错了
- 很多 `maker-only` 对象更适合作为特种策略，而不是普通总库对象

如果研究员不理解执行仿真，就会不断把不适合主总库的对象往主总库推。

## 60.11 延迟建模为什么要分层

很多团队在执行仿真里只做一个“固定延迟”，例如统一延迟一根 `bar`。这比完全不做要好，但还不够。

更合理的做法是把延迟拆成几层：

- 信号计算延迟
- 数据落地延迟
- 网络传输延迟
- 订单路由延迟
- 交易所撮合等待

在研究报告里，不一定要求每一层都能极精确建模，但至少应该区分：

- 理想零延迟
- 研究可接受延迟
- 生产保守延迟

这样才能看清楚一个对象到底是：

- 对任何小延迟都脆弱
- 还是只对极端延迟脆弱

这两类对象的工程含义完全不同。

## 60.12 Maker 成交概率不能只写成一个常数

把 `maker_fill_probability` 固定成某个常数，是执行仿真里很常见的偷懒方式。但真实成交概率通常取决于：

- 当前 `spread`
- 队列位置
- 盘口补单速度
- 主动单到达强度
- 价格是否马上逃离

因此更成熟的做法是让成交概率至少依赖几类上下文变量，而不是固定写死。即使暂时没有完整撮合引擎，也可以做分层近似：

- 高流动性时段高成交概率
- 低流动性时段低成交概率
- 队列靠前高成交概率
- 队列靠后低成交概率

其意义在于，让研究结论更接近真实世界，而不是把 `maker` 结果建立在静态乐观假设上。

### 60.13 撤单逻辑为什么必须写进仿真

挂单策略不只是“挂了会不会成交”，还包括“何时撤、何时改价、何时转 `taker`”。如果仿真没有撤单逻辑，那么：

- 你可能虚构了大量本不会继续挂着的订单
- 也可能低估了实际会追价成交的成本

至少应定义几类基础策略：

- 固定等待后撤单
- 条件触发撤单
- 部分成交后继续挂剩余量
- 超时后转 `taker`

这些规则一旦不同，最终净收益和 `missed fill` 统计会差很多。所以执行仿真从来不是“手续费表 + 点差”那么简单。

### 60.14 执行仿真最终应该输出哪些字段

为了让执行仿真能真正进入主评估主线，建议标准输出至少包含：

- `gross_return`
- `net_return`
- `spread_cost`
- `fee_cost`
- `market_impact_proxy`
- `delay_bucket`
- `fill_ratio`
- `missed_fill_rate`
- `partial_fill_loss`
- `maker_flag`
- `queue_score`

如果是多视角评估，还应支持：

- `ideal_view`
- `taker_view`
- `maker_optimistic_view`
- `maker_conservative_view`

这些字段不是给前端“堆参数”用的，而是为了支持统一的研究、评估、入库和复盘流程。没有统一字段，后续不同团队对同一个因子的执行语义会出现版本分叉。

### 60.15 什么样的执行结果应该直接阻止推进

执行仿真不是“补充说明”，它本身就可以成为否决条件。以下情况通常应直接阻止推进：

- 理想视角很好，但 `taker` 稳定为负
- 一格延迟后核心指标断崖式下降
- `maker` 结果完全依赖极乐观成交率
- `missed_fill_rate` 极高，策略可实现性很弱

- 结果只在极少数流动性最好的时段成立

这类对象未必完全没价值，但通常不适合作为普通总库因子。它们更可能属于：

- 特种执行策略研究
- 限定场景条件策略
- 仅作模型上下文输入

如果团队缺少这条纪律，就会不断把“只能在假设世界里赚钱”的对象推进到生产准备阶段。

## 60.16 研究、执行、风控为什么必须看同一份执行报告

执行仿真不应该被拆成三个平行世界：

- 研究看理想收益
- 执行看成交细节
- 风控看极端场景

更好的组织方式是三方共用同一份底层报告，只是各自关注不同字段。原因很简单：

- 研究决定要不要继续推
- 执行决定能不能落地
- 风控决定最坏情况下会发生什么

如果三方看的是三套口径不同的报告，最终一定会出现：

- 研究觉得可以做
- 执行觉得成交不了
- 风控觉得尾部太差

并且谁也无法说明分歧到底来自哪里。统一执行报告是组织协作问题，不只是回测引擎问题。

## 61. 自动化挖掘源头控换手专章

前文已经说过要从源头惩罚高换手，但这里要把它真正写成一套可执行方法。

### 61.1 为什么不能只靠后处理降换手

如果一个自动化生成器在搜索时只追求：

- 高 TS-IC
- 高短期收益

而完全不管：

- 状态切换频率
- 零轴来回闪烁
- 短时噪声敏感性

那么它生成的大部分结果，天然就会偏向高换手和高脆弱性。此时后处理再怎么修，也常常只是亡羊补牢。

### 61.2 Reward 里怎么直接惩罚换手

建议在自动化挖掘 reward 中显式加入：

```
Reward = a * predictive_score
         + b * net_pnl_score
         - c * turnover_penalty
```

```
- d * flip_penalty
- e * latency_sensitivity_penalty
- f * spread_sensitivity_penalty
```

其中：

- `turnover_penalty` 惩罚平均换手
- `flip_penalty` 惩罚短时间内的方向反复切换
- `latency_sensitivity_penalty` 惩罚 delay 一格就崩的对象

### 61.3 源头模板就要有迟滞结构

自动化生成模板不应只包含连续值输出，也应直接支持：

- `deadband`
- `hysteresis`
- `state machine`
- `minimum_holding`
- `time_of_day_gate`

这样生成器才有机会直接发现“可交易的信号结构”，而不是永远先发现高噪声连续信号，再交给下游修。

### 61.4 不鼓励零轴附近高频翻转

自动化挖掘系统可以直接对以下结构降权：

- 零轴穿越极其频繁
- 小幅噪声就切换方向
- `signal` 的局部 Lipschitz 过高
- 对很小输入扰动极敏感

### 61.5 搜索空间也要控换手

搜索空间本身就决定会不会天然生成高换手对象。

如果一开始允许：

- 过短 `lookback`
- 过高灵敏度
- 任意 `threshold`
- 没有持有期约束

那系统几乎一定会倾向于产出高换手近邻因子。

建议在搜索空间层面就控制：

- `lookback` 最小值
- `threshold` 平滑范围
- 最短持有期
- 允许的状态切换上限

### 61.6 自动化矿机的反馈闭环

对于 `alphaprobe`、`quantaalpha` 这类自动化矿机，建议保留以下反馈对象：

- `turnover_failure_modes.json`
- `flip_pattern_report.json`



- `latency_fragility_report.json`
- `spread_kill_report.json`

这些对象可以回流给下一轮搜索，形成真正的“反高换手学习”。

## 62. 时序入模前细则专章

虽然更细的模型端实现规范已经拆到单独文档里，但这份时序总文档仍然应该保留一版“时序研究视角的入模前说明”。

### 62.1 为什么时序特征进入模型前比横截面更危险

因为时序模型最容易被这些问题污染：

- 非平稳漂移
- 样本窗口重叠
- 序列打包错误
- label 泄露
- 训练和推理的特征口径不一致

### 62.2 Sliding Window 的研究含义

时序入模不是简单把因子拼成二维表，而是经常要构造成：

```
[Batch, Time_Steps, Num_Features]
```

这里最关键的不是张量形状本身，而是：

- `Time_Steps` 是多少
- 是否包含未来信息
- 不同窗口之间是否严重重叠
- horizon 是否和标签对齐

### 62.3 ADF 与平稳性检查

ADF 不意味着“显著才是好特征”，但它能帮助研究员识别：

- 这个特征是不是存在强漂移
- 是否只是长期趋势项的反映
- 是否需要差分或分数差分

### 62.4 Fractional Differencing 的位置

它更适合被看作：

- 一个研究工具
- 一个特征工程可选项

而不是所有时序因子的默认入口。

### 62.5 哪些对象更适合作为模型端 feature

通常更适合入模的是：

- 稳定的连续特征
- regime 概率向量

- pattern score
- 成本与流动性上下文变量

而不是未经治理的最终交易信号。

## 63. 时序总库规则引擎表

最后把前面很多散落规则，收成一张更像“规则引擎输入”的表，方便后面真落系统。

| 规则组        | 触发条件                        | 默认动作                                      |
|------------|-----------------------------|-------------------------------------------|
| 生存失败       | net_return_after_spread < 0 | 直接归档或仅保留研究记录                              |
| 延迟脆弱       | latency_degradation 过大      | 打 latency_hyper_sensitive，进入执行审查          |
| maker 约束   | taker 负、maker 正             | 打 maker_only，禁止默认生产使用                     |
| 高换手        | turnover 极高且无足够补偿           | 路由 Tier 2X，优先做 deadband / hysteresis      |
| 时段脆弱       | 只在少数时段有效                    | 打 session_sensitive，建议 time_of_day_gating |
| regime 条件化 | 只在某 regime 有效               | 打 regime_conditioned，仅作条件性因子              |
| 家族冗余       | family_corr 过高              | 家族内排序，少量回总库，其余进 Tier 3D                   |
| 基准风险重合     | 正交化后大幅失效                    | 降级为上下文变量或储备对象                             |

### 63.1 这张表的意义

这张表不是为了好看，而是为了后续你把它回填到主需求文档时，能直接映射成：

- admission policy
- routing rule
- optimization trigger
- execution constraint

因此，本章内容本质上即为未来规则引擎的配置母版。

## 64. 工业级评估门槛与建议阈值

报告里有一个很重要但还没有被我完全写透的点，就是分钟频时序因子的很多指标不能只“定义出来”，还需要给出工业语境下的参考阈值。这里先给出一版建议口径。需要强调的是，这些阈值不是永恒真理，而是企业级系统里的第一道门槛和预警线。

### 64.1 预测能力门槛

| 指标          | 建议观察方式         | 参考阈值           | 说明                |
|-------------|----------------|----------------|-------------------|
| TS-IC       | 样本外滚动均值        | 稳定大于 0.03~0.05 | 分钟频里这个量级已经可能有经济价值 |
| Rank TS-IC  | 滚动 + 分 session | 稳定为正           | 对抗极端值污染           |
| IC Win Rate | 正向窗口占比         | > 55% 更稳       | 帮助判断普适性           |

|                      |          |              |                |
|----------------------|----------|--------------|----------------|
| Directional Accuracy | 总体 + 分阈值 | 53%~55%+ 可关注 | 单看这个不够，要配合盈亏比  |
| Threshold Hit Rate   | 多阈值扫描    | 随阈值升高不能快速塌陷  | 极端信号真实性很关键     |
| OOS R^2              | 纯样本外     | 稳定为正即可       | 高频环境里小的正值也可能有用 |

### 64.2 收益风险门槛

| 指标            | 参考阈值                          | 说明              |
|---------------|-------------------------------|-----------------|
| net_sharpe    | > 1.5 可研究, > 2.0 较强, > 3.0 很强 | 必须是扣主要成本后的净口径   |
| sortino       | > 2.0 更稳                      | 对非正态收益更友好       |
| calmar        | > 1.5 可关注, > 2.0 更优           | 适合看分钟频回撤生存度     |
| profit_factor | > 1.5 勉强, > 1.75 更稳, > 2.0 很好 | 高频里要留足摩擦安全垫     |
| GPR           | > 1.5 较好, > 2.0 很好            | 看利润积累质量         |
| Ulcer Index   | 越低越好                          | 需要结合收益看, 不能单独判断 |

### 64.3 执行与摩擦门槛

| 指标                          | 参考门槛            | 说明                                |
|-----------------------------|-----------------|-----------------------------------|
| turnover                    | 没有绝对阈值, 必须与收益联动 | 分钟频里高换手不是原罪, 但必须有补偿               |
| latency_degradation         | 延迟一格后不应出现灾难性崩塌  | 如果掉 50% 以上, 要高度警惕                 |
| bid_ask_crossing_cost_share | 不应吞掉大部分毛收益      | 若点差吞噬主要利润, 基本不可交易                 |
| maker_dependency_ratio      | 越低越好            | 若必须完全依赖 maker optimistic 才成立, 要降级 |

### 64.4 同质化和入库门槛

| 指标                       | 参考口径              | 说明             |
|--------------------------|-------------------|----------------|
| library_corr_max         | 不宜长期过高            | 与库内过高重合说明增量不足  |
| family_corr              | 应明显低于父因子近邻家族的其他版本 | 否则只是参数近邻       |
| family_rank_within_batch | 应进入家族前列           | 不进入前列不建议抢占总库席位 |
| incremental_ir           | 应明确为正             | 不然没有必要新增总库资产   |

### 64.5 这些阈值怎么用

这些阈值最适合做三件事：

- 预警线
- 路线
- 详情页红黄绿灯

不建议把它们简单理解成“单一硬门槛全部卡死”。更合理的方式是多维联合判断，例如：

- TS-IC 一般，但 `net_sharpe`、`profit_factor`、`latency_degradation` 很稳，仍可能是好因子
- 胜率很好，但 `bid-ask crossing cost` 太高，则仍不该直接入可交易总库

## 65. 高频时序因子的正交化与基准剥离

报告里还强调了一个非常工业的问题：很多新因子看上去有效，其实只是已知高频风险暴露或市场状态的再包装。这一点必须单独成章。

### 65.1 为什么要做时序正交化

分钟频时序因子特别容易和以下对象混在一起：

- 市场方向 `beta`
- 波动率水平
- 流动性压力
- 开盘冲击
- 尾盘重平衡效应

如果不剥离这些，系统很容易把：

- 市场普遍趋势
- 流动性时段效应
- 已知微观结构偏误

误判成“新 `alpha`”。

### 65.2 可以对哪些基准做剥离

建议至少考虑以下高频基准：

- `market_direction_proxy`
- `volatility_regime_proxy`
- `spread_regime_proxy`
- `session_dummy`
- `liquidity_stress_proxy`
- `opening_range_proxy`

### 65.3 滚动正交化比静态正交化更合理

高频环境中的相关结构变化很快，因此正交化不应完全依赖静态样本期拟合。更合理的做法是：

- 训练窗内做滚动回归
- 每个窗口更新 `beta`
- 预测期只复用已知 `beta`

### 65.4 正交化后的评估结果应该怎么看

如果一个因子在正交化后：

- 依然保留稳定的 `TS-IC`

- 依然保留稳定的净收益
- 和家族对象的同质化明显下降

那它更有资格被视为真正增量  $\alpha$ 。

如果正交化后几乎全部消失，则更应被理解为：

- 状态代理
- 风险代理
- 或执行上下文变量

而不是总库新主因子。

## 66. 夜盘、跨时区与 Session 切片细则

这一块在报告里的工业意味很强，我认为当前文档也还需要更明确。

### 66.1 夜盘不是简单跨日

对期货来说，夜盘会带来：

- 日期切换
- 流动性结构切换
- 宏观消息窗口切换
- 亚洲与欧美时区接力

因此夜盘数据不能简单地按自然日切断。建议至少显式保留：

- `trading_session_id`
- `session_open_ts`
- `session_close_ts`
- `is_night_session`

### 66.2 跨时区切片必须进入评估主线

对于 `crypto` 和全球交易资产，建议至少按：

- 亚洲时段
- 欧洲时段
- 北美时段
- 时区交叠时段

分别出具：

- `TS-IC`
- `net_sharpe`
- `turnover`
- `cost_share`

### 66.3 Session Dummy 为什么是重要上下文

许多分钟频因子并不是对“价格路径本身”有预测，而是对“某个时间段的典型交易机制”有响应。如果不显式加入 `session` 上下文，就会导致：

- 模型端误学
- 评估端误判
- 优化器误改

## 67. 分钟频详情页的工业级图表清单

报告里的很多工业评估点，最后都应该落到详情页图表里，否则团队很难真正消化这些结论。

建议时序因子详情页至少支持以下图表：

- gross\_vs\_net\_equity\_curve
- holding\_period\_decay\_curve
- latency\_degradation\_curve
- threshold\_hit\_rate\_heatmap
- session\_slice\_performance\_bar
- regime\_slice\_performance\_bar
- spread\_sensitivity\_curve
- turnover\_vs\_return\_scatter
- family\_neighbor\_comparison
- maker\_vs\_taker\_pnl\_compare
- drawdown\_duration\_curve
- cost\_breakdown\_stack

### 67.1 为什么图表层也是知识层

这份时序文档未来会发给组员吸收，因此光有公式和表格还不够。分钟频很多问题不看图根本发现不了：

- 哪一段时间赚钱
- 延迟一格会不会崩
- 换手是不是只集中在某个 regime
- family 近邻里到底谁是真正代表版本

所以图表层不只是前端体验层，也是团队认知层。

## 68. 进一步补录报告内容的计划收口

到当前这一步，报告里最核心、最工业化、最值得吸进来的结构性内容，已经大部分转化成了正式章节。后面继续补的重点，将不再是“有没有大块没写”，而更多是：

- 哪些章节还能继续加厚
- 哪些公式还能再补齐
- 哪些工业案例还能继续扩展

至此，本文档已从基础的时序因子框架说明，演进为完整的企业级时序因子实施总册。

## 69. 分钟频回测协议详细规范

前文已经多次提到执行仿真和成本评估，但如果要让团队未来真按这份文档写代码，分钟频回测协议还需要进一步细化。原因很简单：分钟频策略的回测误差，往往不是几个百分点，而是能直接把一个看似漂亮的因子从正收益打成负收益。

### 69.1 回测协议的最小输入

分钟频回测不应只接收一个 `factor_value` 列和一系列未来收益。至少应有：

- event\_ts
- decision\_ts
- signal\_value

- `signal_state`
- `mid_price`
- `bid_price`
- `ask_price`
- `last_trade_price`
- `volume`
- `spread`
- `session_id`
- `is_night_session`
- `latency_profile`
- `commission_profile`

如果是盘口和更高频对象，还应有：

- `book_snapshot_ref`
- `queue_estimate`
- `fill_probability`

## 69.2 价格口径必须并行保留

分钟频回测至少要同时支持以下价格口径：

- `mid_price`
- `last_trade_price`
- `bid_ask_crossed_price`
- `maker_fill_price`

这样做不是“学术洁癖”，而是为了清楚区分：

- 理想世界
- 实际成交世界
- 被动挂单世界

## 69.3 信号到持仓的映射协议

回测引擎必须清楚地区分三个层级：

- `factor_value`
- `signal_state`
- `position_target`

因为：

- 因子值可能是连续值
- 信号状态可能是 `[-1, 0, 1]`
- 最终持仓目标可能还要叠加风控、仓位预算、session 限制

建议保留以下映射对象：

- `signal_mapping_policy`
- `position_sizing_policy`
- `risk_override_policy`

## 69.4 延迟模型

分钟频策略的延迟不应只模拟一个固定值。建议至少支持三层：

- 固定 1 bar 延迟

- 固定若干秒延迟
- 随时间段和市场状态变化的延迟

原因在于：

- 开盘和尾盘的真实延迟影响不同
- 高波动时段的撮合和网络负载也不同

## 69.5 手续费和滑点模型

最简单的手续费模型是：

```
fee_cost_t = traded_notional_t * fee_rate
```

但在分钟频里，仅靠手续费远远不够，还需要：

- 跨点差成本
- 冲击成本
- 参与率惩罚
- 部分成交和 missed fill 损失

## 69.6 订单执行类型

回测协议建议显式支持：

- market
- aggressive\_limit
- passive\_limit
- maker\_only

并记录：

- execution\_type
- fill\_ratio
- effective\_spread\_paid
- queue\_position\_score

## 69.7 Session 切片是回测主线，不是附录

分钟频回测报告默认就应拆：

- 开盘段
- 上午段
- 午盘段
- 尾盘段
- 夜盘段
- 重大事件窗口

因为很多分钟频因子只在少量时段有价值，如果回测协议不直接支持切片，研究员后面就会靠手工分析，既慢又容易出错。

## 69.8 回测协议输出产物

建议分钟频回测至少输出：

- execution\_trace.parquet
- fill\_log.parquet



- latency\_degradation.json
- cost\_breakdown.json
- session\_slice\_report.json
- regime\_slice\_report.json
- maker\_taker\_compare.json
- delay\_sensitivity\_curve.json
- hold\_decay\_curve.json

## 70. 高频微观结构噪音与伪信号识别

很多团队讨论分钟频因子时，只谈“有没有 alpha”，很少谈“哪些是噪音伪装成 alpha”。实际上，高频世界最难的往往不是构造信号，而是识别伪信号。

### 70.1 买卖价差反弹伪信号

最经典的一类伪信号是 bid-ask bounce。如果系统直接用成交价计算极短期收益，会把买卖价切换带来的机械跳动误认为价格有方向性。

典型症状：

- 超短时窗反转异常好看
- 一旦改用 mid-price 或扣跨点差，效果骤降

### 70.2 零成交与僵死 bar 伪信号

低活跃时段，某些分钟可能没有成交。这时：

- 前向填充是必要的
- 但长期 stale 值会让某些因子产生“稳定有效”的错觉

因此系统需要单独记录：

- stale\_length
- no\_trade\_bar\_count
- forward\_fill\_ratio

### 70.3 异常 spread 放大伪信号

有时信号之所以看起来有效，只是因为 spread 在某时段极端放大，而价格和盘口的真实信息并没有同步改善。

系统应单独诊断：

- 因子表现是否依赖高 spread 区间
- spread 收敛后是否还能保留预测力

### 70.4 Session 偏差伪信号

有些因子本质上只是：

- 开盘流动性冲击
- 午盘僵尸行情
- 尾盘重平衡

而不是普遍的分钟频 alpha。如果不拆时段，它们会伪装成稳定因子。

### 70.5 市场大 beta 伪信号

如果某因子只是在：

- 上涨市里顺着趋势赚钱
- 下跌市里反着亏

它可能只是一个市场方向代理，而不是独立 alpha。正交化和 regime 切片正是为了识别这类伪信号。

## 70.6 数据对齐错误伪信号

最危险也最隐蔽。

包括：

- merge 时把未来慢变量提前广播
- 用未来窗口统计量
- 盘口和成交时间戳错位
- 不同交易所数据未统一时区

这类错误常常会制造“极其漂亮但根本不可能实盘”的结果。

## 71. 资产与场景长案例扩写

前文已有几个案例，但如果要把这份文档发给组员吸收，还需要再扩成更完整、更接近工业情境的长案例。

### 71.1 案例 D：股指期货分钟频趋势因子从研究到入库

场景设定：

- 标的：股指期货主力合约
- 原始数据：逐笔成交 + 盘口快照
- 采样方式：volume bars
- 目标：捕捉日内趋势延续

第一步，Phase1 先构造底层微观结构特征：

- signed\_volume\_imbalance
- run\_length
- realized\_vol\_short
- queue\_pressure

第二步，merge 更慢的上下文变量：

- session\_id
- daily\_vol\_regime
- opening\_range\_breakout\_state

第三步，Phase2 构造高层表达：

- trend\_start\_score
- trend\_persistence\_score
- regime\_filtered\_trend\_score

第四步，进入评估：

- TS-IC
- Threshold Hit Rate
- Holding Period Decay
- Latency Degradation
- gross vs net

- session slice

如果结果显示：

- 预测强
- 净收益还可以
- 但开盘前 15 分钟换手爆炸

则系统应：

- 打 needs\_time\_of\_day\_gating
- 路由到 Tier 2X
- 尝试 deadband、最短持有期和时段过滤

如果优化后：

- net\_sharpe 提升
- turnover 明显下降
- library\_corr\_max 没过高

才允许少量版本进入总库。

## 71.2 案例 E：Crypto 波动切换因子

场景设定：

- 标的：BTC
- 原始数据：逐笔成交
- 采样方式：realized vol bars
- 目标：捕捉波动压缩后的释放

研究链：

- Phase1 用 realized vol bars 表达信息事件
- 底层特征聚焦：
  - 波动率压缩
  - 成交额爆发
  - spread 变化
  - order flow imbalance
- merge 更慢的上下文：
  - 全球时区 session
  - 长窗口 vol regime
  - 过去若干小时趋势方向

评估重点：

- breakout 前后的 Threshold Hit Rate
- 亚洲、欧洲、北美时段切片
- maker / taker 差异
- 高 spread 阶段敏感性

可能出现的结论：

- 因子只在欧美重叠时段强
- taker 语义下边际不足
- maker 下仍能存活

此时合理的路由不是直接入主总库，而是：

- 标 maker\_only
- 标 session\_sensitive
- 先入 Tier 3D 或条件性总库

71.3 案例 F：个股日内开盘区间模式对象

场景设定：

- 标的：高流动性美股
- 原始数据：1 分钟 bar + 开盘前补充上下文
- 目标：识别 opening range breakout 是否真实

这个对象最好的定位，往往不是直接信号，而是：

- pattern authenticity score

工程上应做：

- 单独评估其作为 gating 特征时的价值
- 比较：
  - 不加该 score 的趋势策略
  - 加该 score 过滤后的趋势策略

这样往往更能说明它的真实工业价值。

72. 统一评估口径与字段责任表

如果未来团队要完全按文档复现系统，只有指标和规则还不够，还必须明确“谁生产、谁消费、字段怎么解释”。

72.1 关键字段责任表

| 字段                     | 含义            | 谁生产    | 谁消费        |
|------------------------|---------------|--------|------------|
| TS_IC                  | 时序信息系数        | 评估引擎   | 准入、前端、优化工厂 |
| threshold_hit_rate     | 阈值命中率         | 评估引擎   | 准入、详情页     |
| latency_degradation    | 延迟损失          | 执行仿真层  | 准入、执行约束层   |
| maker_dependency_ratio | 对 maker 的依赖程度 | 执行仿真层  | 准入、标签层     |
| family_corr            | 家族内重合度        | 同质化分析层 | 准入、优化工厂    |
| session_slice_report   | 分时段表现报告       | 评估引擎   | 准入、详情页     |
| regime_slice_report    | 分状态表现报告       | 评估引擎   | 准入、优化工厂    |
| cost_breakdown         | 成本拆解          | 执行仿真层  | 准入、详情页     |

72.2 统一口径为什么比公式本身更重要

很多上下游分歧，不是因为不懂公式，而是因为：

- 有人用毛收益
- 有人用净收益

- 有人用 mid-price
- 有人用 bid/ask crossed
- 有人用固定延迟
- 有人没算延迟

如果不先统一口径，后面的所有讨论都会反复打架。

## 73. 从知识文档到工程实现的最后一层映射

这份文档最终还是要服务团队吸收和后续工程实现，所以最后再补一层“知识 -> 系统模块”的映射。

### 73.1 研究层需要拿走什么

- 分类体系
- 模式识别方法库
- 微观结构因子库
- 高换手治理思路
- 案例和误区

### 73.2 评估层需要拿走什么

- 49-57
- 64-67
- 69-72

这些章节可以直接演化成：

- 指标字典
- 评估 DAG
- 详情页图表产物
- Admission Rule 配置

### 73.3 优化工厂需要拿走什么

- 39-41
- 61
- 63

这些章节可以直接演化成：

- 优化动作模板
- reward 约束
- source family 管理
- 路由规则

### 73.4 模型层需要拿走什么

- 16
- 62

它们帮助模型组理解：

- 哪些时序对象适合作为 feature
- 哪些只是 gating 或 regime 上下文
- 入模前应关注哪些风险

## 74. 当前版本的整体完成度说明

到这里为止，这份时序因子总说明文档已经不再只是一个简短框架，而是一个相当厚的企业级长说明书：

- 有基础认知
- 有工业生产链
- 有非时间聚合
- 有模式识别和状态识别
- 有自动化挖掘
- 有优化与高换手治理
- 有时序评估母版
- 有执行仿真
- 有微观结构专章
- 有规则引擎和详情页图表清单
- 有案例、术语表、检查清单和实施路线图

后续当然还能继续扩得更长，但当前已经具备“发给组员系统吸收”的基础形态，而且后面回填主需求文档和 HTML 时，也已经有足够厚的母版可以抽取。

## 75. 分资产时序因子 Playbook

为了让组员真正拿这份文档做工作，不仅要知道“通用方法论”，还要知道不同资产上应该优先考虑什么、避开什么。这里给出一版企业级 `playbook`。

### 75.1 Futures Playbook

#### 优先研究对象

- 趋势延续
- 波动压缩释放
- 订单流推进
- 日盘/夜盘切换
- 主力合约换月附近的结构变化

#### 优先使用的数据

- 逐笔成交
- 最优 bid/ask
- 多档盘口
- 主连与可交易合约映射
- session 元信息

#### 最值得优先验证的指标

- `Latency Degradation`
- `Holding Period Decay`
- `session_slice_sharpe`
- `net_return_after_spread`
- `family_corr`

#### 最常见死亡原因

- 夜盘和日盘逻辑混在一起
- 换月处理错误
- 趋势收益只是市场方向  $\beta$
- 点差和滑点没扣够

### 75.2 Crypto Playbook

#### 优先研究对象

- 波动率状态切换
- 成交额推进
- 交易所间流动性差异
- 24 小时时段轮动
- breakout 与 squeeze release

#### 优先使用的数据

- 逐笔成交
- 交易所级行情
- 订单簿快照
- 资金费率和链上慢变量

#### 最值得优先验证的指标

- session\_slice\_performance
- maker\_vs\_taker\_pnl\_compare
- spread\_sensitivity\_curve
- regime\_stability
- OOS  $R^2$

#### 最常见死亡原因

- 只在某一交易所有效
- spread 和手续费高估不足
- 24/7 时段差异被忽略
- 波动环境切换后失效

### 75.3 Equity Intraday Playbook

#### 优先研究对象

- opening range behavior
- intraday reversal
- liquidity vacuum rebound
- 尾盘再平衡
- 新闻冲击后结构修复

#### 优先使用的数据

- 1 分钟 bar
- 逐笔成交
- 开盘竞价 / 尾盘集合竞价上下文
- 盘口快照

#### 最值得优先验证的指标

- 开盘 / 午盘 / 尾盘切片表现
- Threshold Hit Rate
- latency\_degradation
- market\_beta\_orth\_residual
- maker\_dependency\_ratio

#### 最常见死亡原因

- 只是开盘机制的时段代理
- 个股流动性断裂没有过滤
- beta / 行业冲击没剥离

- 回测误用了理想成交价

## 75.4 Options Playbook

### 优先研究对象

- 隐含波动率状态
- skew / term structure 变化
- gamma 压力
- 期权成交与标的联动

### 最需要警惕

- 合约滚动
- 到期日效应
- 隐波曲面漂移
- 流动性极不均匀

### 建议定位

期权时序因子更适合做高阶研究专题，不建议作为团队第一批落地对象。

## 76. 不同频率的时序因子 Playbook

同样是时序因子，不同频率的物理世界完全不同。这里按频率补一版工作手册。

### 76.1 Daily

特点：

- 噪声较低
- 执行压力较低
- 更适合结合慢变量和 regime

更适合：

- 趋势类
- 波动率状态类
- 多天持有的条件性 timing

### 76.2 1min

特点：

- 是工程复杂度和样本丰富度的一个平衡点
- 已经明显受 spread、latency、session 影响
- 也是当前最值得优先工业化的频率之一

更适合：

- intraday trend
- breakout / squeeze
- 微观结构增强型因子
- session-sensitive 模式对象

### 76.3 5min / 15min

特点：



- 比 1min 更稳
- 换手更可控
- 对大多数研究团队更友好

很多团队最终实盘落地，会发现 5min 或 15min 比 1min 更有性价比。

## 76.4 Tick / Snapshot

特点：

- 信息最丰富
- 工程和执行成本最高
- 最容易被伪信号污染

建议定位：

- 作为研究上游和 Phase1 输入层
- 不宜直接成为大团队默认策略输出层

## 77. 失败反例库

这份文档如果只写成功方案，会让人误以为时序因子世界是线性的。这里补一组典型反例，帮助团队建立“哪里最容易死”的直觉。

### 77.1 反例 A：超短窗口反转因子

构造：

- 用过去 2 到 3 个 bar 的收益反向做信号

表面现象：

- 样本内方向准确率很高
- 频繁翻转
- 毛收益不错

真实问题：

- bid-ask bounce
- spread 一扣全没
- latency 一加就崩

结论：

这类对象必须先经过中间价口径重评、跨点差重评和死区优化，否则不应进入可交易库。

### 77.2 反例 B：只在开盘前 15 分钟有效的“全天因子”

表面现象：

- 全天平均 TS-IC 为正
- 回测曲线看起来还行

拆开后发现：

- 盈利几乎全部来自开盘极短窗口
- 其他时段接近随机甚至为负

结论：

它应被重定义为：

- `open_drive` 或 `opening_range` 条件对象

而不是泛化为全天候总库因子。

### 77.3 反例 C：只在 `maker optimistic` 下成立

表面现象：

- 回测很亮眼

问题：

- 一切都依赖理想挂单成交
- `queue` 稍微保守一点，收益大幅回落

结论：

应该打：

- `maker_only`
- `queue_position_critical`

并降级为更受限的使用方式。

### 77.4 反例 D：只是市场趋势 `beta`

表面现象：

- 单看收益很强
- 单看方向准确率也不错

问题：

- 高波趋势日赚钱
- 震荡时几乎没有独立能力
- 正交化后大幅衰减

结论：

它更像市场 `beta` 的高频投影，不应被当成独立时序 `alpha` 入总库。

## 78. 数据质量与审计专章

企业级时序体系的另一块底盘，是数据审计。很多分钟频因子的问题，根本不是公式问题，而是数据层的问题。

### 78.1 必须审计的对象

- 原始 tick 完整率
- 盘口快照丢包率
- 时区统一情况
- session 切换正确率
- 合约映射正确率
- forward fill 比例
- stale bar 比例
- merge 泄露风险

78.2 建议审计产物

- tick\_data\_quality\_report.json
- book\_snapshot\_integrity\_report.json
- session\_alignment\_report.json
- forward\_fill\_audit.json
- merge\_leakage\_audit.json
- contract\_roll\_mapping\_report.json

78.3 数据问题为什么要写进知识文档

因为时序因子的很多伪 alpha，本质是数据脏、数据漏、数据错位，而不是研究本身真的找到了规律。团队如果没有这层认知，后面只会不断在错误数据上优化公式。

79. 实施任务拆解表

为了让这份文档最后真的能服务团队落地，这里补一张更偏工程和协作的任务拆解表。

| 模块                   | 核心任务                                        | 主要负责方       |
|----------------------|---------------------------------------------|-------------|
| 数据接入                 | 逐笔、盘口、session、合约映射接入                        | 数据平台组       |
| Sampling Engine      | time/volume/dollar/realized_vol bars        | 时序工程组       |
| Phase1 Engine        | 微观结构与底层因子计算                                 | 时序研究组 + 工程组 |
| Merge Layer          | 低频上下文 merge 与 PIT (Point-in-Time, 防穿越时间) 校验 | 数据平台组 + 评估组 |
| Phase2 Engine        | 高层时序因子、模式与状态对象计算                            | 时序研究组       |
| Evaluation Engine    | 预测、收益、执行、稳定性、样本外评估                          | 因子评估组       |
| Optimization Engine  | 高换手修复、deadband、family 管理                    | 优化工厂组       |
| Registry / Library   | 总库、储备库、标签、路由                                | 因子基建组       |
| Explorer / Dashboard | 时序详情页、图表、审计展示                               | 前端 + 平台组    |

79.1 为什么拆任务也要放进来

因为你这份文档最终不仅是知识补充，也是在帮助团队形成共同语言。把模块与责任初步拆开，能显著降低后续“这块谁来做”的沟通成本。

80. 当前版本对你目标的对应关系

你当前的目标有几个层次，这里我最后对照一下，方便后续继续扩时不跑偏。

第一，这份文档要成为“时序因子所有相关东西的总说明书”。

当前已经覆盖：

- 挖掘
- 处理

- 模式识别
- 状态识别
- 评估
- 入库
- 优化
- 入模前操作
- 执行仿真
- 数据质量
- 工程架构

第二，这份文档要足够全面，能发给组员吸收。

当前已经有：

- 分类体系
- 方法论
- 表格
- 公式
- 案例
- 反例
- 术语表
- 检查清单

第三，这份文档以后要成为回填主需求文档和 HTML 的母版。

当前已经有：

- 时序专属预处理模板
- 时序专属评估母版
- 时序专属标签与规则
- 时序专属执行与入库门槛

当前文档已趋于成熟，具备作为基建母版供下游抽取的条件。后续迭代将侧重于深化细节、扩充案例，而非基础框架的重建。